

Health Data Science: AI and Knowledge Translation

M. Ehsan. Karim

2026-05-12

Table of contents

The Project	13
What We Aim to Achieve	13
Dive into Our Modules	14
How Our Content is Presented	15
Open Copyright License	15
How to Cite	15
 I Course Planning	 16
1 Syllabus vs HDSx Inventory Completion Matrix	17
1.1 Extracted Schedule	17
1.2 Folder Inventory Snapshot	20
1.3 Completion Matrix	21
1.4 Key Mismatches	28
1.5 Proposed Normalized Structure	29
1.6 Migration Notes	31
1.7 Immediate Build Priorities	32
 II Week 0: Asynchronous Onboarding	 33
Week 0: Asynchronous Onboarding	34
Purpose	34
Required Student Tasks	34
TODO Before Release	34
Definition of Done	35
 III Week 1: Health Data, KT & Ethics	 36
Health Data & Ethics	37
Overview	37
Learning Objectives	37

The Open Data Landscape	38
How to Find, Evaluate, and Cite a Dataset	39
Privacy, Ethics, and Responsible GenAI Use	40
Indigenous Data Sovereignty (OCAP® & CARE) — Awareness	41
File Formats & Metadata (What You Need to Know Now)	42
Looking Ahead: The Lab Bench	42
Portfolio Prep (Ungraded)	43
Activity: Data Intake Card	44
Goal	44
Student Template	44
TODO Before Release	45
Reproducibility Check	45
 IV Week 2: Modern Workflows	 46
Modern Workflows	47
Overview	47
workflows1.qmd The Analogy: Public Computer and Cloud Drive	48
workflows2.qmd Getting Started: GitHub Account and Classroom	49
workflows3.qmd Launching Your Codespace	50
workflows4.qmd Tour of VS Code: The “Big Three” Zones	51
workflows5.qmd Concepts: IDE, Notebook, Script . .	52
workflows6.qmd Concepts: Literate Programming . .	53
workflows7.qmd Project Structure: Files and Folders	54
workflows8.qmd Tutorial: Your First Quarto Docu- ment	54
workflows9.qmd Tutorial: Stage, Commit, and Sync .	56
workflows10.qmd Stopping Your Codespace (Saving Credits)	58
workflows11.qmd Troubleshooting	59
workflows12.qmd Knowledge Check	60
workflows13.qmd Assignment 1: Your First Quarto Report	60
workflows14.qmd Quick Reference	61

2	The Analogy: Public Computer vs Cloud Drive	63
2.1	Repository (Cloud Drive)	63
2.2	Codespace (Public Computer)	64
3	Tour of the Repository	66
3.1	Step-by-step	66
3.2	Task	68
4	Opening Your First Codespace	69
4.1	Steps	69
4.2	The “Boot Up” Phase	71
5	Reopening a Codespace	72
5.1	Steps	72
6	VS Code Orientation	74
6.1	Explorer (left)	74
6.2	Editor (center)	76
6.3	Terminal (bottom)	76
7	Files vs Folders	78
7.1	The Golden Rule of Health Data	78
7.2	Files vs. Folders (The Mental Model)	78
7.3	Create a folder	79
7.4	Typical structure	79
8	The Magic Moment: Render Quarto	81
8.1	Why do we “Render”?	81
8.2	Step 1 — Open the file	81
8.3	Step 2 — Edit name (The YAML Header)	82
8.4	Step 3 — Render	82
8.5	If it fails (Troubleshooting)	82
9	Commit Your Work	84
9.1	Steps	84
9.2	Verify (Trust but Verify)	86
10	Troubleshooting: Rebuild Container	87
10.1	When to use this?	87
10.2	Steps	87

11 Stop Your Codespace	89
11.1 Correct shutdown	89
12 Final Checklist	91
12.1 Reflection	91
 V Week 3: R with AI	 92
Week 3: R with AI	93
Overview	93
r1.qmd Packages: Opening the Toolbox	94
r2.qmd Core Constructs: Boxes, Toasters, and the Assembly Line	94
r3.qmd Data Import and First Contact	96
r4.qmd Data Transformation: The Five dplyr Verbs	97
r5.qmd Control Flow: Auditing AI Decisions	99
r6.qmd Writing a Simple Function	100
r7.qmd Debugging: Reading the Red Text	101
r8.qmd AI Safety: The Audit Checklist	102
r9.qmd Exporting Clean Data	103
r10.qmd Literate Analysis in Quarto	103
r11.qmd Knowledge Check	104
r12.qmd Assignment: The “Mad Libs” Report	105
r13.qmd Quick Reference	106
 13 The Setup (The Cockpit)	 107
13.1 Verify Your AI Extensions	107
13.2 Set the AI Model to “0x”	108
13.3 Interface Review: Where does R actually live?	109
13.4 Interface Review: Working with .qmd and .Rmd files	110
 14 Packages (The Toolbox)	 111
 15 Basics: Boxes & Toasters	 112
15.1 Variables (The Box)	112
15.2 Functions (The Toaster)	112
 16 Control Flow	 113
16.1 The Pipe %>% (The Assembly Line)	113
16.2 If / Else (The Fork in the Road)	113

16.3 What happens	114
16.4 Loops (The Treadmill)	114
16.5 Example	114
16.6 What happens	115
16.7 Audit Check (Important)	115
17 The Loading Dock (Data Import)	116
18 The Detective (Debugging)	118
19 AI Safety: The Audit Checklist	119
20 The Paper Trail (Comments)	120
21 Assignment: The “Mad Libs” Report	122
22 Final Summary Checklist	123
 VI Week 4: Git & Collaboration	 124
Git & Collaboration	125
Overview	125
git1.qmd Concepts: The “Save Game” of Reproducible Science	126
git2.qmd Tutorial: Time Travel in Codespaces	126
git3.qmd Repo Hygiene: .gitignore and Relative Paths	127
git4.qmd Branching: Safe Experimentation	129
git5.qmd Pull Requests: The Peer Review Gateway	130
git6.qmd AI Auditing for Version Control	130
git7.qmd Setting Up Your Group Repository	132
git8.qmd Knowledge Check	132
git9.qmd Practice Exercise: The Full Workflow Drill	133
git10.qmd Quick Reference: Common Git Actions in VS Code	133
 Concepts	 135
 Tutorial	 138
 Repo Hygiene	 142

Branching	144
Pull Requests	146
AI Auditing	149
Setting Up Your Group Repository	150
Knowledge Check	151
Practice Exercise: Full Workflow	152
Quick Reference	153

VII Week 5: Polyglot Awareness & R Deepening 154

Python with AI	155
Overview	155
python1.qmd Concepts: Python vs R—When and Why?	156
python2.qmd Setup: Jupyter in Codespaces	157
python3.qmd Packages and Environment Hygiene . . .	157
python4.qmd Translation Traps: What Breaks When You Switch Languages	159
python5.qmd Tutorial: Loading and Viewing Data . .	160
python6.qmd Reading Tracebacks (Bottom-Up)	160
python7.qmd The AI Workflow: One Step, One Cell .	161
python8.qmd Interoperability: A Taste of <code>reticulate</code>	162
python9.qmd Creating a Python Script (<code>.py</code>)	163
python10.qmd Git Integration: Track Your Translations	164
python11.qmd Term Project: Milestone 1	165
python12.qmd Knowledge Check	165
python13.qmd Practice Exercise: The Bilingual Translation	166
python14.qmd Quick Reference	167
23 Tutorial 1: Python vs. R - When and Why?	168
23.1 Overview	168
24 Tutorial 2: Setting Up Your Python Environment	170
24.1 Creating Your First Notebook	170
24.2 The Variables Pane	171

25 Tutorial 3: Packages and Reproducibility	173
25.1 R <code>library()</code> vs. Python <code>import</code>	173
25.2 Managing <code>requirements.txt</code>	173
26 Tutorial 4: Common Translation Traps	175
26.1 Trap 1: The Zero-Index	175
26.2 Trap 2: Indentation	175
26.3 Trap 3: Assignment	176
27 Tutorial 5: Loading and Inspecting Health Data	177
27.1 Loading Data with Pandas	177
28 Reading Tracebacks	178
28.1 How to Read Errors	178
29 Tutorial 7: The ‘One Step, One Cell’ AI Workflow	179
29.1 Rule 1: One Step, One Cell	179
29.2 Rule 2: Parity Audit	180
30 Tutorial 8: Mixing R and Python”	181
30.1 Why Interoperability?	181
31 Tutorial 9: Creating Reusable Python Scripts	183
31.1 When to Use a Script	183
32 Tutorial 10: Tracking Your Progress with Git	184
32.1 Cleaning Your Notebooks	184
33 Tutorial 11: Term Project Milestone 1	185
33.1 Submission Checklist	185
34 Tutorial 12: Python Knowledge Check	186
35 Tutorial 13: The Bilingual Translation Exercise	187
35.1 Cleaning Your Notebooks	187
36 Tutorial 14: R to Python Cheat Sheet	188
 VIII Week 6: Data Visualization	 189
Visual Storytelling	190
Overview	190

TODO Before Release	190
Definition of Done	190
Activity: AI Visual Audit	192
Goal	192
TODO Before Release	192
Student Output	192
 IX Week 7: EDA, Table 1 & AI Auditing	 193
Exploring Data	194
Overview	194
TODO Before Release	194
Definition of Done	194
Activity: Table 1 and AI Audit	195
Goal	195
TODO Before Release	195
Student Output	195
 X Week 8: Dashboard Prototypes for KT	 196
Interactive Dashboards	197
Overview	197
TODO Before Release	197
Definition of Done	197
Starter Dashboard	199
Goal	199
TODO Before Release	199
Required Elements	199
 XI Week 9: Communication of Scientific Findings	 200
Dynamic Presentations	201
Overview	201
TODO Before Release	201
Definition of Done	201

Revealjs and Peer Feedback	202
Goal	202
TODO Before Release	202
Peer Feedback Prompts	202
 XII Week 10: Writing & Publishing Reports	 203
Reproducible Reporting	204
Overview	204
TODO Before Release	204
Definition of Done	204
 Report Template	 206
TODO Before Release	206
Suggested Sections	206
 XIII Week 11: Portfolio Surgery	 207
Advanced Topics	208
Overview	208
TODO Before Release	208
Definition of Done	208
 Reproducibility Stress Test	 209
Goal	209
TODO Before Release	209
Failure Log Template	209
 XIV Weeks 12-13: Project Presentations	 210
Week 12: Term Project Presentations, Part 1	211
Overview	211
TODO Before Release	211
Definition of Done	211
 Week 13: Term Project Presentations, Part 2	 212
Overview	212
TODO Before Release	212
Definition of Done	212

XV Assignments and Milestones	213
Assignments	214
Assignment Map	214
TODO	215
Assignment Template	216
Student-Facing Brief	216
Required Files	216
Reproducibility Contract	216
Grading Checklist	217
Project Milestones	218
TODO	219
Milestone Template	220
Deliverable	220
Required Evidence	220
Definition of Done	220
 XVIReference: Data Sources	 221
Data Source: NHIS	222
Data Source: NHANES	230
Data Source: BRFSS	239
Data Source: CCS	246
Data Source: CADS	252
Data Source: CTNS	257
Data Source: CSADS	259
Data Source: PHAC Health Infobase	263
Data Source: CCDSS	268
Data Source: GHO	273
Data Source: Open Portal	281

Data Source: BC Data Catalogue	285
XVI Assistant Playbook	290
Assistant Playbook	291
Operating Principles	291
Editing Rules	291
Reproducibility Expectations	292
Review Cadence	292
Definition-of-Done Checklists	293
Course Page Done	293
Assignment Done	293
Milestone Done	293
Reproducibility Done	294
AI Audit Done	294

The Project

Welcome to the course notes for **Health Data Science: AI and Knowledge Translation**.

This platform bridges the gap between traditional epidemiology and modern data science. Unlike standard textbooks that focus on a single language, this resource adopts a **“Polyglot”** approach—teaching R and Python side-by-side—and integrates **Artificial Intelligence** directly into the workflow.

Whether you are a health researcher looking to modernize your skills or a data scientist seeking to understand epidemiological rigor, you are in the right place.

Here, we offer:

- **“Zero to AI Co-Pilot” Training:** Learn to use Large Language Models (LLMs) to generate, debug, and translate code while critically auditing them for bias and hallucinations.
- **Real-World Data:** Guides on navigating “messy” open data sources such as NHANES and WHO, rather than sanitized “toy” datasets.
- **Knowledge Translation:** Step-by-step walkthroughs on building interactive dashboards (Shiny/Dash) and dynamic reports to communicate findings to policymakers.

What We Aim to Achieve

We are on a mission to:

- **Democratize Compute Access:** Eliminate hardware barriers by using cloud-native environments (Google Colab, Kaggle) that run on any device.

- **Bridge the “Polyglot” Gap:** Provide “Rosetta Stone” tutorials where concepts are implemented in both R and Python, allowing learners to be fluent in both.
- **Teach “AI-Resilience”:** Shift the focus from manual syntax generation to **rigorous code auditing**, security validation, and reproducibility.
- **Enable Professional Portfolios:** Help students transform static analyses into public-facing data science portfolios.

Dive into Our Modules

Embark on a journey through our structured modules, designed to take you from data literacy to advanced AI workflows.

Note

The tutorials are designed with a consistent structure to provide a cohesive learning experience. Here is what you can expect in each chapter:

1. **Overview:** A concise summary of learning objectives and topics.
2. **Concepts:** Core theoretical materials (slides, video lessons).
3. **Polyglot Tutorials:** In-depth walkthroughs with tabbed code blocks allowing you to toggle between **R** and **Python** solutions.
4. **AI Auditing:** Specific sections dedicated to verifying and “stress-testing” AI-generated code for the chapter’s topic.
5. **Knowledge Check:** Ungraded self-assessments to reinforce key concepts.
6. **Practice Exercises:** Hands-on tasks using real-world health data to build your portfolio.

How Our Content is Presented

All our resources are hosted on an easy-to-access GitHub page. The format? Engaging text, reproducible code, clear outputs, and crisp videos that distill complex topics.

This document is created using [Quarto](#), allowing us to weave analysis, code, and interpretation into a single reproducible document.

Open Copyright License

[CC-BY 4.0](#)



How to Cite

The BibTeX format can be downloaded from [here](#).

Part I

Course Planning

1 Syllabus vs HDSx Inventory Completion Matrix

Status: baseline audit created before the normalized folder migration. The normalized structure described below has now been applied to the source tree.

Source syllabus: `syllabus/SPPH-381H-Course-Outline-v4.qmd` and `syllabus/SPPH-381H-Course-Outline-v4.pdf`.

Inventory checked: `root` `.qmd` source files, `_quarto.yml`, `assets/`, `examples/`, `images/`, and `syllabus/`. Generated `docs/`, `.git/`, and `.Rproj.user/` were excluded from the inventory assessment.

1.1 Extracted Schedule

Week	Topic	Activity / deliverable
0 (Async)	Onboarding	GitHub account, orientation video, pre-course survey. Due before first class.
1 (Sept 10)	Health Data, KT & Ethics	Open data portals, Data Intake Card, provenance, stewardship, responsible AI. No graded submission.

Week	Topic	Activity / deliverable
2 (Sept 17)	Modern Workflows	Fork template, launch Codespace, edit/render Quarto, commit/sync. Assignment 1 due Mon 4 PM.
3 (Sept 24)	R with AI	R basics, tidyverse, functions, debugging with LLMs, <code>renv</code> , Quarto documentation. Assignment 2 due Mon 4 PM.
4 (Oct 1)	Git & Collaboration	Commits, branches, pull requests, repo hygiene, group repository setup. Assignment 3 and M0 due Mon 4 PM.
5 (Oct 8)	Polyglot Awareness & R Deepening	Python/pandas comparison, R-to-Python translation, AI parity checks, R practice. Assignment 4 due Mon 4 PM.

Week	Topic	Activity / deliverable
6 (Oct 15)	Data Visualization	AI-assisted ggplot2/plotly, visual clarity/bias/misrepresentation checks. Assignment 5 and M1 due Mon 4 PM.
7 (Oct 22)	EDA, Table 1 & AI Auditing	EDA, Table 1, planted-error AI audit, provenance/stewardship revision. Assignment 6 and M2 due Mon 4 PM.
8 (Oct 29)	Dashboard Prototypes for KT	Minimal dashboard-style KT product, Quarto/Shiny concepts, deployment pathway choice. Assignment 7 due Mon 4 PM.
9 (Nov 5)	Communication of Scientific Findings	Audience-specific KT, plain-language summaries, revealjs slides, citations, peer review. Assignment 8 due Mon 4 PM.

Week	Topic	Activity / deliverable
10 (Nov 12)	Writing & Publishing Reports	Quarto reports, citations, interactive elements, GitHub Pages, portfolio publishing. Assignment 9 and M3 due Mon Nov 16, 4 PM.
11 (Nov 19)	Portfolio Surgery & Optional Advanced Deployment	Fresh Codespace reproducibility test, path/dependency/README fixes, optional deployment/Python pathway. Assignment 10 and M4 due Mon Nov 23, 4 PM.
12 (Nov 26)	Term Project Presentations, Part 1	M5 presentations, live Q&A, peer review. Slides due before class.
13 (Dec 3)	Term Project Presentations, Part 2	M5 presentations, live Q&A, peer review. Slides due before class.
Final	Final Portfolio	Final portfolio due Dec 11, Fri 4 PM.

1.2 Folder Inventory Snapshot

Area	Current contents	Notes
Root Quarto source	<code>index.qmd</code> , <code>_quarto.yml</code> , 11 module part files, many numbered child <code>.qmd</code> files	Source is mostly flat, making week/module ownership hard to scan.
syllabus/	PDF and QMD syllabus source	Good source of truth for schedule and assessment logic.
assets/data/	NHANES_Demo_Converter	Only one CSV data asset found under shared assets.
assets/images/	Image subfolders for data1-data12, workflow*, git*, python*, r*, plus week3	Assets are topic-fragment based, not normalized by course week.
examples/	bmi39.R, demo5.R, NHANES .rds files	Useful for R/EDA examples, but not tied to weeks or assignments.
images/	License/support images	Small global image folder separate from <code>assets/images</code> .
docs/	Rendered Quarto output	Treat as generated output, not source curriculum inventory.

1.3 Completion Matrix

Status key: **Ready** = source is substantial and surfaced in navigation; **Partial** = source exists but has gaps, placeholders, or navigation mismatch; **Missing** = no clear source content found.

Week	Syllabus need	Current matching inventory	Status	Gap / action
0	Async onboarding module	Syllabus-only; no dedicated onboarding .qmd found	Missing	Add <code>week00-onboarding/</code> or an onboarding chapter with GitHub setup, orientation link placeholder, and survey instructions.
1	Health data, ethics, KT, Data Intake Card	<code>data.qmd</code> ; <code>data1.qmd</code> ; extra <code>data2.qmd</code> - <code>data12.qmd</code> ; <code>assets/images/data1-data12</code>	Partial	Strong content exists, but only <code>data1.qmd</code> is in <code>_quarto.yml</code> ; decide whether data-source pages are Week 1 references or a separate data appendix.

Week	Syllabus need	Current matching inventory	Status	Gap / action
2	Codespaces, Quarto basics, reproducible structure, commits	<code>workflows.qmd</code> ; <code>workflows1.qmd</code> - <code>workflows11.qmd</code> ; <code>assets/images/workflows</code>	Ready	Good coverage. Normalize the naming and keep the parent/child relationship explicit.
3	R with AI, tidyverse, functions, debugging, <code>renv</code>	<code>r.qmd</code> ; <code>r1.qmd</code> - <code>r10.qmd</code> ; <code>assets/images/r*</code> ; <code>examples/NHANES</code> files	Ready	Add explicit <code>renv</code> activity/checkpoint if not already embedded in assignment materials.
4	Git collaboration, PRs, repo hygiene, group setup	<code>git.qmd</code> ; <code>git1.qmd</code> - <code>git10.qmd</code> ; <code>assets/images/git*</code>	Ready	Looks aligned; M0 group setup should be made visibly tied to Week 4.

Week	Syllabus need	Current matching inventory	Status	Gap / action
5	Python/pandas awareness and R deepening	<code>python.qmd</code> ; Partial <code>python1.qmd</code> - <code>python14.qmd</code> ; <code>assets/images/python*</code>		<code>python.qmd</code> is substan- tial, but <code>_quarto.yml</code> only exposes <code>python1.qmd</code> ; surface or consoli- date the rest. R- deepening assign- ment materials are not obvious.
6	Data visu- alization with gg- plot2/plotly and AI visual audit	<code>viz.qmd</code> ; <code>viz1.qmd</code>	Missing	Current files are placeholder- ers. Add real tuto- rial/activity content and assets.

Week	Syllabus need	Current matching inventory	Status	Gap / action
7	EDA, Table 1, planted-error AI audit, prove-nance update	<code>eda.qmd</code> ; <code>eda1.qmd</code> ; NHANES example data/scripts	Missing	Current EDA files are placeholders. Build the activity around the existing NHANES examples and add audit materials.
8	Dashboard-style KT prototype, Quarto/Shiny concepts	<code>dashboards.qmd</code> ; <code>dashboards1.qmd</code>	Missing	Current files are placeholders. Need a minimal dashboard path, starter template, and testing instructions.

Week	Syllabus need	Current matching inventory	Status	Gap / action
9	Scientific communi- cation, revealjs slides, peer feedback	<code>comms.qmd</code> ; <code>comms1.qmd</code>	Missing	Current files are placeholders. Add revealjs tutorial, plain- language summary activity, and peer review template.
10	Reproduciblereports.qmd reporting, publish- ing, citations, GitHub Pages	<code>reports1.qmd</code> ; <code>references.bib</code> empty	Missing	Current files are placeholders. Need report template, citation workflow, bibliogra- phy example, and publishing checklist.

Week	Syllabus need	Current matching inventory	Status	Gap / action
11	Portfolio reproducibility stress test, optional deployment	advanced.qmd advanced1.qmd ; maybe reports/dashboards overlap	Missing	Current advanced files are placeholders. Create a portfolio surgery checklist and fresh Codespace test protocol.
12	Presentations Part 1	No dedicated presentation week materials beyond comms* placeholders	Missing	Add presentation rubric/checklist and peer feedback form.
13	Presentations Part 2	No dedicated presentation week materials beyond comms* placeholders	Missing	Add repeat presentation logistics or combine with Week 12 materials.

Week	Syllabus need	Current matching inventory	Status	Gap / action
Final	Final portfolio	Syllabus has reproducibility contract; no dedicated portfolio submission package found	Partial	Convert the syllabus reproducibility contract into a student-facing final portfolio check-list/template.

1.4 Key Mismatches

1. The source tree is organized as topic families (`data`, `workflows`, `r`, `git`, `python`, etc.), while the syllabus is organized as weeks.
2. The first five technical weeks are substantially developed; Weeks 6-13 are mostly placeholders.
3. Several developed files are not surfaced in `_quarto.yml`: `data2.qmd`-`data12.qmd` and most `python*.qmd` files.
4. Parent files such as `data.qmd`, `workflows.qmd`, `r.qmd`, `git.qmd`, and `python.qmd` contain substantial chapter content but are used as Quarto `part:` pages, while many child pages are short fragments. This creates duplication and discoverability friction.
5. Assignment and milestone deliverables are present in the syllabus, but there is no obvious normalized assignment template folder.

1.5 Proposed Normalized Structure

Recommended target: keep the Quarto book, but normalize source around syllabus weeks and move reusable references into appendices.

```
1 /
2 _quarto.yml
3 index.qmd
4 syllabus/
5   SPPH-381H-Course-Outline-v4.qmd
6   SPPH-381H-Course-Outline-v4.pdf
7   syllabus-inventory-completion-matrix.md
8 weeks/
9   week00-onboarding/
10     index.qmd
11     assets/
12   week01-health-data-ethics/
13     index.qmd
14     activity-data-intake-card.qmd
15     assets/
16   week02-modern-workflows/
17     index.qmd
18     tutorials/
19     assets/
20   week03-r-with-ai/
21     index.qmd
22     tutorials/
23     assets/
24   week04-git-collaboration/
25     index.qmd
26     tutorials/
27     assets/
28   week05-polyglot-r-deepening/
29     index.qmd
30     tutorials/
31     assets/
32   week06-visualization/
33     index.qmd
34     activity-ai-visual-audit.qmd
```

```
35     assets/
36 week07-eda-ai-audit/
37     index.qmd
38     activity-table1-audit.qmd
39     assets/
40 week08-dashboard-kt/
41     index.qmd
42     starter-dashboard/
43     assets/
44 week09-science-communication/
45     index.qmd
46     peer-feedback-template.qmd
47     assets/
48 week10-reporting-publishing/
49     index.qmd
50     report-template.qmd
51     assets/
52 week11-portfolio-surgery/
53     index.qmd
54     reproducibility-test.qmd
55     assets/
56 week12-presentations/
57     index.qmd
58     rubric.qmd
59 week13-presentations/
60     index.qmd
61 assignments/
62     assignment01-workflows/
63     assignment02-r-with-ai/
64     assignment03-git/
65     assignment04-polyglot/
66     assignment05-visualization/
67     assignment06-eda-ai-audit/
68     assignment07-dashboard/
69     assignment08-communication/
70     assignment09-reporting/
71     assignment10-portfolio-surgery/
72 milestones/
73     m0-group-formation/
74     m1-proposal/
```

```

75     m2-preliminary-analysis/
76     m3-project-update/
77     m4-peer-review/
78     m5-presentation/
79     final-portfolio/
80 references/
81     data-sources/
82         nhis.qmd
83         nhanes.qmd
84         brfss.qmd
85         ccs.qmd
86         cads.qmd
87         ctns.qmd
88         csads.qmd
89         phac-infobase.qmd
90         ccdss.qmd
91         gho.qmd
92         open-portal.qmd
93         bc-data-catalogue.qmd
94     cheatsheets/
95     troubleshooting/
96 examples/
97     r/
98     python/
99     data/
100 assets/
101     shared/

```

1.6 Migration Notes

Current source	Proposed home
data.qmd	weeks/week01-health-data-ethics/index.qmd
data1.qmd-data12.qmd and assets/images/data*	references/data-sources/ plus matching assets
workflows.qmd, workflows1.qmd- workflows11.qmd	weeks/week02-modern-workflows/
r.qmd, r1.qmd-r10.qmd	weeks/week03-r-with-ai/

Current source	Proposed home
<code>git.qmd</code> , <code>git1.qmd-git10.qmd</code>	<code>weeks/week04-git-collaboration/</code>
<code>python.qmd</code> , <code>python1.qmd-python14.qmd</code>	<code>weeks/week05-polyglot-r-deepening/</code>
<code>viz.qmd</code> , <code>eda.qmd</code> , <code>dashboards.qmd</code> , <code>comms.qmd</code> , <code>reports.qmd</code> , <code>advanced.qmd</code>	Expand into Weeks 6-11 modules
<code>examples/bmi39.R</code> , <code>examples/demo5.R</code> , <code>examples/data/*.rds</code>	Keep under <code>examples/</code> , but add week references in activities

1.7 Immediate Build Priorities

1. Decide whether the Quarto book should be navigated by week or by topic. For syllabus alignment, week-based navigation is the cleaner primary structure.
2. Surface existing developed content currently missing from `_quarto.yml`, especially `data2.qmd-data12.qmd` and `python2.qmd-python14.qmd`, or move it into a reference appendix.
3. Build Weeks 6-11, because those are the largest content gaps.
4. Add assignment and milestone folders so deliverables are first-class artifacts rather than only syllabus text.
5. Create a final portfolio checklist from the syllabus reproducibility contract.

Part II

Week 0: Asynchronous Onboarding

Week 0: Asynchronous Onboarding

Purpose

Students complete the basic account and orientation work before the first class so Week 1 can focus on health data, ethics, and course expectations.

Required Student Tasks

- Create a GitHub account using a professional username.
- Watch the course workflow orientation video posted on Canvas.
- Complete the pre-course survey.
- Confirm access to Canvas and the course GitHub entry point.

TODO Before Release

Content TODO

- Add the final Canvas orientation video link.
- Add the pre-course survey link or instructions.
- Add a screenshot walkthrough for creating a GitHub account with a professional username.
- Add a brief troubleshooting note for students who already have a GitHub account.

Definition of Done

- The page gives students one clear path to complete onboarding without installing local software.
- All external links have been checked in a fresh browser session.
- The GitHub username guidance is privacy-aware and professional.
- The page states what students should bring to Week 1.

Part III

Week 1: Health Data, KT & Ethics

Health Data & Ethics

Overview

This chapter sets the stage by defining the core philosophy of the course: data science does not end with a p-value, but with Knowledge Translation (KT). Students are introduced to the “Last Mile” of research—communicating findings effectively—and the ethical landscape of modern health data, including Indigenous Data Sovereignty (OCAP® principles) and the risks of AI hallucinations. You will explore open data sources like NHANES and establish the ground rules for privacy and security before touching a single line of code.

Learning Objectives

- Explore the landscape of **open health data** (e.g., NHANES, WHO, World Bank, BC Data Catalogue) and learn where high-quality public datasets live.
- Practice finding a dataset, skimming its description/metadata, and recording basic provenance.
- Review **privacy and ethics** for public data and **responsible GenAI use**.
- Learn the meaning and importance of **Indigenous Data Sovereignty** (specifically OCAP® for First Nations) and CARE principles.
- Identify common **file formats** and **metadata** to support reproducibility.

Note: Week 1 is **light on code** and **heavy on awareness**. There is **no graded assignment submission** this week. Instead, you will complete

a “Data Intake Card” to prepare for your future portfolio.

The Open Data Landscape

Open data can accelerate health research—if we understand **what** we’re downloading and **how** to use it responsibly. This week, we locate credible sources and map what “good” looks like.

Typical Sources (Examples)

- **National health surveys** (e.g., *NHANES* style): De-identified microdata with documentation and code-books.
- **International agencies** (e.g., *WHO Global Health Observatory* style): Indicators aggregated by country/region over time.
- **Government open data portals** (e.g., *BC Data Catalogue* style): Administrative or program data with varying schemas.
- **Multilateral / development banks** (e.g., *World Bank Data* style): Socio-economic indicators for cross-country comparisons.

Instructor Note: In class, we will share specific links via Canvas. The goal today is *pattern recognition*—you should be able to recognize a trustworthy portal, find the **About/Documentation**, and locate a **download** link.

What to Look For on a Portal

1. A human-readable **About** page for the dataset.
2. Clear **licensing/terms of use**.
3. **Metadata** (definitions of variables, units, time coverage).

4. A **download** button (CSV/JSON, etc.) and/or API.
5. A **citation** suggestion.

Mini-Activity (10 minutes)

Pick any reputable portal (from Canvas links). Find a dataset you personally care about (e.g., smoking prevalence, hypertension, BMI). Write down:

- Dataset name + link (URL)
 - Who publishes it
 - Time coverage
 - One variable you understand immediately
 - One variable whose meaning is unclear
-

How to Find, Evaluate, and Cite a Dataset

A. Finding & Quick Evaluation

When you land on a dataset page, scan for:

- **Scope:** Population, geography, years.
- **Data Level:** Individual/micro vs. aggregated indicators.
- **Update Cadence:** Last updated date.
- **Documentation:** Codebook, methodology PDF.

If any of these are missing, reconsider using it or document the gap.

B. The Data Intake Card (Copy this block into your notes)

Use this template every time you bring a dataset into your project.

```

1 dataset_name: ""
2 publisher_org: ""
3 permalink_url: ""
4 publisher_contact_or_about: ""
5 license_terms: ""          # e.g., CC BY 4.0; "public domain"; "restricted"
6 geography: ""              # e.g., Canada, global, BC, health authority
7 time_coverage: ""          # e.g., 2005-2024, point-in-time
8 data_level: ""             # microdata, aggregate indicators
9 variables_of_interest:
10   - name: ""
11     definition: ""
12     unit: ""
13     caveats: ""
14 potential_risks:
15   - privacy: "low/med/high (why?)"
16   - misinterpretation: "what could be misunderstood?"
17 indigenous_data_relevance: "yes/no/unknown (explain if yes)"
18 how_to_cite: ""            # copy the suggested citation if provided
19 date_accessed: "YYYY-MM-DD"

```

Privacy, Ethics, and Responsible GenAI Use

Even open data can be used irresponsibly. This course expects **ethical use**: - **No Personal Identifiers**: We only use public, de-identified data in Week 1. - **Attribution**: Acknowledge sources and licenses. - **Context**: Avoid overstating what aggregate indicators can support.

Using GenAI Responsibly This Week

GenAI (e.g., a coding co-pilot or chat assistant) may be used to:

- **Summarize** a dataset description into plain language.
- **Draft** a first pass of a data dictionary from a codebook.
- **Suggest** variable interpretations you will verify.

You must:

- **Disclose** in your notes when AI is used (what prompt, what output).
- **Verify** with the official documentation.
- **Accept Accountability** for errors (AI is not a source of truth).

We will return to GenAI auditing later in the term; this week is limited to **summaries** and **notes**—**no data uploads** to AI tools.

Indigenous Data Sovereignty (OCAP® & CARE) — Awareness

Before downloading or using data about Indigenous Peoples, be aware that **Indigenous Data Sovereignty** affirms rights and interests in data that pertain to Indigenous Peoples. Two widely cited frameworks you should **know exist**:

- **OCAP® Principles:** specifically referring to **First Nations** ownership, control, access, and possession of data.
- **CARE Principles:** Collective benefit, Authority to control, Responsibility, Ethics.

What this means for Week 1: - If your chosen dataset includes Indigenous content, flag it on your **Data Intake Card**.
- Read the publisher's guidance and respect any use conditions.
- When in doubt, **ask the instructor** before using or publishing analyses.

We acknowledge these frameworks now and revisit concrete applications later in the course.

File Formats & Metadata (What You Need to Know Now)

Common formats you'll meet:

- **CSV:** Rows/columns; simplest to start.
- **JSON:** Nested; common for APIs.
- **XLSX:** Spreadsheets; watch for merged cells and headers.
- **GeoJSON / Shapefile:** Spatial data; used for maps (later weeks).

Best practices this week:

- **Download** the smallest useful file (e.g., one year) to explore the structure. - Keep the raw file **unchanged** in a **data/** folder (we will create this next week).
- Create a short **README section** in your notes describing:
 - Where you got it.
 - What the columns mean (link to codebook).
 - Any obvious pitfalls (missing values, units).

Looking Ahead: The Lab Bench

Next week, we leave the browser and enter the lab. We will introduce **GitHub Codespaces**, your cloud-based “Public Computer” where we will load this data.

To prepare for next week:

1. Ensure you have a **GitHub Account**.
2. Ensure you can access the course repository link on Canvas.

Portfolio Prep (Ungraded)

Your Week 1 task (Complete/Incomplete):

1. Choose **one** dataset page from a reputable portal (links on Canvas).
2. Fill out the **Data Intake Card** (copy/paste the YAML block and complete it).
3. Save this card; you will use this dataset for your first real project in Week 5.

I confirm I understand:

- ☐ The difference between open **aggregate indicators** and microdata.
- ☐ How to find documentation/metadata for a dataset.
- ☐ My responsibility when using GenAI (disclose, verify, be accountable).
- ☐ That datasets involving Indigenous Peoples require extra care and may need instructor guidance before use.

Activity: Data Intake Card

Goal

Students practice evaluating whether a health dataset is usable, ethical, documented, and appropriate for reproducible analysis.

Student Template

```
1 dataset_name: ""
2 publisher_org: ""
3 permalink_url: ""
4 license_terms: ""
5 geography: ""
6 time_coverage: ""
7 data_level: ""
8 variables_of_interest:
9   - name: ""
10     definition: ""
11 known_limitations:
12   - ""
13 stewardship_notes: ""
14 ai_use_note: ""
```

TODO Before Release

Content TODO

- Add 2-3 approved starter portals from Canvas.
- Add one completed sample Data Intake Card.
- Add a short rubric for provenance, licensing, variable clarity, and stewardship.
- Decide whether students submit this on Canvas, GitHub, or both.

Reproducibility Check

- The dataset link is stable or includes an access date.
- The publisher and license are named.
- Variable definitions come from source documentation, not AI guesses.
- The student records what they asked AI and how they verified the answer.

Part IV

Week 2: Modern Workflows

Modern Workflows

Overview

Summary: This week you set up the computational environment you will use for the rest of the course. By the end, the technology should fade into the background so you can focus on health data science, not installation errors. We adopt a **“Zero to AI Co-Pilot”** approach: you do not need to install R, Python, or Git on your personal laptop. Everything runs in a cloud-native environment called GitHub Codespaces.

Learning Objectives:

- Understand the “public computer vs cloud drive” analogy for cloud computing.
- Create a GitHub account, join the course GitHub Classroom, and accept the starter assignment.
- Launch a Codespace and identify the three key areas of the VS Code interface.
- Distinguish between an IDE, a notebook, and a script, and explain why this course uses Quarto notebooks.
- Explain the concept of literate programming and why it matters for reproducible health research.
- Edit and render a Quarto (`.qmd`) document: YAML header, Markdown text, and code chunks.
- Maintain a reproducible project structure using the `data/` and `output/` folder convention.
- Stage, commit, and sync changes using the VS Code Source Control panel.
- Stop your Codespace to conserve your free monthly hours.

workflows1.qmd The Analogy: Public Computer and Cloud Drive

To understand how we work in this course, forget about “servers” and “clients.” Imagine you are working in a university computer lab.

The Repository (Your Cloud Drive)

Think of **GitHub** as your personal **Google Drive** or **Dropbox**.

- It is where your files live permanently.
- If your laptop crashes, the files are still here.
- **Key concept:** This is your “Master Copy.”

The Codespace (The Public Computer)

Think of a **Codespace** as a **public computer in the library**.

- It is a temporary machine you borrow to do your work.
- It comes pre-installed with all the software you need (R, Python, Quarto), so you don’t have to install anything on your own laptop.
- **Crucial difference:** Just like a public library computer, it can be wiped clean. If you close the browser without sending your work back to your Cloud Drive, your changes may be lost.

The Workflow

1. **Log in:** Launch your Codespace (open the public computer).
2. **Work:** Edit files using the computer’s software.
3. **Save to the cloud:** Commit and push your work back to your Cloud Drive (GitHub).
4. **Log off:** Stop the Codespace to save your free hours.

This cycle — launch, work, commit, stop — is the rhythm of every week in this course.

workflows2.qmd Getting Started: GitHub Account and Classroom

Step 1: Create a GitHub Account

If you do not already have one:

1. Go to github.com and click **Sign up**.
2. Use your UBC email address (this helps with student benefits).
3. Choose a professional username — this will appear on your public portfolio later in the course.

Tip

Portfolio thinking: Your GitHub profile will become part of your professional presence. Pick a username you would be comfortable putting on a CV (e.g., `jchen-epi` rather than `xXcoolcoder99Xx`).

Step 2: Join the Course GitHub Classroom

1. Open the **GitHub Classroom** invitation link posted on Canvas.
2. Accept the invitation. This connects your GitHub account to the course roster.
3. Accept the **starter assignment**. GitHub Classroom will automatically create a personal repository for you, preloaded with the files you need for this week.

Step 3: Confirm Your Repository

Navigate to the repository that was just created. You should see:

- A file list including `practice_report.qmd`, `README.md`, and a `data/` folder.
- A formatted README below the file list explaining the project.

If you see these, you are ready to launch your Codespace.

workflows3.qmd Launching Your Codespace

1. On your repository page, click the green `<>` **Code** button.
2. Select the **Codespaces** tab.
3. Click **Create codespace on main**.

The first launch takes 2–3 minutes while the environment builds. Subsequent launches are faster.

Note

Reopening an existing Codespace: After you stop or close your Codespace, do **not** create a new one each time. Go to github.com/codespaces, find your existing Codespace, and click its name to restart it. Creating multiple Codespaces wastes your hours and causes confusion.

workflows4.qmd Tour of VS Code: The “Big Three” Zones

The VS Code interface can look intimidating — it is designed for professional software engineers. You only need to know three areas. Safely ignore 90% of the buttons.

1. The Explorer (Left Sidebar)

Your **file cabinet**. It shows folders and files in your project. You will use this to open `.qmd` files, navigate the `data/` folder, and create new files.

2. The Editor (Center)

Your **work area**. This is where you read and write code and text. You can have multiple files open in tabs.

3. The Terminal (Bottom Panel)

The **engine room** where R and Python actually run. You will not type commands here often — most of your work happens in the Editor — but this is where output and error messages appear.

Tip

Lost a panel? Use the top menu: **View → Explorer** or **View → Terminal** to bring them back. You can also press `Ctrl+`` (backtick) to toggle the Terminal.

workflows5.qmd **Concepts: IDE, Notebook, Script**

The syllabus mentions three ways to write code. Understanding the difference helps you choose the right tool throughout the course.

IDE (Integrated Development Environment)

VS Code is an **IDE** — a text editor with superpowers: file browsing, syntax highlighting, built-in terminal, extensions, and version control. It is the container that holds everything else.

Script

A plain text file containing only code (e.g., `.R` or `.py`). Runs top-to-bottom. Best for reusable utilities and automation. You will write your first script in Week 5.

Notebook

A document that **interleaves code and narrative**. You run code in chunks and see results inline. Examples include Jupyter notebooks (`.ipynb`, Week 5) and Quarto documents (`.qmd`, which you use starting today).

This course primarily uses Quarto notebooks because they combine the strengths of notebooks (interactivity, narrative) with the ability to produce publication-quality reports, slides, and websites — all from a single file.

workflows6.qmd Concepts: Literate Programming

The Big Idea

Traditional workflows separate code from writing: you run an analysis in one program, copy results into a Word document, and hope nothing changes. **Literate programming** weaves code, results, and interpretation into a single document. When the data changes, you re-render and the entire report updates automatically.

This is not just convenient — it is a **scientific obligation**. In health research, reviewers and regulators need to see exactly how you went from raw data to conclusions. A literate document is a transparent, auditable record of your reasoning.

How Quarto Implements This

A `.qmd` file has three ingredients:

1. **YAML header** (metadata at the top, between `---` lines)
2. **Markdown text** (human-readable prose)
3. **Code chunks** (executable R or Python code in fenced blocks)

When you **Render**, Quarto runs every code chunk, captures the output (tables, figures, numbers), and weaves everything into a finished HTML, PDF, or Word document.

You will use literate programming in every subsequent week of this course: your assignments, your EDA (Week 7), your reports (Week 10), and your final portfolio are all Quarto documents.

workflows7.qmd Project Structure: Files and Folders

A reproducible project needs a consistent folder structure. This course uses the following convention:

```
my-project/  
  data/           ← Raw data. Never edit files here directly.  
  output/         ← Cleaned data, figures, and tables generated by your code.  
  practice_report.qmd ← Your Quarto notebook.  
  README.md       ← Explains what this project is.  
  .gitignore      ← Tells Git which files to ignore (more in Week 4).
```

Rules

- **data/ is read-only.** Import from it; never save into it.
- **output/ is generated.** Your code writes cleaned data and figures here. In Week 3, you will export `output/clean_data.csv`.
- **Use relative paths.** Always refer to files relative to the project root (e.g., `"data/patients.csv"`), never absolute paths (e.g., `"/home/user/Documents/patients.csv"`). This ensures your code runs on any machine.

Activity: In the Explorer pane, click the **New Folder** icon and create the `output/` folder if it does not already exist.

workflows8.qmd Tutorial: Your First Quarto Document

Step 1: Open the Starter File

In the Explorer, click `practice_report.qmd`. It opens in the Editor.

Step 2: Understand the YAML Header

At the very top, between `---` lines, you will see something like:

```
1 ---
2 title: "My First Report"
3 author: "Your Name"
4 format: html
5 ---
```

Change `author:` to your actual name. The YAML header controls metadata and output format.

Step 3: Read the Markdown

Below the YAML, you will see plain text with formatting:

- `#` Heading creates a section title.
- `**bold**` makes text **bold**.
- `- item` creates a bullet point.

This is **Markdown** — a lightweight way to format text without a word processor.

Step 4: Read the Code Chunks

Look for grey blocks that start with ````{r}` and end with `````. These are **code chunks**. Everything inside them is R code that will execute when you render.

```
1 ```{r}
2 # This is a code chunk
3 2 + 2
4 ```
```

Step 5: Edit a Code Chunk

Find a code chunk in the starter file. Make a small change — for example, change a number or add a comment. This is your first edit.

Step 6: Render

Click the **Render** button (blue arrow icon at the top of the editor). A preview window appears showing your formatted report with code output embedded.



Tip

What just happened: Quarto ran every code chunk, captured the results, combined them with your Markdown text, and produced an HTML document. This is literate programming in action.

workflows9.qmd Tutorial: Stage, Commit, and Sync

Your work currently exists only on the public computer (Codespace). To save it permanently to your Cloud Drive (GitHub), you need to **commit**.

Step 1: Open Source Control

Click the **Source Control** icon on the left sidebar (it looks like a branching graph). You will see a list of files that have changed since your last commit.

Step 2: Stage Your Changes

Hover over a changed file name. Click the + icon that appears. This **stages** the file — it tells Git “I want to include this change in my next snapshot.”

Note

Staging vs committing: Staging selects *which* changes to include. Committing takes the snapshot. Think of staging as putting items on the conveyor belt; committing is pressing the “scan” button at checkout.

Step 3: Write a Commit Message

In the text box at the top of the Source Control panel, type a short, descriptive message:

Updated author name and edited first code chunk
in practice report

A good commit message answers: *what did I change and why?*

Step 4: Commit

Click the **Commit** button (checkmark). Your changes are now saved as a snapshot in your local history.

Step 5: Sync with GitHub

Click **Sync Changes** (in the Source Control panel or the status bar at the bottom). This pushes your commit to GitHub — your Cloud Drive.

Verify: Go to your repository on github.com and confirm that your changes appear. You should see your commit message next to the updated file.

 Warning

If you skip the **Sync step**, your commit exists only on the Codespace. If the Codespace is deleted, the commit goes with it. Always Sync.

workflows10.qmd Stopping Your Codespace (Saving Credits)

GitHub Codespaces gives you a monthly allowance of free hours (approximately 60 core-hours). **If you leave the browser tab open, the meter keeps running**, even if you are not actively working.

The Right Way to Stop

1. Go to github.com/codespaces.
2. Find your active Codespace.
3. Click the **three dots (...)** → **Stop codespace**.

What Happens If You Just Close the Tab?

The Codespace runs for another 30 minutes before it times out automatically. That is 30 minutes of wasted credits every time.

 Warning

Policy: It is your responsibility to manage your compute hours. If you run out before the month resets, you will need to wait or pay out of pocket. Get in the habit of stopping your Codespace at the end of every session.

workflows11.qmd Troubleshooting

Sometimes things break. Here is a graduated approach — try the simplest fix first.

Level 1: Reload the Window

Press F1 (or Ctrl+Shift+P / Cmd+Shift+P) to open the **Command Palette**. Type `Reload Window` and select it. This restarts VS Code without rebuilding the entire environment.

Level 2: Restart the R or Python Session

If R or Python is frozen, open the Command Palette → type `R: Restart R Session` (or the equivalent for Python). This clears the session without touching your files.

Level 3: Rebuild the Container (Nuclear Option)

If nothing else works:

1. Open the **Command Palette**.
2. Type `Rebuild Container`.
3. Select **Codespaces: Rebuild Container**.

This shuts down and rebuilds the entire environment from scratch. It solves nearly all technical glitches but takes a few minutes.

Warning

Before rebuilding: Make sure you have committed and synced your work. Uncommitted changes may survive a rebuild, but there is no guarantee.

workflows12.qmd Knowledge Check

1. In the analogy, what is the “Cloud Drive” and what is the “Public Computer”?
 2. Why don’t you need to install R or Python on your laptop for this course?
 3. Name the “Big Three” zones of the VS Code interface and what each is for.
 4. What is the difference between an IDE, a notebook, and a script?
 5. What is literate programming, and why does it matter for health research?
 6. What are the three ingredients of a .qmd file?
 7. What is the difference between **staging** and **committing**?
 8. Why must you click **Sync Changes** after committing?
 9. How do you properly stop a Codespace to save your free hours?
-

workflows13.qmd Assignment 1: Your First Quarto Report

Due Friday 4 pm. Submit via Canvas.

Task

Edit and render the starter `practice_report.qmd`, demonstrating that you can navigate your Codespace, edit Quarto, and commit your work.

Step-by-Step Workflow

1. **Launch** your Codespace from the GitHub Classroom repository.
2. **Edit the YAML:** Change `author:` to your name.

3. **Add a Markdown section:** Below the existing content, add a new heading (**## My Notes**) and write 2-3 sentences about what you learned this week. Use at least one Markdown feature (bold, italics, or a bullet list).
4. **Edit a code chunk:** Modify an existing code chunk or add a new one. For example:

```

1  ```{r}
2  # My first R calculation
3  2 + 2
4  ```

```

5. **Render** the document. Confirm the output looks correct.
6. **Commit and Sync:**
 - Open Source Control.
 - Stage your changed files (+).
 - Write a commit message: e.g., "Complete Week 2 assignment: edit YAML, add notes, render report".
 - Click **Commit**, then **Sync Changes**.
7. **Stop** your Codespace.
8. **Verify** on github.com that your commit appears in the repository.
9. **Submit** the repository link on Canvas.

workflows14.qmd Quick Reference

Action	How
Launch Codespace	Repository → green <> Code → Codespaces → Create codespace on main
Reopen Codespace	github.com/codespaces → click Codespace name
Stop Codespace	github.com/codespaces → ... → Stop codespace

Action	How
Open Explorer	View → Explorer or click file icon (left sidebar)
Open Terminal	View → Terminal or Ctrl+`
Render Quarto	Click Render button (blue arrow) at top of editor
Stage a file	Source Control → hover file → click +
Commit	Type message → click
Sync to GitHub	Click Sync Changes
Reload window	Command Palette (F1) → Reload Window
Rebuild container	Command Palette (F1) → Codespaces: Rebuild Container

2 The Analogy: Public Computer vs Cloud Drive

To understand the workflow, imagine working in a university computer lab.

2.1 Repository (Cloud Drive)

Important Distinction: While GitHub is your “Cloud Drive,” it does **NOT** work like Google Drive.

- **Google Drive:** Auto-saves every keystroke.
- **GitHub:** Works like **Email**. You can write a draft (save in Codespace), but the cloud never sees it until you hit “Send” (Commit & Push).
- Stores your files permanently

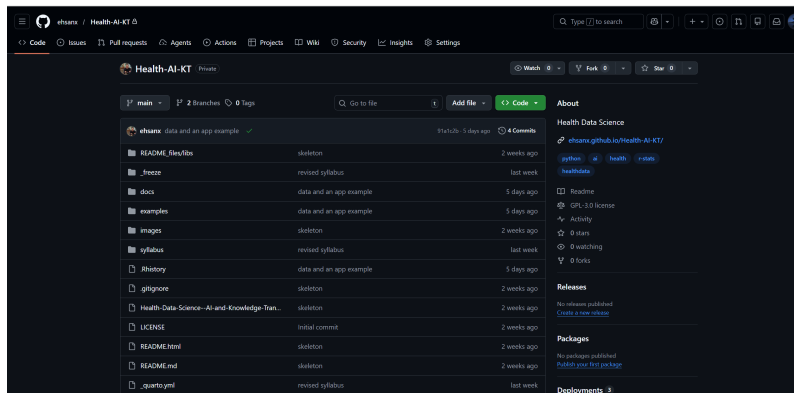


Figure 2.1: The code will be stored and accessible on GitHub as long as the company exists. Given that it’s owned by a large corporation like Microsoft, it will likely last for a long time.

- Safe if your computer crashes
- This is your **master copy**

2.2 Codespace (Public Computer)

Codespaces is like a library computer:

- **Temporary machine** It is a disposable environment. If it breaks, we just delete it and get a new one.

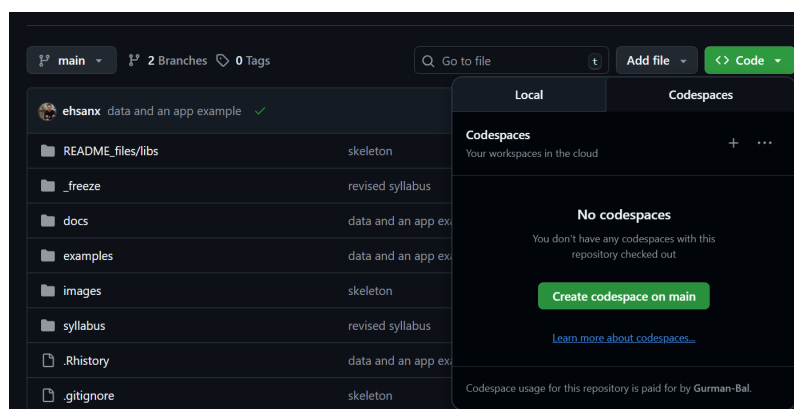


Figure 2.2: Can be accessed by clicking the <> Code button, navigating to the Codespaces tab, and then starting up a Codespace which contains all the file in the Repo.

- **Pre-installed software** It comes with R, Python, and Quarto installed, so you don't have to install anything on your own laptop.
- **The “Two-Save” Rule (Crucial)** Because this is a temporary computer, there are two steps to saving:
 1. **Ctrl+S (Save):** This only saves to the temporary machine. If the machine times out (closes), this is lost.
 2. **Commit & Push:** This sends your work back to the Repository (Cloud Drive). This is the only way to be safe.

Warning: If you close the tab without “Committing” (Sending), your work is erased.

2.2.1 Check for understanding

Answer:

- Where are your permanent files stored?
- What happens if you press **Ctrl+S** but never commit?

3 Tour of the Repository

3.1 Step-by-step

1. Open your course repository on GitHub

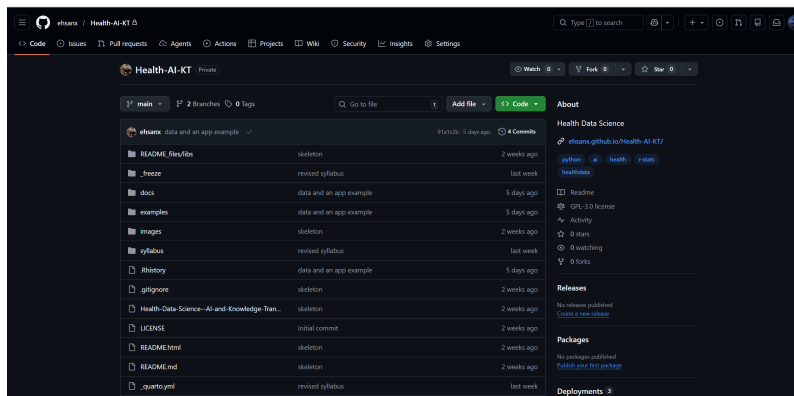
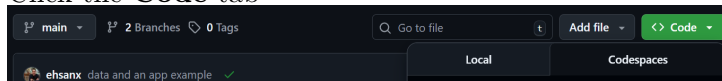


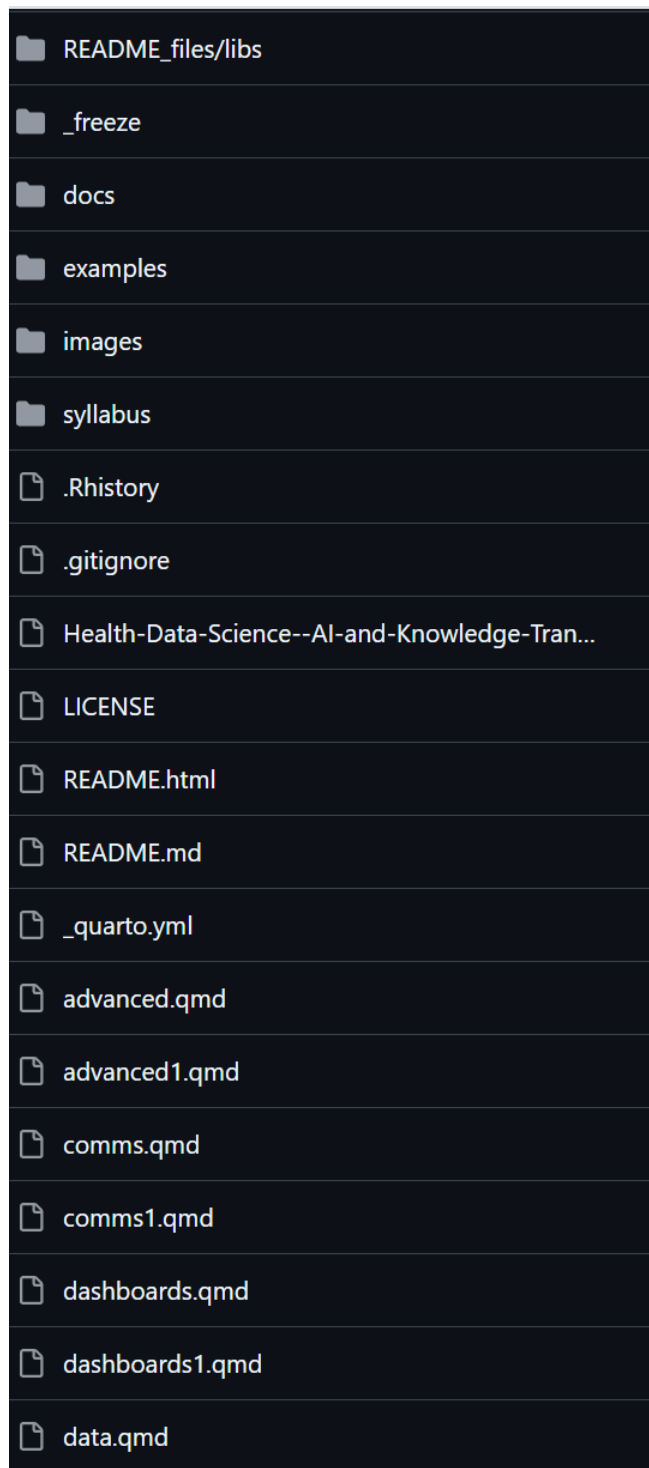
Figure 3.1: Main github page

2. Click the **Code** tab



- **Why?** GitHub has many tabs (Issues, Actions, Settings). The **Code** tab is your “Reset” button. If you ever get lost in a menu, click this to go back to your files.

3. Look at the file list



- This shows the **current state** of the project in the Vault.
- **Crucial:** If a file is listed here, it is safe. If it is *not* here, it only exists on your temporary “Desk” (Codespace) and is at risk of being lost.

4. Scroll to read the README

- The `README.md` file is displayed automatically at the bottom. Think of it as the “Cover Sheet” or “Instruction Manual” for your project.

3.2 Task

Find this file:

`practice_report.qmd`

Verification: If you can see `practice_report.qmd` in this list, it means the file is safely stored in the cloud (The Vault).

If you can see it, it is safely stored in the cloud.

4 Opening Your First Codespace

This process is like walking into the library and turning on a computer. You are creating a fresh, temporary environment in the cloud.

4.1 Steps

1. Click the green **Code** button
 - This button is usually for downloading files, but we are using it to launch a computer.

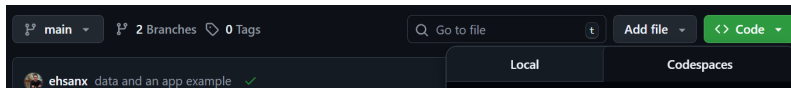


Figure 4.1: Code button

2. Click the **Codespaces** tab
 - **Crucial Choice:** You will see two tabs: “Local” and “Codespaces.”
 - **Local:** Means downloading to your own messy laptop (Hard mode).
 - **Codespaces:** Means using the pre-installed cloud computer (Easy mode). Always choose this.

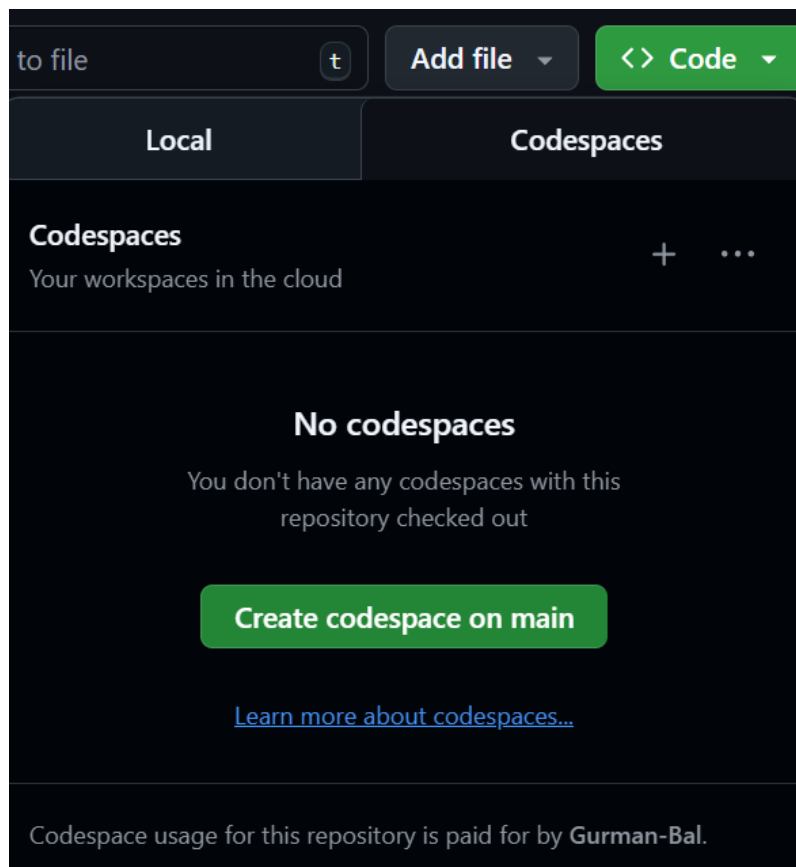


Figure 4.2: Codespaces tab

3. Click **Create Codespace on main**

- This tells GitHub to build a new computer for you from scratch.

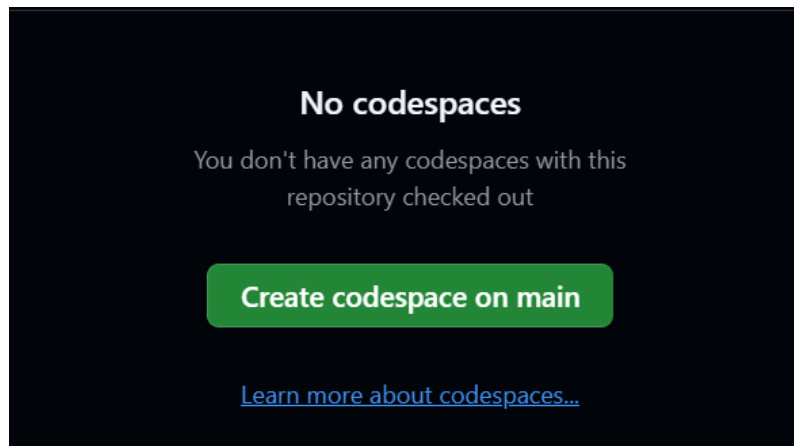


Figure 4.3: Create Codespace

4.2 The “Boot Up” Phase

Wait 2–3 minutes.

- **What is happening?** GitHub is silently installing R, Python, Quarto, and all the course libraries for you.
- **The Analogy:** Think of this as the “ventilation system” starting up in a lab. It takes a moment to get the environment ready.

Success: You should see **VS Code** (a dark-themed code editor) appear in your browser window.

5 Reopening a Codespace

The Rule: Do not create a new Codespace each time you log in.

- **Why?** If you click the green “Create” button again, GitHub builds a *second* computer.
- **The Risk:** If you left uncommitted work (Drafts) on “Computer A” and you start “Computer B,” your work will appear lost. It isn’t lost: it’s just on the other computer.

To resume exactly where you left off:

5.1 Steps

1. Go to the **Codespace Dashboard**: <https://github.com/codespaces>
 - Think of this as the “Front Desk” where all your active computers are listed.
2. Find your Codespace
 - Look for the box labeled with your course repository name.
 - **Note:** GitHub gives each Codespace a funny random name (e.g., *literate-computing-machine* or *glorious-couscous*). This is the name of your rented desk.

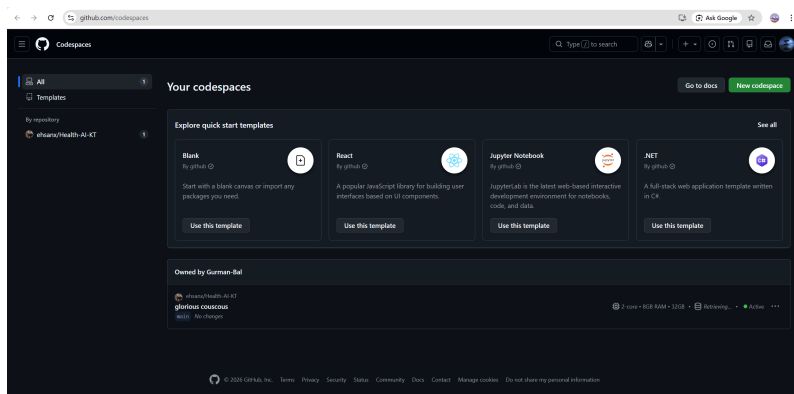


Figure 5.1: GitHub Codespaces

3. Click the **Name** of the Codespace

- Do **not** click the “New codespace” button in the corner. Click the text link of the funny name.

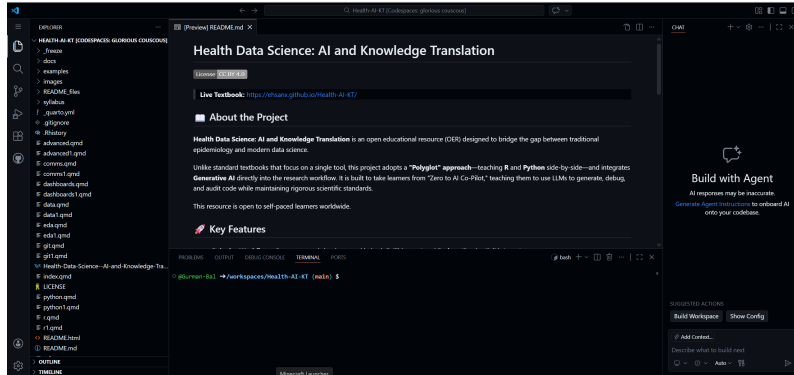


Figure 5.2: Your Codespace

4. Wait for it to resume

- **Benefit:** Resuming an existing Codespace is much faster than creating a fresh one because the software is already installed.

6 VS Code Orientation



When you open a Codespace, you are using a version of **VS Code** in your browser. It has the same interface as the desktop version used by professional software engineers at Google and Microsoft.

Don't Panic: The interface looks complex because it has hundreds of features. **You only need to know three areas.** You can safely ignore 90% of the buttons you see.

6.1 Explorer (left)

The “File Cabinet”

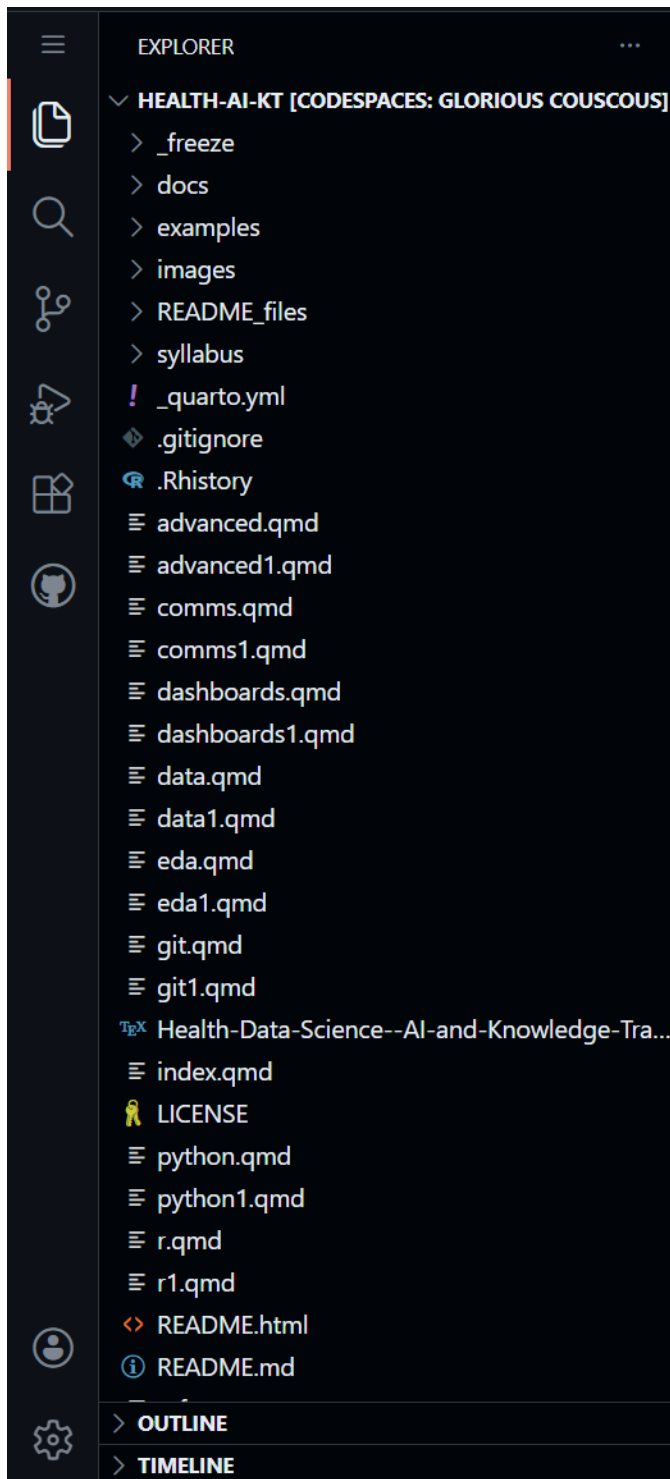


Figure 6.1: Shows files and folders.

- This shows your folders (directories) and files.
- Think of this as your navigation pane. Double-click a file here to open it in the Editor.

6.2 Editor (center)

The “Notebook”

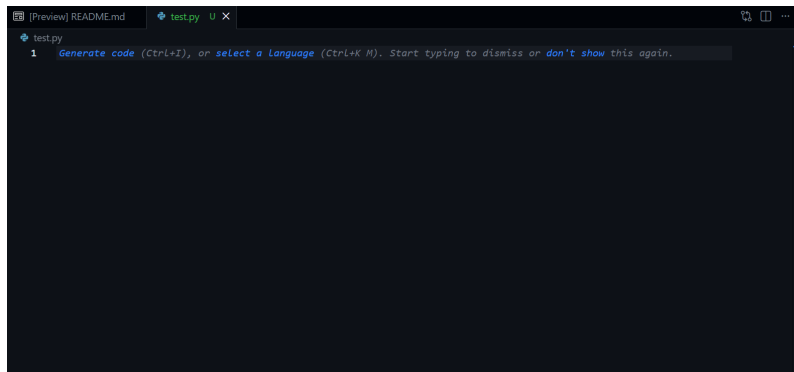


Figure 6.2: Where you type.

- This is where you do your actual work: typing code and writing text.
- In this course, you will mostly be editing `.qmd` (Quarto) files here.

6.3 Terminal (bottom)

The “Engine Room”

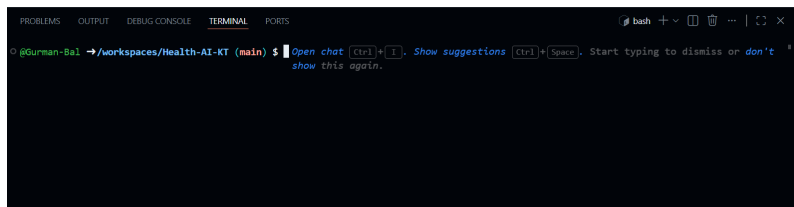


Figure 6.3: Terminal window

- This is where R and Python actually run.
- You will see text scrolling here when you “Render” a document. It looks like “The Matrix,” but it is usually just the computer telling you what it is working on.

If the terminal disappears (or you accidentally close it):

View → Terminal

6.3.1 Mini Task

Open and close a few files in the Explorer to get comfortable with the layout.

7 Files vs Folders

We use a consistent structure to ensure reproducibility. In Data Science, being “organized” isn’t just about being neat—it is a requirement for your code to work on someone else’s machine.

7.1 The Golden Rule of Health Data

We treat data like a **Crime Scene**—never touch the evidence.

- **Start:** `data/` (The Crime Scene). **READ ONLY.**
 - *Rule:* Never open a dataset in Excel and “fix” a typo. That is tampering with evidence.
 - *Correct Way:* Load the “dirty” data into R/Python and fix the typo using code. This leaves a trail of exactly what you changed.
- **End:** `output/` (The Evidence). All graphs and cleaned tables go here.
- **The Detective:** Your code (`.qmd`) moves information from Start to End.

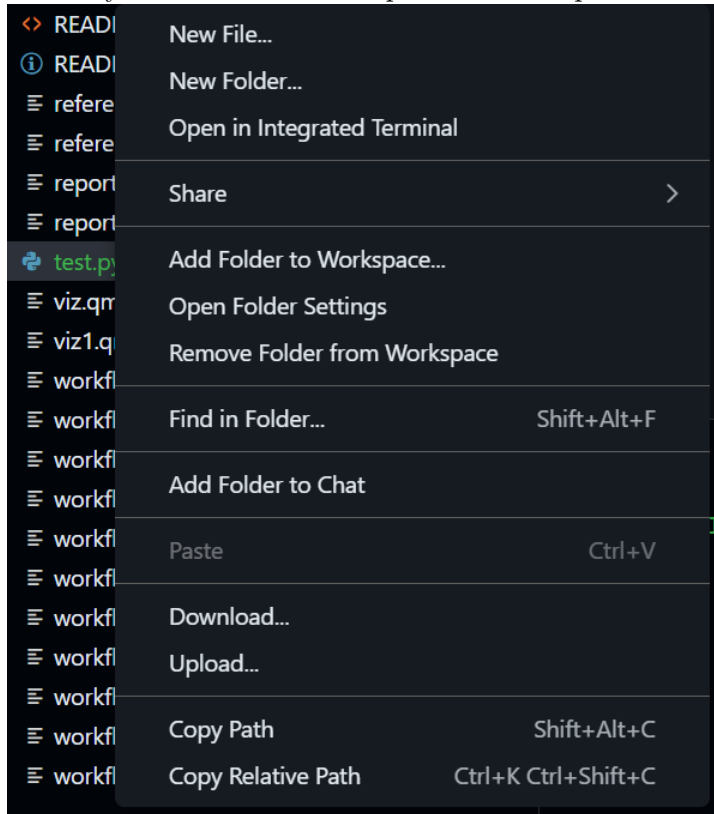
7.2 Files vs. Folders (The Mental Model)

- **Files (e.g., `report.qmd`):** These are **sheets of paper**. You write on them.
- **Folders (e.g., `data/`):** These are **labeled boxes**. You store things in them.

7.3 Create a folder

In Explorer:

1. Right-click anywhere in the blank space of the Explorer



pane.

2. Select **New Folder**
3. Name it:

images

Pro Tip: Always use **lowercase** and **no spaces** for folder names (e.g., `my_data`, not `My Data`). Computers struggle with spaces in file paths.

7.4 Typical structure

Your project should always look like this:

data/	raw data
output/	generated results
images/	screenshots
.qmd	reports

Remember: The `data` folder is for *input*. The `output` folder is for *results*. They should never mix.

8 The Magic Moment: Render Quarto

8.1 Why do we “Render”?

In the old days, you would run code, copy a graph, save it as a JPG, and paste it into Word. If the data changed, you had to redo every step manually.

The Better Way: Think of your code (`.qmd` file) as a **Recipe** and the final report (HTML/PDF) as the **Meal**. * **Editing:** You don’t change the meal; you change the recipe. * **Rendering:** This is the “cooking” process. When you click **Render**, the computer follows your instructions from scratch to create a fresh meal.

This is the secret to reproducibility: if you give your recipe to a friend, they can cook the exact same meal.

8.2 Step 1 — Open the file

1. Go to the **Explorer** pane (Left side).
2. Locate and click on the file:

```
practice_report.qmd
```

This will open the “Source Code” in the center Editor window. You will see a mix of normal text and code chunks.

8.3 Step 2 — Edit name (The YAML Header)

Look at the very top of the file. You will see a block of text sandwiched between three dashes (---). This is called the **YAML Header**. It controls the document’s metadata.

Find the line that says `author`:

```
author: "Your Name"
```

Task: Delete “Your Name” and type your actual name inside the quotes. Do not remove the quotes!

8.4 Step 3 — Render

1. Look for the **Render** button at the top of the Editor pane (it often looks like a Blue Arrow pointing right, or simply says “Render”).
2. **Click it.**
3. **Observe:**
 - **The Terminal (Bottom):** You will see text scrolling quickly. This is R/Python executing your code chunks.
 - **The Preview (Right):** A new window should appear showing a beautiful, formatted report with your name on it.

Congratulations! You just performed “Knowledge Translation”—turning raw code into a human-readable document.

8.5 If it fails (Troubleshooting)

Rendering involves many moving parts, so it sometimes gets stuck.

- **The “Wait” Rule:** If nothing happens for 20 seconds, the background process might be initializing. Give it a moment.
- **The “Retry” Rule:** Sometimes the connection hiccups. Click **Render** one more time.
- **The “Red Text” Rule:** Look at the **Terminal** tab at the bottom. If you see red text, it is an error message. Read it—it usually tells you exactly what went wrong (e.g., “File not found” or “Syntax error”).

9 Commit Your Work

Your work is currently only on the “Rented Desk” (Codespace). It is a **Draft**. To save it permanently to the “Vault” (GitHub), you must **Commit** and **Push** (Send).

The Analogy: Think of this like shipping a package. You have packed the box (saved the file), but now you need to put a label on it and hand it to the courier.

9.1 Steps

1. Click the **Source Control** icon
 - It looks like a **blue bubble** or a **connect-the-dots graph** on the far left sidebar.
 - This opens the “Shipping Department.” You will see a list of every file you changed.

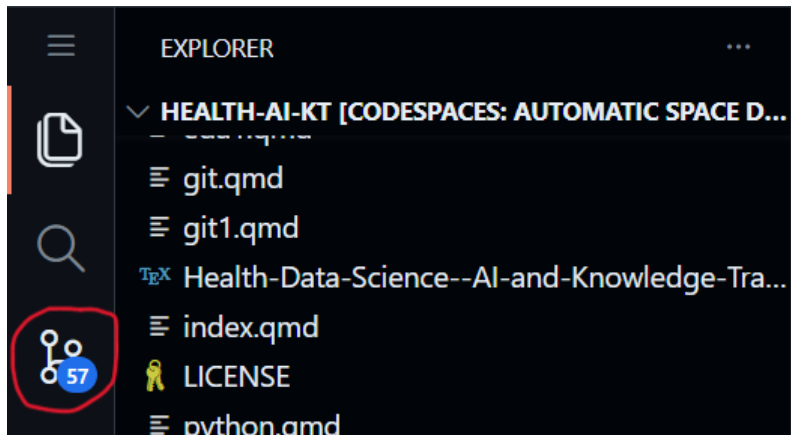


Figure 9.1: Source control icon circled in red

2. Type a **Commit Message**

- **The Label:** You cannot ship a package without a label.
- **What to write:** Be descriptive but brief. Imagine you are leaving a note for your future self about *what* you did.
- *Example:* “Updated author name in practice report”.
- *Bad Example:* “stuff” or “fix”.

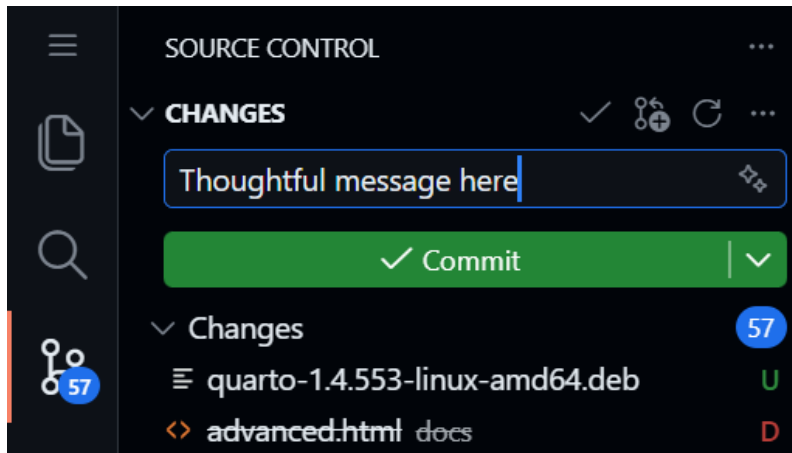


Figure 9.2: This is required

3. Click **Commit and Sync** (or Commit and Push)

- This is the “**Send**” button.
- **Action:** It wraps up your changes, stamps them with the time and your message, and teleports them to the Cloud Vault.
- *Note:* You might be asked to click “Always” or “Yes” to confirm the sync.

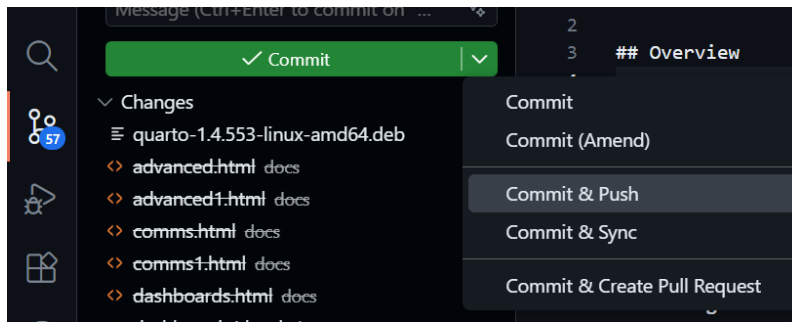


Figure 9.3: This will push your changes to the remote branch

9.2 Verify (Trust but Verify)

Did it actually work?

1. Go back to your **GitHub Repository** tab (The Vault).
2. **Refresh** the page.
3. Look at the file list. You should see your “Commit Mes-
sage” displayed next to the file you just edited.

10 Troubleshooting: Rebuild Container

Sometimes the cloud computer gets “stuck.” Maybe the terminal froze, or R crashed.

The Fix: We don’t fix broken computers; we get new ones. “Rebuilding the Container” is the cloud equivalent of “**Have you tried turning it off and on again?**”—but better, because it gives you a fresh machine while keeping your files safe.

10.1 When to use this?

If things break:

- **Terminal frozen:** You type, but nothing appears.
- **Render fails:** You get weird errors that make no sense.
- **Environment issues:** R or Python can’t find a package that was there yesterday.

10.2 Steps

1. Open the Command Palette

- Press `F1` (or `Ctrl+Shift+P` on Windows / `Cmd+Shift+P` on Mac).
- A search bar will appear at the very top of the screen. This is the “God Mode” search bar for VS Code.

2. Search

- Type: Rebuild Container

3. Execute

- Click on the option that says: **Codespaces: Re-build Container**

4. Wait (~2-5 minutes)

- The screen will reload.
- **Don't Panic:** You might see a black screen with scrolling text logs. This is normal: it is reinstalling the software.
- When your files reappear in the Explorer, you are ready to go.

Did I lose my work?

- **Saved files:** No. Anything you pressed **Ctrl+S** on is safe.
- **Unsaved typing:** Yes.
- **R Memory:** Yes. You will need to re-run your code chunks from the top.

11 Stop Your Codespace

Codespaces is not free forever. You have a monthly allowance (approx. 60 hours/month for free accounts).

The Analogy: Think of a Codespace like a **Taxi**.

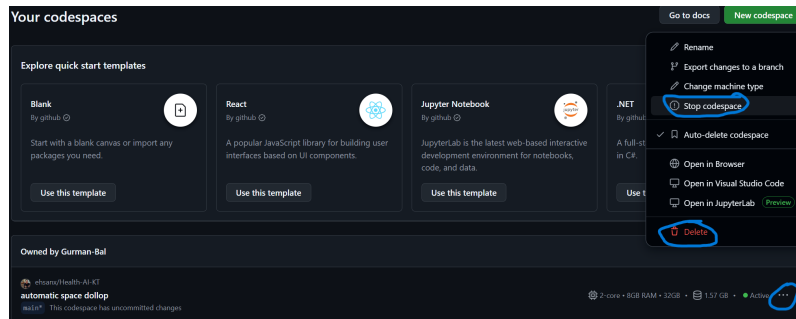
- **Working:** When you are coding, the meter is running.
- **The Trap:** If you just close your browser tab, it is like getting out of the taxi but asking the driver to “wait here.” **The meter keeps running.**
- **The Fix:** You must explicitly “end the ride” (Stop the Codespace).

11.1 Correct shutdown

1. Go to the **Codespace Dashboard**: <https://github.com/codespaces>
 - This is your command center.
2. Find your active Codespace
 - Look for the green text that says “Active”.
3. Click ... (The “Meatball” Menu)
 - It is located on the far right side of the Codespace box.
4. Click **Stop Codespace**
 - **Stop:** This “pauses” the computer. It saves your installed packages and RAM state so you can resume quickly next time. **(Recommended)**
 - **Delete:** This destroys the computer. You will have to wait 2-3 minutes for the “Boot Up” phase next time you start.

Critical Warning: Closing the browser tab is **NOT** enough.

- GitHub keeps the machine running for **30 minutes** after you close the tab (in case your internet flickered).
- **The Cost:** If you just close the tab every day, you will waste ~15 hours of your monthly quota on empty time.



12 Final Checklist

- ☐ I understand repository vs Codespace
- ☐ I rendered a `.qmd` file
- ☐ I committed changes
- ☐ I know how to rebuild container
- ☐ I know how to stop Codespace

12.1 Reflection

Write 2–3 sentences:

- What was easiest?
- What was confusing?
- What questions do you still have?

Part V

Week 3: R with AI

Week 3: R with AI

Overview

Summary: In Week 2 you learned how to use your Codespace, edit a Quarto document, and make your first commit. Now you learn how to speak R—not by memorizing syntax, but by directing an AI co-pilot and *auditing* what it writes. Your job is to be the pilot: you decide what the analysis should do, the AI drafts the code, and you verify every line before it flies.

Learning Objectives:

- Understand that R is a calculator that needs specific, case-sensitive instructions.
- Load packages with `library()` and explain why this is needed each session.
- Use core constructs: variables (assignment), functions (with arguments), and the pipe (`%>%` / `|>`).
- Perform basic data transformations using `dplyr` verbs: `filter()`, `select()`, `mutate()`, `summarise()`, `group_by()`.
- Inspect a dataset on first contact: structure, dimensions, missingness.
- Identify control flow (`if/else`, loops) in AI-generated code and audit whether it is correct.
- Write and adapt a simple custom function.
- Apply an AI safety checklist to catch hallucinations, wrong column names, and unsafe operations.
- Document your analysis in Quarto by weaving code chunks with Markdown narrative.
- Commit incrementally as you work (reinforcing Week 2 Git basics).

r1.qmd Packages: Opening the Toolbox

Install vs Load

- `install.packages("tidyverse")` — **buying** the toolbox. Done once per environment (already done in your course Codespace).
- `library(tidyverse)` — **opening** the toolbox. Required every time you restart R or reopen your Codespace.

```
1 library(tidyverse)
```

Note

If you see an error like `there is no package called 'tidyverse'`, the package is not installed in your Codespace. Ask your TA — this should be pre-configured.

r2.qmd Core Constructs: Boxes, Toasters, and the Assembly Line

Variables: The Labeled Box

R has no memory unless you store results in a named box using `<-`.

```
1 patient_age <- 55
2 clinic_name <- "Vancouver General"
```

Warning

Case sensitivity: `Age`, `age`, and `AGE` are three different boxes. This is one of the most common bugs in AI-generated code. Always check that column names match your data exactly.

Functions: The Toaster

A function takes **input** and returns **output**. Arguments (inside the parentheses) control its behaviour.

```
1 mean(c(72, 85, 90), na.rm = TRUE)
```

- `c(72, 85, 90)` is the input (a vector of numbers).
- `na.rm = TRUE` tells R to ignore missing values. **Always look for this** when auditing AI code that works with health data — missingness is the norm, not the exception.

The Pipe: The Assembly Line

The pipe passes the result of one step into the next. Think of it as “**and then...**”

```
1 data %>%  
2   filter(age > 65) %>%  
3   summarise(mean_bmi = mean(bmi, na.rm = TRUE))
```

Read this as: “*Take the data, and then keep only patients over 65, and then calculate the average BMI.*”

Tip

You will see two pipe symbols in the wild: `%>%` (from the `magrittr` / `tidyverse` packages) and `|>` (built into R 4.1+). They do the same thing for our purposes. If AI generates the other one, don’t panic — both work in your Codespace.

r3.qmd Data Import and First Contact

Loading Data

Use relative paths so your code runs on any machine (including your grader's Codespace).

```
1 my_data <- read_csv("data/patients.csv")
```

How to get the path right: Right-click the file in the VS Code Explorer → **Copy Relative Path** → paste it into your code.

AI Prompt:

Read the CSV file at “data/patients.csv” and save it as my_data. Use the readr package.

First Contact: Inspect Before You Analyze

Never start analysis without looking at your data. Run these four checks:

```
1 # How big is it?
2 dim(my_data)           # rows x columns
3
4 # What are the column names and types?
5 glimpse(my_data)
6
7 # Quick summary statistics
8 summary(my_data)
9
10 # How much is missing?
11 colSums(is.na(my_data))
```

Open the **Environment** pane (top-right in RStudio or via the R extension) to see your variables and click a data frame to view it in a grid.

 Warning

Health data reality: Real datasets like NHANES have substantial missingness. Knowing *where* and *how much* is missing is not optional — it shapes every downstream decision.

r4.qmd Data Transformation: The Five dplyr Verbs

These five verbs are the core of data wrangling in R. You will use them in nearly every assignment and in your term project.

filter() — Keep rows that match a condition

```
1 # Patients aged 65 and older
2 seniors <- my_data %>% filter(age >= 65)
```

select() — Keep (or drop) columns

```
1 # Keep only ID, age, and BMI
2 slim <- my_data %>% select(id, age, bmi)
```

mutate() — Create or modify a column

```
1 # Add a BMI category column
2 my_data <- my_data %>%
3   mutate(bmi_category = case_when(
4     bmi < 18.5 ~ "Underweight",
5     bmi < 25  ~ "Normal",
```

```

6     bmi < 30    ~ "Overweight",
7     TRUE      ~ "Obese"
8   ))

```

summarise() — Collapse rows into a summary

```

1 my_data %>% summarise(mean_age = mean(age, na.rm = TRUE))

```

group_by() + summarise() — Summaries by group

```

1 my_data %>%
2   group_by(sex) %>%
3   summarise(
4     n = n(),
5     mean_bmi = mean(bmi, na.rm = TRUE)
6   )

```

AI Prompt:

Using `dplyr`, filter my dataset to patients over 65, group by sex, and calculate the mean BMI for each group. Handle missing values.



Tip

Audit the AI output: After running the AI-generated code, check that (a) the column names match your actual data, (b) `na.rm = TRUE` is present where needed, and (c) the row counts make sense (e.g., the filtered data should be smaller than the original).

Git checkpoint: You have now loaded, inspected, and transformed data. Stage your `.qmd`, write a commit message describing what you did, and commit. Don't wait until the end.

r5.qmd Control Flow: Auditing AI Decisions

You will not write many `if/else` blocks or loops by hand. But AI frequently generates them, so you need to **read** them.

If/Else: The Fork in the Road

```
1 if (mean_age > 50) {  
2   print("Older cohort")  
3 } else {  
4   print("Younger cohort")  
5 }
```

Audit question: Is the threshold (50) appropriate for my research question, or did the AI pick an arbitrary number?

Loops: The Treadmill

```
1 for (col in c("age", "bmi", "sbp")) {  
2   print(summary(my_data[[col]]))  
3 }
```

Audit questions: Does the loop have a clear end condition? Could this be replaced with a simpler vectorized operation (e.g., `summarise(across(...))`)?

Note

In R, loops are rarely the best tool. If AI gives you a loop, ask: “*Can this be done without a loop using dplyr or purrr?*” The answer is usually yes.

r6.qmd Writing a Simple Function

The syllabus asks you to write at least one custom function. A function packages a repeated task so you don't copy-paste code.

Example: A Reusable Summary

```
1 summarise_column <- function(data, column_name) {  
2   data %>%  
3     summarise(  
4       n_total   = n(),  
5       n_missing = sum(is.na(.data[[column_name]])),  
6       mean_val  = mean(.data[[column_name]], na.rm = TRUE),  
7       sd_val    = sd(.data[[column_name]], na.rm = TRUE)  
8     )  
9   }  
10  
11 # Use it  
12 summarise_column(my_data, "bmi")  
13 summarise_column(my_data, "age")
```

Your Task

1. Ask your AI co-pilot to write a function that takes a data frame and a column name and returns summary statistics.
2. **Audit the code:** Does it handle NA? Does it work on a numeric column? What happens if you pass a categorical column?
3. **Adapt it:** Change one thing (e.g., add a median, or add a `group_by` argument). This is the difference between accepting code blindly and understanding it.

AI Prompt:

Write an R function called `summarise_column` that takes a data frame and a column name (as a string) and returns the count, number of missing values,

mean, and standard deviation. Then show me how to call it on the “bmi” column of `my_data`.

r7.qmd Debugging: Reading the Red Text

Errors are normal. AI makes mistakes, you make typos, and data surprises you.

The “Fix It” Loop

1. **Read** the error message. Look for keywords: `not found`, `unexpected`, `non-numeric`.
2. **Copy** the full error text.
3. **Paste** it into your AI chat and ask:

Here is an R error from my Codespace: [paste]. Explain what went wrong in one sentence and give corrected code.

Common Errors and What They Mean

Error message	Likely cause
<code>object 'X' not found</code>	Typo in variable name, or you forgot to run an earlier chunk
<code>could not find function "X"</code>	Forgot <code>library(tidyverse)</code> or misspelled the function
<code>non-numeric argument to binary operator</code>	Trying to do math on a text/factor column
<code>unexpected symbol</code>	Missing comma, parenthesis, or pipe

 Warning

AI debugging guardrail: When AI suggests a fix, re-read the corrected code before running it. Does the fix address the *actual* error, or did AI rewrite the whole chunk and change your logic?

r8.qmd AI Safety: The Audit Checklist

Every time you accept AI-generated code, run through this checklist:

1. **Column names:** Are they spelled exactly as they appear in `glimpse(my_data)`?
2. **Data types:** Is the code doing math only on numeric columns?
3. **Missingness:** Is `na.rm = TRUE` present where needed?
4. **Privacy:** Does the code avoid printing individual patient rows that could be identifiable?
5. **Logic:** Does the code do what *you* intended, or what the AI *assumed* you meant?

The Paper Trail Rule

Every time you use AI to write a code chunk, add a `#` comment above it explaining what you asked for and what you verified:

```
1 # AI prompt: "Calculate mean BMI by sex, handling NAs"
2 # Verified: column names match, na.rm = TRUE present, output has 2 rows (M/F)
3 my_data %>%
4   group_by(sex) %>%
5   summarise(mean_bmi = mean(bmi, na.rm = TRUE))
```

This comment trail is your audit log. It demonstrates accountability (required by the course AI policy) and helps you debug later.

r9.qmd Exporting Clean Data

After cleaning and transforming, save your work so future weeks can pick up where you left off.

```
1 # Save a clean version for Week 4 and beyond
2 write_csv(my_data, "output/clean_data.csv")
```



Tip

Reproducibility habit: Your Quarto document should contain *every step* from raw data to clean export. Anyone (including your grader) should be able to render it from scratch and get the same `clean_data.csv`.

Git checkpoint: Your analysis is complete. Stage, commit with a clear message (e.g., "Complete Week 3: load, inspect, transform, and export patient data"), and Sync.

r10.qmd Literate Analysis in Quarto

Your `.qmd` file is not just code — it is a **document**. Weave narrative around your code chunks to explain what you are doing and why.

Pattern

““markdown ## Data Inspection

We begin by examining the structure of the dataset to understand variable types and missingness before any transformations.

```
{r, eval=FALSE} glimpse(my_data) colSums(is.na(my_data))
```

The dataset contains `nrow(my_data)` observations across `ncol(my_data)` variables. Notably, BMI is missing for approximately 12% of records, which we handle with `na.rm = TRUE` in downstream summaries.

This interleaving of code and prose is **literate programming** — the methodology that underpins reproducible research in this course. Practice it now; you will need it for your term project report (Week 10) and final portfolio.

r11.qmd Knowledge Check

1. What is the difference between `install.packages()` and `library()`?
2. Why is `na.rm = TRUE` important when working with health data?
3. What does `my_data %>% filter(age > 65) %>% summarise(n = n())` do? Read it aloud using “and then.”
4. Name the five core `dplyr` verbs and what each does in one sentence.
5. You see the error object `'BMI' not found` but your column is called `bmi`. What went wrong?
6. What is the difference between `%>%` and `|>`? Does it matter for this course?
7. Why should you write `#` comments above AI-generated code?
8. What four checks should you run immediately after loading a new dataset?

r12.qmd Assignment: The “Mad Libs” Report

Task: Build a short Quarto report that loads, inspects, transforms, and summarizes a health dataset — using your AI copilot for code generation and your own judgment for verification.

Step-by-Step Workflow

1. Open `week3_assignment.qmd` in your Codespace.
2. **Load packages and data:** Use AI to write the `library()` and `read_csv()` calls. Verify the data loaded correctly with `dim()` and `glimpse()`.
3. **Inspect missingness:** Run `colSums(is.na(...))`. Add a Markdown paragraph interpreting what you see.
4. **Transform:** Use `dplyr` verbs to:
 - Filter to a meaningful subgroup (e.g., age `> 18`).
 - Create at least one new column with `mutate()`.
 - Produce a grouped summary with `group_by()` + `summarise()`.
5. **Write a function:** Create (or adapt from AI) a reusable summary function. Call it on at least two different columns.
6. **Export:** Save your cleaned data to `output/clean_data.csv`.
7. **Document:** Every code chunk should have:
 - A `#` comment stating what you asked the AI and what you verified.
 - A Markdown paragraph below interpreting the result.
8. **Render** the document to confirm it runs cleanly from top to bottom.
9. **Git:** Stage, write a descriptive commit message, Commit, and Sync.

Deliverable

A rendered `.qmd` that a reader can follow as a narrative: what data you started with, what you did to it, what you found, and why your decisions were justified.

r13.qmd Quick Reference

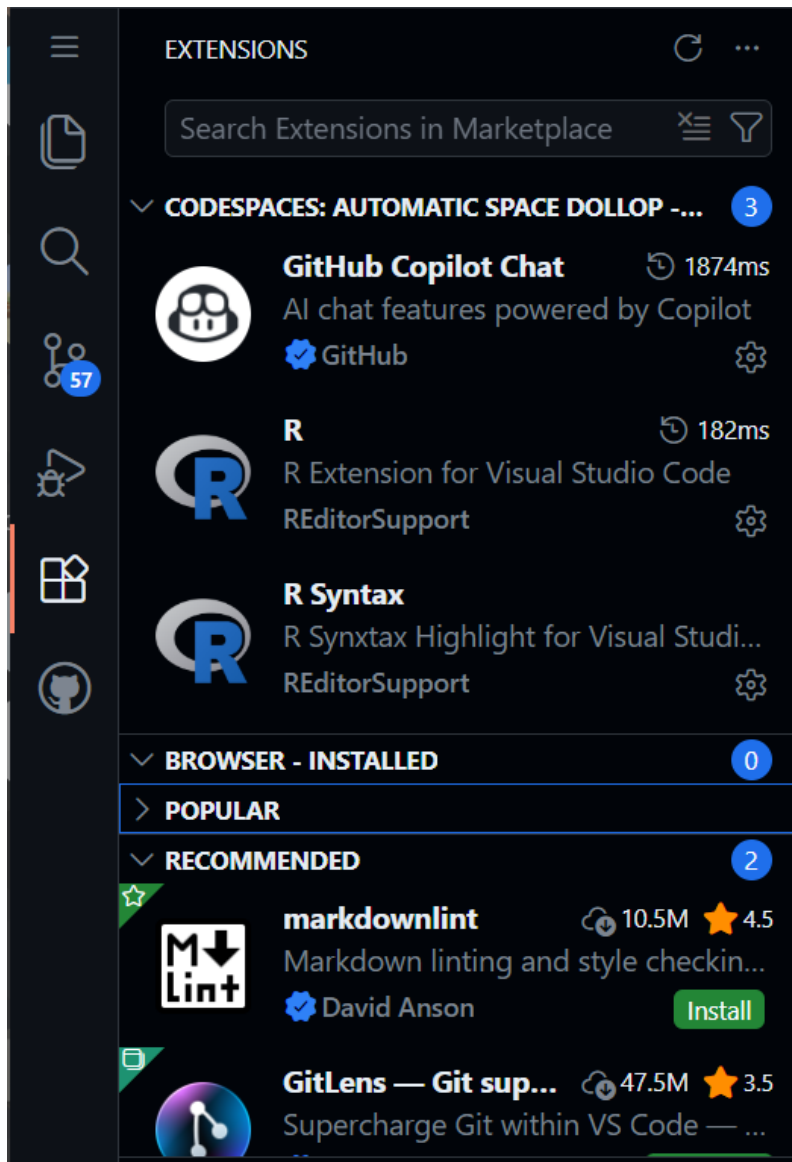
Concept	R code	Notes
Load packages	<code>library(tidyverse)</code>	Every session
Read CSV	<code>read_csv("data/files.csv")</code>	Use relative paths
Assignment	<code>x <- 5</code>	The <code><-</code> arrow stores a value
Inspect structure	<code>glimpse(df)</code>	Column names, types, preview
Dimensions	<code>dim(df)</code>	Rows $\times$ columns
Summary stats	<code>summary(df)</code>	Min, median, mean, max, NAs
Check missingness	<code>colSums(is.na(df))</code>	Count NAs per column
Filter rows	<code>filter(df, age > 65)</code>	Keep matching rows
Select columns	<code>select(df, id, age, bmi)</code>	Keep named columns
New column	<code>mutate(df, bmi_cat = ...)</code>	Create or modify
Summarize	<code>summarise(df, m = mean(x, na.rm = TRUE))</code>	Collapse to one row
Group + summarize	<code>group_by(df, sex) %>% summarise(...)</code>	Per-group summaries
Export	<code>write_csv(df, "output/clean.csv")</code>	Save for later
Pipe	<code>%>%</code> or <code> ></code>	“And then...”

13 The Setup (The Cockpit)

Before we start flying, we need to check our instruments in **VS Code**. If your environment isn't set up correctly, your code will not run.

13.1 Verify Your AI Extensions

1. Click the **Extensions** icon on the far-left sidebar (it looks like four squares).
2. In the search bar, ensure both **GitHub Copilot Chat** and **R** are installed and active.



13.2 Set the AI Model to “0x”

1. Open the Copilot Chat sidebar (Shortcut: **Ctrl+Alt+I** or **Cmd+Option+I** on Mac).
2. Look for the model selector dropdown at the bottom of the chat window.

3. **Crucial Check:** Choose a model with a “0x” **multiplier** (like GPT-4o 0x). If you do not do this, you will quickly run out of your free student quota.
 - *Troubleshooting:* If the Chat asks you to “Sign In” or “Subscribe,” your Student Pack is not active. Use your institutional login at copilot.microsoft.com or authenticate via your university email at <https://m365.cloud.microsoft/chat/>.

13.3 Interface Review: Where does R actually live?

To learn R, we are going to start in the “sandbox.” Here is how your screen is laid out:

- **The Console (Bottom): Your Engine Room.** *Start here!* This is where you can type R commands directly next to the > prompt and press Enter to see immediate results. If you make a mistake, it will show up here in red text.
- **The Environment (Top Right): Your Work Bench.** Whenever you save a piece of data (like a patient’s age), it will appear in a list here so you can verify R actually “remembers” it.
- **The Explorer (Left Sidebar): Your File Cabinet.** This is where you find your **data/** folder containing your CSV files.
- **The Editor (Top Center): Your Laboratory Notebook.** Typing in the Console is great for testing, but it forgets everything when you close the window. Later in the lesson, we will use the Editor to write and save our permanent code.

13.4 Interface Review: Working with .qmd and .Rmd files

When you open a .qmd (Quarto) or .Rmd (RMarkdown) file, you must know where to look. Here is how your coding workflow maps to your screen:

- **The Explorer (Left Sidebar): Your File Cabinet.** This is where you find your reports and your `data/` folder. You cannot run code here; you only click files to open them.
- **The Editor (Top Center): Your Laboratory Notebook.** This is where you write your text narrative and your R code.
 - *Note:* R code must be placed inside grey **Code Chunks** (starting with ````\r{` and ending with `````). To run a chunk, click the small green “Play” triangle in the top right corner of that specific grey box.
- **The Console (Bottom): Your Engine Room.** When you click the green “Play” button in the Editor, the code is sent down here to be executed. If there is an error, it will show up here in red.
 - *Pro Tip:* You can type quick math directly into the console (like `100 / 4`), but it will not be saved in your final report.
- **The Environment (Top Right): Your Work Bench.** Whenever you load data or save a calculation, it will appear in a list here so you can verify R actually “remembers” it.

14 Packages (The Toolbox)

In R, tools come in “packages.” Think of `install.packages` as buying a physical toolbox, and `library()` as opening the lid.

- **To open your tools:** Type this in your code and run it:

```
library(tidyverse)
```

Note: You must run `library(tidyverse)` every single time you restart your Codespace.

15 Basics: Boxes & Toasters

R is a system of storage (Boxes) and transformation (Toasters).

15.1 Variables (The Box)

Use `<-` to save data into a name.

Code snippet

```
Save the number 55 into a variable called patient_age patient_age
```

15.2 Functions (The Toaster)

Functions take input, perform an action, and return output.
Look at the snippet below

Code snippet

```
calculate_rectangle_area <- function(length, width) {  
  area <- length * width  
  return(area)  
}
```

```
# Example usage:  
calculate_rectangle_area(10, 5) # Returns 50
```

16 Control Flow

16.1 The Pipe %>% (The Assembly Line)

The symbol %>% means “**and then...**”.

Both of these lines are identical, which would you prefer to read? (Note |> is another way to write %>%)

```
print(round(sqrt(10), 2))
```

```
10 |> sqrt() |> round(2) |> print()
```

For a real world example of how one may use the pipe

```
library(dplyr)
```

```
mtcars %>%  
  filter(wt > 3) %>%          # 1. Filter for cars heavier than 3,000 lbs  
  group_by(cyl) %>%          # 2. Group by number of cylinders  
  summarize(avg_mpg = mean(mpg)) # 3. Calculate the average MPG
```

16.2 If / Else (The Fork in the Road)

An **if/else** statement lets the computer make a decision.

The program checks a condition:

- If it is **TRUE**, it runs one block of code.
- If it is **FALSE**, it runs another.

```
temperature <- 5
if (temperature > 10) {
  print("You can wear a t-shirt")
} else {
  print("Wear a jacket")
}
```

16.3 What happens

1. The computer checks: `temperature > 10`
2. `5 > 10` is **FALSE**
3. So the **else** part runs.
4. Output:

"Wear a jacket"

Think of it like a **fork in the road**:

- One path if the condition is true
- Another path if it's false.

16.4 Loops (The Treadmill)

A loop repeats code multiple times.

Instead of writing the same instruction again and again, you let the computer do it.

16.5 Example

```
for (i in 1:5) {
  print(i)
}
```

16.6 What happens

The computer runs the code **5 times**.

Steps:

1. `i = 1` → print 1
2. `i = 2` → print 2
3. `i = 3` → print 3
4. `i = 4` → print 4
5. `i = 5` → print 5

Output:

1 2 3 4 5

16.7 Audit Check (Important)

When using loops, ask:

1. Does the loop end?

Bad example (infinite loop):

```
while (TRUE) {  print("This never stops") }
```

2. Could this be simpler?

Sometimes R has built-in functions that replace loops.

Instead of:

```
total <- 0 for (i in 1:5) {  total <- total + i }
```

You can write:

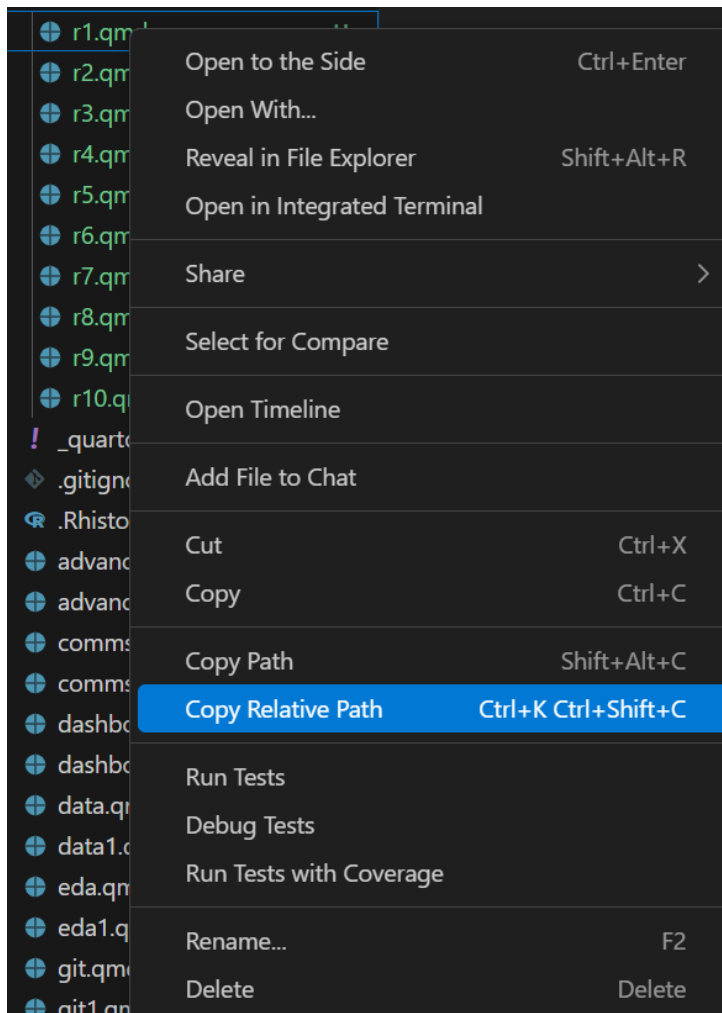
```
sum(1:5)
```

This is **shorter, safer, and faster**.

17 The Loading Dock (Data Import)

Don't type file paths from memory; you will likely make a typo.

1. Find a data file in the **Explorer** (left sidebar).
2. **Right-click** the file → Select **Copy Relative Path**.



3. Ask AI: *“Read the CSV at [Paste Path Here] and save it as my_data.”*

18 The Detective (Debugging)

AI makes mistakes. When you see **Red Text** (errors) in the Console:

1. **Copy** the Red Text.
2. **Paste** it into Copilot Chat.
3. **Ask:** *“Explain this error and fix the code below: [Paste Code]”*

Try running this line and ask AI why it doesn't work:

```
"apple" + 5
```

19 AI Safety: The Audit Checklist

Use this checklist before accepting any AI code:

- **Spelling:** Are column names spelled exactly as they appear in the data?
- **Case Sensitivity:** Is it `Age` or `age`? (R sees these as different things).
- **Missingness:** Did the AI remember `na.rm = TRUE` for `math`?
- **Privacy:** Did the AI try to print the whole dataset (potentially exposing patient IDs)?

20 The Paper Trail (Comments)

The Rule: Every time you use AI to write code, write a # comment in plain English above it explaining what you told the AI to do.

Code snippet

```
# I asked AI to filter for patients with BMI over 30 and count them # [AI CODE BELOW]
```

Below is an example of non readable code

```
# What is x? What is 1.05? Why are we subsetting?
f <- function(x, y) {
  z <- x[x$a > y, ]
  r <- sum(z$b) * 1.05
  return(r)
}

res <- f(df, 100)
```

And here is an example of readable code

```
library(dplyr)

# Calculate the total revenue for high-value orders including a 5% tax
calculate_total_revenue <- function(order_data, minimum_price_threshold) {

  sales_tax_rate <- 1.05

  total_revenue <- order_data |>
    filter(price > minimum_price_threshold) |>
```

```
    summarize(total = sum(price)) |>
    pull(total)

    return(total_revenue * sales_tax_rate)
}

# Usage:
final_report_value <- calculate_total_revenue(current_orders, minimum_price_threshold = 100)
```

Just by reading through each line a reader should understand what is going on. One does not need to comment every single line, but rather comment whenever something complex is happening on the code line below it. And as always source work which is not yours.

21 Assignment: The “Mad Libs” Report

1. Open `week3_assignment.qmd`.
2. Use Copilot to import the Week 1 dataset.
3. Use the **Pipe** to filter and summarize one variable.
4. **Audit:** Add a `#` comment explaining your prompt.
5. **Render** the file to HTML.
6. **Commit and Push** to GitHub.

22 Final Summary Checklist

- ☐ Extensions are active (Copilot + R).
- ☐ Using **0x model** in Chat.
- ☐ Loaded `library(tidyverse)`.
- ☐ Data imported via **Relative Path**.
- ☐ All code chunks have an **Audit Comment (#)**.
- ☐ Handled **NA** values correctly.
- ☐ Document **Rendered** and **Pushed**.

Part VI

Week 4: Git & Collaboration

Git & Collaboration

Overview

Summary: You already know how to save files and make a basic commit in your Codespace. Now, learn to use Git as a *time machine* and *collaboration engine* for your analysis—tracking every change, safely undoing mistakes, working with teammates through branches and Pull Requests, and keeping a professional audit trail of your research workflow.

Learning Objectives:

- Distinguish between saving a file and taking a version-control “snapshot” (commit).
 - Read code differences (diffs) to see exactly what changed.
 - Use the Codespaces Source Control UI to revert to a previous working version.
 - Create and switch branches to isolate experimental work.
 - Open a Pull Request on GitHub and understand its role in peer review.
 - Configure `.gitignore` and use relative paths for repo hygiene.
 - Use AI to translate diffs into clear, professional audit logs.
 - Set up a group repository and agree on collaboration norms for the term project.
-

git1.qmd Concepts: The “Save Game” of Reproducible Science

Why Git Matters in Health Research

- **The audit trail:** Why `analysis_final_v3.qmd` is unsafe in health research; Git gives you a permanent, searchable history.
- **The timeline:** Your repository is a sequence of safe checkpoints (commits). Each commit records *who* changed *what* and *why*.
- **Reading the diff:** Red = removed. Green = added. This is the visual language of reproducibility.

Quick Recap (from Week 2)

You already know how to stage, commit, and sync in your Codespace. This week, we go deeper: branching, collaboration, and building habits that will carry through your term project.

git2.qmd Tutorial: Time Travel in Codespaces

Scenario: We will intentionally break last week’s R script and use Git to rescue it.

Step 1: Confirm Your Baseline

Open your `.qmd` from last week. Make a small, meaningful edit (e.g., improve a comment). Stage (+), write a descriptive commit message, and commit (). This is your safety net.

Step 2: The Disaster

Delete a crucial line of R code in a chunk (e.g., the line that loads the data). **Save** the file (Ctrl/Cmd + S).

Step 3: Assess the Damage

Open **Source Control** → click the modified file → view the side-by-side **diff** to see exactly what changed. Practice reading the red (removed) and green (added) lines.

Step 4: The Undo Button

Hover over the file name in the Source Control panel and click **Discard Changes** (the curved arrow) to restore the file to the **last committed** version.



Tip

Key takeaway: Saving writes to disk. Committing writes to *history*. Only committed work can be recovered by Git.

git3.qmd Repo Hygiene: .gitignore and Relative Paths

Why This Matters

As your projects grow, you will generate files that should *not* be tracked: rendered HTML, temporary data caches, `.DS_Store`, API keys. Tracking these clutters your history and can leak sensitive information.

.gitignore Walkthrough

1. Open (or create) the `.gitignore` file in the root of your repository.
2. Add common patterns:

```
# Rendered outputs
*.html
*_files/
```

```
# OS junk
.DS_Store
Thumbs.db
```

```
# Sensitive
.env
```

3. Stage, commit with a message like "Add `.gitignore` to exclude rendered files".

Relative Paths

Always use **relative paths** in your code (e.g., `read.csv("data/nhanes.csv")`), never absolute paths (e.g., `read.csv("/home/user/Documents/nhanes.csv")`). This ensures your analysis runs on *any* machine—including your grader's Codespace.

Warning

Reproducibility rule: Your term project must run from start to finish in a *fresh* Codespace. Absolute paths guarantee failure.

git4.qmd Branching: Safe Experimentation

Concept

A **branch** is a parallel copy of your project. You can experiment freely on a branch without affecting the stable **main** version. When your experiment works, you **merge** it back.

Think of it like a lab notebook: **main** is your clean, published record. A branch is your scratch pad.

Tutorial: Create and Merge a Branch in VS Code

Step 1: Create a Branch

- Click the branch name in the bottom-left corner of VS Code (it likely says **main**).
- Select **Create new branch...** and name it something descriptive: **add-age-filter**.

Step 2: Make Changes on the Branch

- Edit your **.qmd** (e.g., add a line of code that filters data by age).
- Stage and commit on this branch.

Step 3: Switch Back to **main**

- Click the branch name again → select **main**. Notice your recent edits *disappear* from the file. They are safely stored on the other branch.

git5.qmd Pull Requests: The Peer Review Gateway

What Is a Pull Request?

A **Pull Request (PR)** is a GitHub feature that says: *“I have changes on a branch. Please review them before they go into **main**.”* PRs are how professional teams and researchers collaborate. You will use them for peer review later in this course.

Tutorial: Open Your First PR

1. After committing changes to a branch (see `git4.qmd`), click **Sync Changes** to push the branch to GitHub.
2. Open your repository on **github.com**. You should see a banner: *“add-age-filter had recent pushes.”* Click **Compare & pull request**.
3. Write a title and short description explaining what you changed and why.
4. Click **Create pull request**.
5. For now, review the diff on GitHub, then click **Merge pull request** → **Confirm merge**.

Tip

Collaboration norm: In your term project, agree with your group that changes go through PRs rather than directly to **main**. This creates a built-in review step.

git6.qmd AI Auditing for Version Control

Treat the LLM as a **Git Translator** that explains diffs and drafts audit-quality messages.

git6a.qmd **Scenario A: Diff Translation (Pre-Commit Audit)**

Prompt:

“I am about to commit these changes. Explain what I changed in plain English. Did I delete anything important?”

Use this to spot accidental deletions or unintended edits before you commit.

git6b.qmd **Scenario B: Auto-Scribe Commit Messages**

Prompt:

“Based on this diff, write a one-sentence, professional commit message for my Quarto document.”

Check that the message captures the **intent** (e.g., "Filter patient cohort to age 65 per protocol"), not just the action ("changed line 42").

git6c.qmd **Scenario C: Panic Translator (Error Handling)**

Prompt:

“VS Code in my GitHub Codespace showed this Git error: [paste]. What does it mean in simple terms, and which **buttons in the VS Code UI** should I click to fix it?”

Warning

Strict Guardrail: Never run random terminal commands suggested by an AI (especially `--force` or `reset --hard`). Prefer the VS Code visual interface, which is safer. If the AI suggests a terminal command, ask it for the GUI equivalent first.

git7.qmd Setting Up Your Group Repository

This week you form your term project groups (**Milestone 0**). Use this session to establish your collaboration workflow.

Steps

1. **One member** creates the group repository from the GitHub Classroom link (provided on Canvas).
2. All other members accept the invitation and clone the repo by launching a Codespace from it.
3. As a group, agree on norms:
 - All changes go through **branches** → **PRs** (no direct pushes to **main**).
 - Commit messages should be descriptive (use the AI auto-scribe technique).
 - Keep the repo clean: use **.gitignore**, relative paths, and a clear folder structure.
4. Each member makes a small “hello” commit (e.g., add your name to the **README.md**) and pushes it to confirm access.

Note

Milestone 0 due Friday: Confirm your group formation on Canvas and ensure everyone can commit to the shared repository.

git8.qmd Knowledge Check

1. What is the difference between **saving** a file and **committing** it?

2. What does a red line indicate in a diff?
 3. Which button restores the last **committed** version of a file?
 4. Why should you use a **.gitignore** file?
 5. What is the purpose of a **branch**?
 6. How does a **Pull Request** support peer review?
 7. Why are relative paths essential for reproducibility?
-

git9.qmd Practice Exercise: The Full Workflow Drill

1. Open your Codespace from last week.
 2. Create a **.gitignore** file with sensible defaults for your project. **Commit**.
 3. Create a new branch called **practice-edit**.
 4. On the branch, make a small improvement to your R script (e.g., add a clarifying comment or improve a variable name).
 5. Use an AI model to draft a clear commit message from your diff. **Commit** on the branch.
 6. Push the branch and open a **Pull Request** on GitHub.
 7. Review the PR diff on GitHub, then merge it.
 8. Back on **main**, intentionally introduce a breaking typo and **Save**.
 9. Use **Discard Changes** to recover the last working version.
-

git10.qmd Quick Reference: Common Git Actions in VS Code

Action	How
Stage changes	Source Control → hover file → click +
Commit	Type message → click
View diff	Click file name in Source Control
Discard uncommitted edits	Hover file → click curved arrow (Discard Changes)
Create a branch	Click branch name (bottom-left) → Create new branch...
Switch branches	Click branch name → select target branch
Merge a branch	Command Palette → Git: Merge Branch...
Push / Sync	Click Sync Changes in status bar
Open a Pull Request	Push branch → go to github.com → Compare & pull request
Add to .gitignore	Edit .gitignore in repo root → stage + commit

Concepts

- **The Audit Trail:** Git records *who* changed *what*.



Selecting any line will show who changed that line and when they did so.

- **The Timeline:** A sequence of safe checkpoints (commits). This shows the reason behind each change and what was change. These checkpoints can always be reverted back to if the need be.

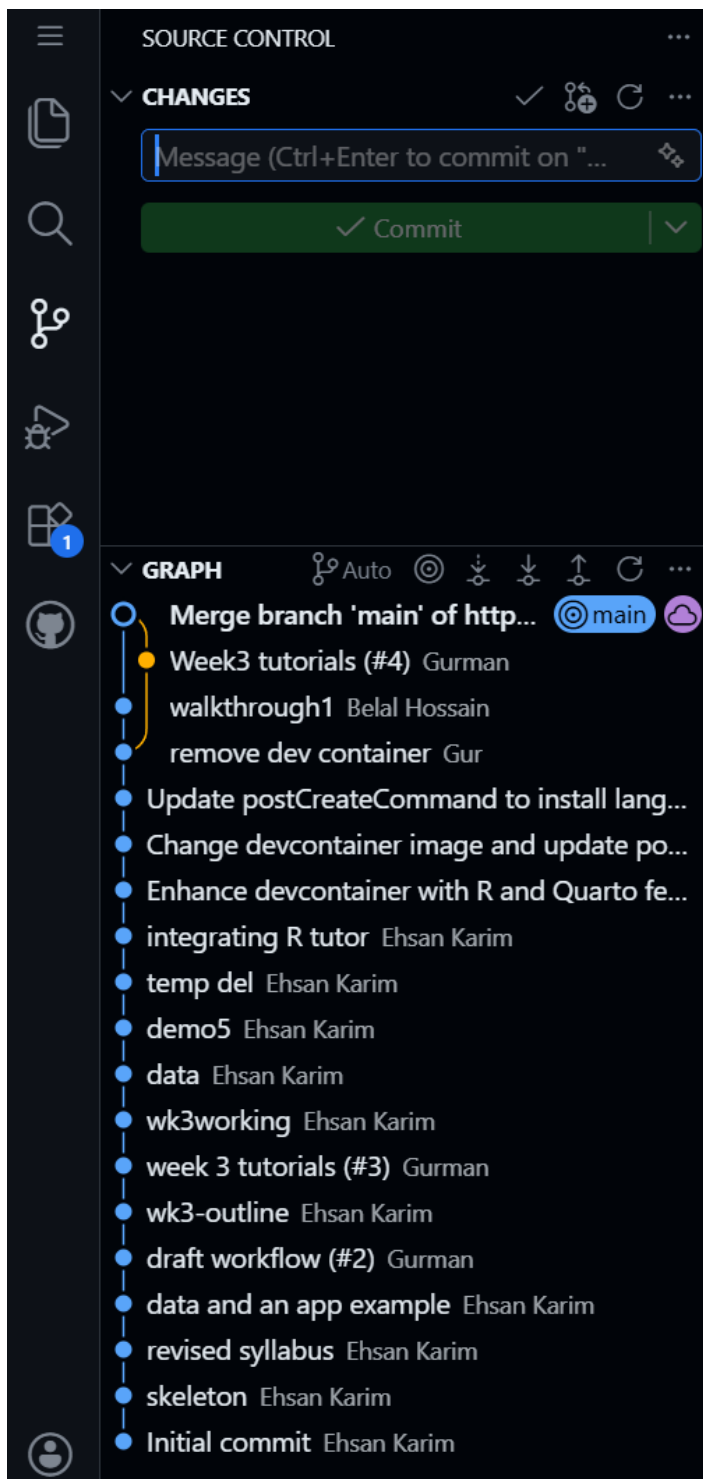
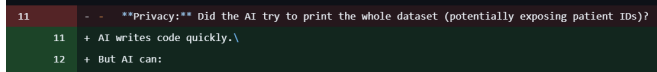


Figure 22.1: This tree shows every commit (checkpoints)

- **The Language of Diffs:**

- Red (-): Removed lines.
- Green (+): Added lines.



```
11 - - **Privacy:** Did the AI try to print the whole dataset (potentially exposing patient IDs)?
12 + AI writes code quickly.
13 + But AI can:
```

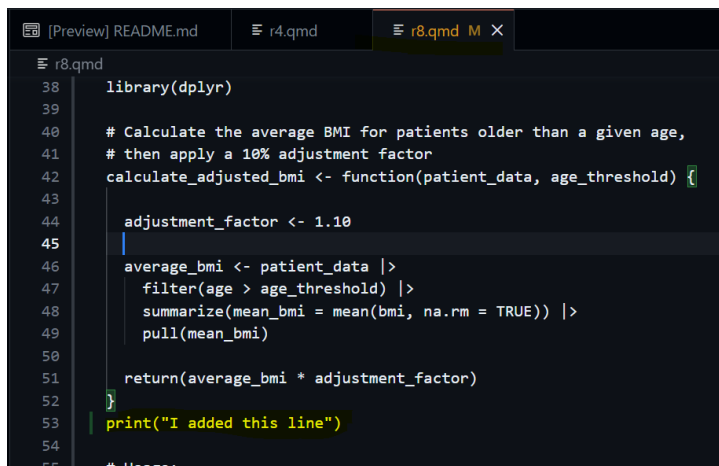
Observing this image, we see that line 11 was replaced, the old data on line 11 is colored red, while the new changes are shown in green.

Key Rule: Saving writes to disk on your commuter. Committing and pushing to the repo writes to history.

Tutorial

Follow these steps

1. **Edit:** Make a change to your code, any file in the repo will do, and save the file.



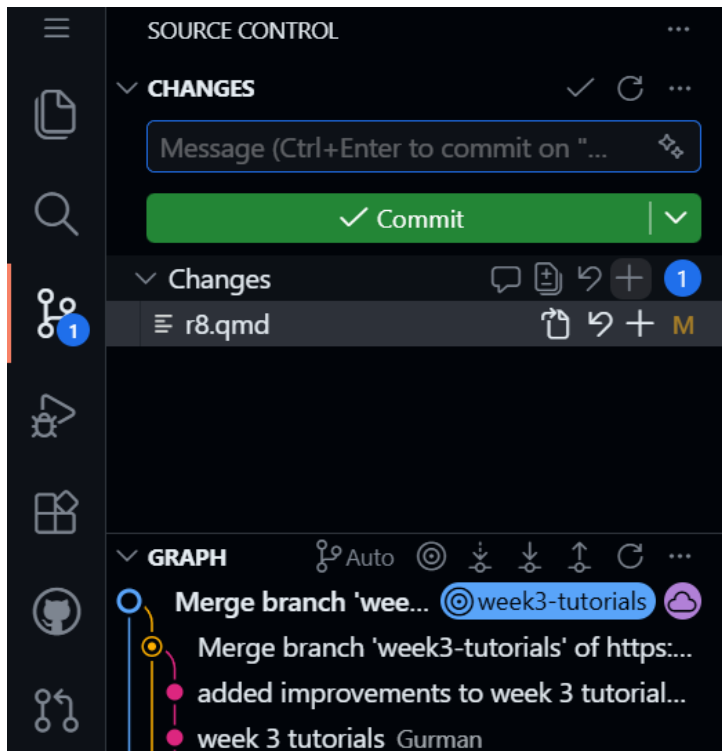
```
38 library(dplyr)
39
40 # Calculate the average BMI for patients older than a given age,
41 # then apply a 10% adjustment factor
42 calculate_adjusted_bmi <- function(patient_data, age_threshold) {
43
44   adjustment_factor <- 1.10
45
46   average_bmi <- patient_data |>
47     filter(age > age_threshold) |>
48     summarize(mean_bmi = mean(bmi, na.rm = TRUE)) |>
49     pull(mean_bmi)
50
51   return(average_bmi * adjustment_factor)
52 }
53 print("I added this line")
54
55 # Usage:
```

Key things to note are that the open file tab has changed colors, this shows that github has acknowledged that the file has been changed.

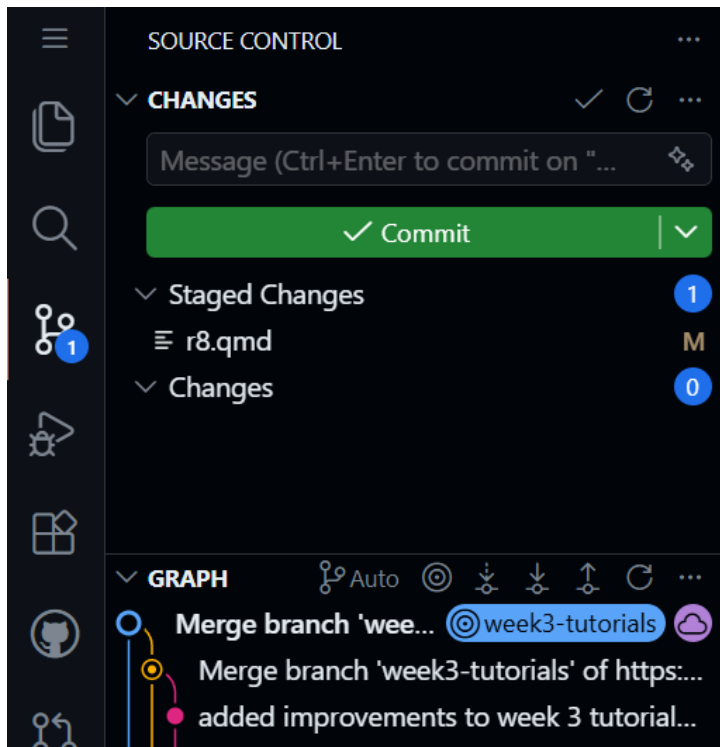
2. **Commit:** Stage and Commit in the Source Control tab.

As we have seen from the previous tutorial, open up the git tab as indicated by the orange-peach highlight. When a file is changed you will be presented with the screen below.

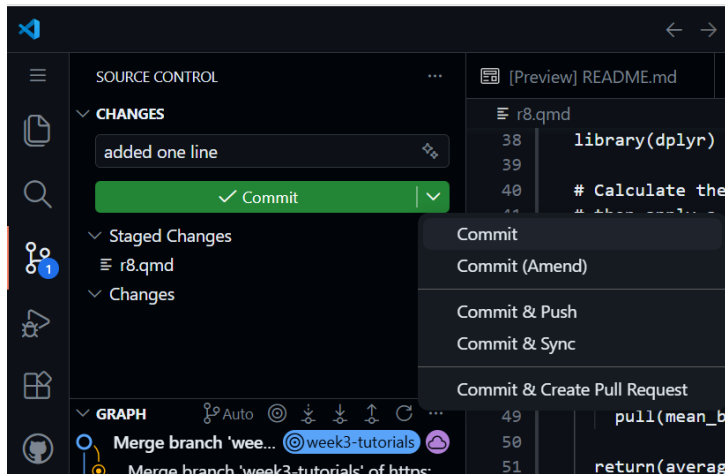
To stage a change, press the + icon.



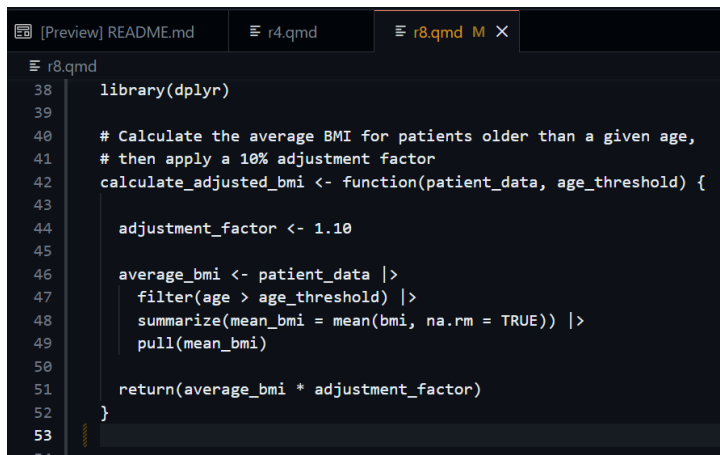
Your menu should look like this now;



Then when you are ready to commit, you can click the big green button! You can also click the down arrow on the big green button to see more options, as well as the commit option. We will discuss those other options later.

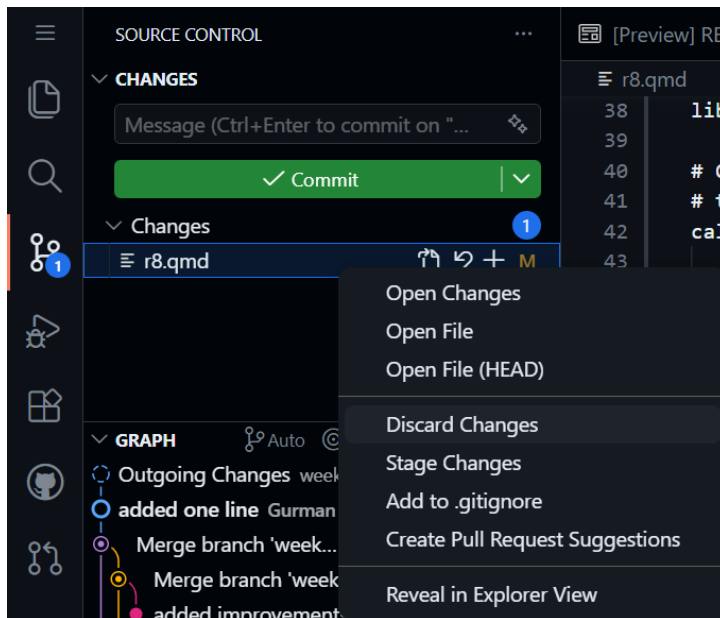


3. **Break:** Delete the code you just added and save.



```
38 library(dplyr)
39
40 # Calculate the average BMI for patients older than a given age,
41 # then apply a 10% adjustment factor
42 calculate_adjusted_bmi <- function(patient_data, age_threshold) {
43
44   adjustment_factor <- 1.10
45
46   average_bmi <- patient_data |>
47     filter(age > age_threshold) |>
48     summarize(mean_bmi = mean(bmi, na.rm = TRUE)) |>
49     pull(mean_bmi)
50
51   return(average_bmi * adjustment_factor)
52 }
53
```

4. **Rescue:** Hover over the file in Source Control again, click the **Discard Changes** option.



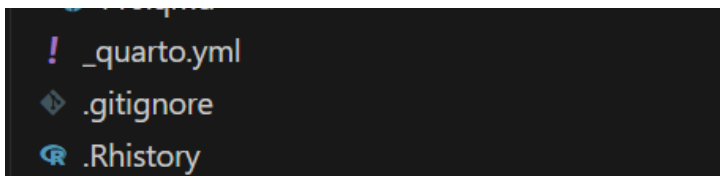
This will revert the file back to what it was before. This can recover data that your computer potentially cannot, its a very powerful in git and showcases one aspect of how git operates. It is a record of all your work and saved chunks.

Repo Hygiene

.gitignore

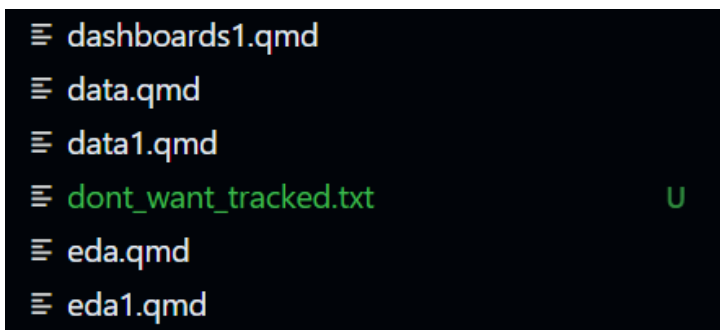
Sometimes there are files you do not want to be tracked by github. This could be anything from files specific to your computer to items like patient data which should not be stored on the cloud unless allowed so by policy.

1. Create a file named **.gitignore** in your root directory:
It has to be this exact spelling for git to recognize it.



You will know its correct when it has its own icon as seen above

2. Then create a new file, any type will do but for this lesson I will create a text file called dont_want_tracked



3. Then open the gitignore file and add the name of the file you dont want tracked into it.

```
.gitignore
1  /.quarto/
2  /_book/
3  /.Rproj.user/
4  *.Rproj
5  .DS_Store
6
7  dont_want_tracked.txt
```

4. Now you will see the new file, which was green is now grayed out. This shows that github is no longer tracking it and coloring it green (which it does for new files).

```
≡ data.qmd
≡ data1.qmd
≡ dont_want_tracked.txt
≡ eda.qmd
≡ eda1.qmd
```

This is simple enough right? Whenever in doubt, ask AI how to stop a file from being tracked by git and it will offer some advice. Now stage and commit this like we have done in the previous tutorial.

Relative Paths

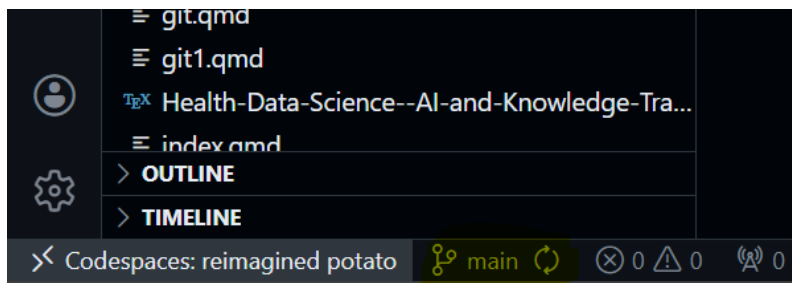
Always use relative paths in your code (e.g., `read.csv("data/nhanes.csv")`), never absolute paths (e.g., `read.csv("/home/user/Documents/nhanes.csv")`). This ensures your analysis runs on any machine—including your grader's Codespace.

Reproducibility rule: Your term project must run from start to finish in a fresh Codespace. Absolute paths guarantee failure.

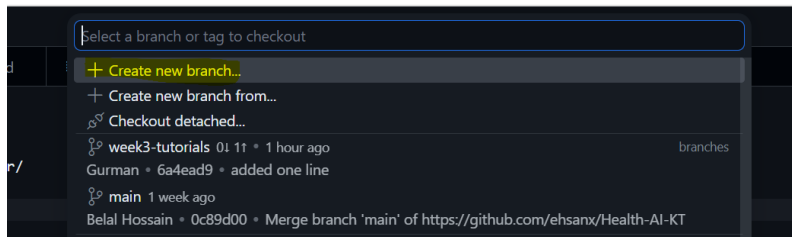
Branching

1. Create:

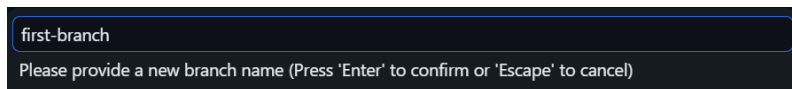
Click branch name (bottom-left).



This will pop up a menu where you have the option to **Create new branch**.



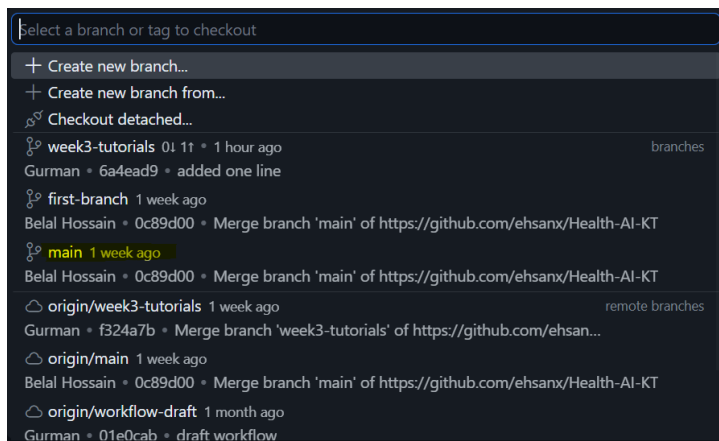
Select this and call it first-branch. Note branch names may not have any spaces and limited special characters, the system will let you know if a name is invalid.



2. Work:

Make changes, like those in the previous tutorial and commit your changes on your branch.

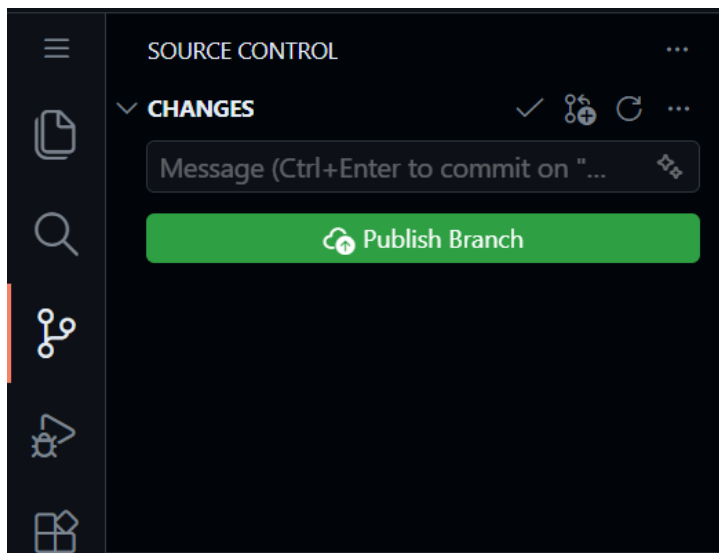
3. **Switch:** Click the branch button on the bottom left again and once this menu pops up, select **main**. Note that your changes vanish (they are safely in the other branch). If you decide to switch back to your branch, your changes will reappear. This is amazing for multitasking on different features and keeping code clean.



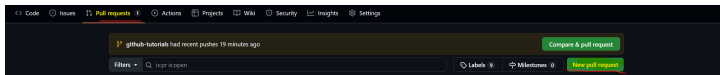
Pull Requests

A **Pull Request (PR)** is a GitHub feature that says: *“I have changes on a branch. Please review them before they go into **main**.”* PRs are how professional teams and researchers collaborate. You will use them for peer review later in this course. This is also the best way to merge your own changes into main, as it forces you to see what has been change and allows one to check what they are submitting to the main branch, the backbone or “trunk” of the tree

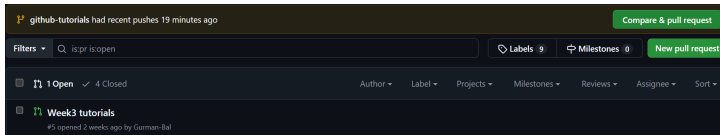
1. **Sync:** Click “Sync Changes” to push your branch to GitHub. This can be blue or green or say publish branch or not. Whichever the case, clicking this button will make everything on your computer update to the online github repo.



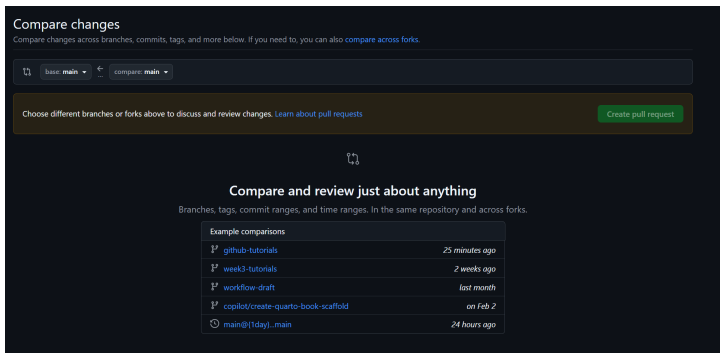
2. **PR:** Go to your repo on github online and click “Compare & pull request.”



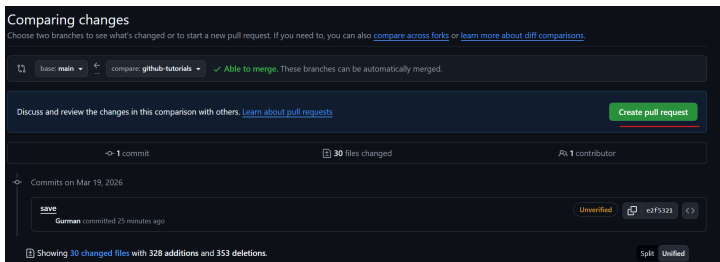
We are then presented with the screen below. Press the big green button “New Pull Request”



Then you are presented this menu. This shows all the different branches that can be merged into main. Select on the one you want to merge into main.



Once you select the one, you can make a pull request! Select this.



3. **Merge:** Review the diff, then click “Merge pull request.” If there are merge conflicts, ask AI the best way to resolve them.

Gurman-Bal and others added 6 commits [last month](#)

week 3 tutorials

Verified8794d84

added improvements to week 3 tutorials

Verified76c47bd

Merge branch 'week3-tutorials' of <https://github.com/ehsanx/Health-AI-KT>

788bc0f

address changes

479773a

Merge branch 'week3-tutorials' of <https://github.com/ehsanx/Health-AI-KT>

f324a7b

Merge branch 'main' into week3-tutorials

Verified17f0242

This branch has not been deployed

No deployments

No conflicts with base branch

Merging can be performed automatically.

Merge pull request

You can also merge this with the command line. [View command line instructions.](#)

Still in progress?

[Convert to draft](#)

148

AI Auditing

Use these prompts with your AI assistant:

- **Audit:** “I am about to commit these changes. Explain what I changed in plain English. Did I delete anything important?”

Use this to spot accidental deletions or unintended edits before you commit.

- **Scribe:** “Based on this diff, write a one-sentence, professional commit message for my Quarto document.”

Check that the message captures the **intent** (e.g., “Filter patient cohort to age 65 per protocol”), not just the action (“changed line 42”).

- **Panic:** “VS Code shows this error: [PASTE ERROR]. What does it mean in simple terms and what buttons should I click?”

Never run random terminal commands suggested by an AI (especially `--force` or `reset --hard`). Prefer the VS Code visual interface, which is safer. If the AI suggests a terminal command, ask it for the GUI equivalent first.

Setting Up Your Group Repository

1. **Invite:** One member creates the repo; others accept the invite.
2. **Clone:** Everyone launches their own Codespace.
3. **Norms:** Agree: “No direct pushes to `main`.”
4. **Test:** Everyone makes a `README.md` update to confirm access.

Knowledge Check

1. **Saving vs. Committing:** Saving = disk; Committing = history.
2. **Red line:** Deletion.
3. **Restore:** Discard Changes button.
4. **.gitignore:** Prevents tracking junk/secrets.
5. **Branch:** Parallel experimental workspace.
6. **PR:** A peer review workflow for merging code.
7. **Relative Paths:** Ensures the project runs on any machine.

Practice Exercise: Full Workflow

Step	Task	Done
1	Edit Code	☐
2	Create Branch	☐
3	AI Draft Msg	☐
4	Stage & Commit	☐
5	Push & PR	☐
6	Merge	☐

Quick Reference

Action	How-To
Stage	Source Control (+)
Commit	Type msg +
View	Click file in Source Control
Diff	
Discard	Left click file in source control and click discard changes
Branch	Bottom-left status bar
Sync	Sync Changes button

Part VII

Week 5: Polyglot Awareness & R Deepening

Python with AI

Overview

Summary: You know how to wrangle data in R (Week 3) and track your history with Git (Week 4). This week, you'll use Large Language Models to translate your existing R skills into Python—running everything inside your GitHub Codespace. You don't need to become a Python expert; you need to be great at *directing the AI* and *verifying the output*.

Learning Objectives:

- Explain when and why a health data scientist might choose Python over R (and vice versa).
- Create and run a Python Jupyter Notebook (`.ipynb`) and a Python script (`.py`) in your Codespace.
- Use the VS Code **Variables** pane and Data Viewer to inspect Python data.
- Map R mental models to Python equivalents (`dplyr` → `pandas`, `ggplot2` → `seaborn/matplotlib`).
- Manage a reproducible Python environment using `requirements.txt`.
- Use `reticulate` to call R from Python (or vice versa) for a simple interoperability demo.
- Avoid “Frankenstein code” via the **One Step, One Cell** AI workflow.
- Apply Week 4 Git skills to track Python translations with meaningful commits.

python1.qmd Concepts: Python vs R—When and Why?

Two Languages, One Codespace

Both R and Python are first-class citizens in your GitHub Codespace. You are not choosing one forever; you are learning when each is the better tool.

Strength	R	Python
Statistical modeling & tidyverse-style wrangling	Mature ecosystem (<code>dplyr</code> , <code>ggplot2</code> , <code>lme4</code>)	Possible, but more verbose
Machine learning & deep learning	Growing (<code>tidymodels</code> , <code>torch</code>)	Dominant (<code>scikit-learn</code> , <code>tensorflow</code> , <code>pytorch</code>)
Dashboards	Shiny (Week 8)	Streamlit, Dash
General-purpose scripting & automation	Less common	Very common

Bottom line for this course: R remains your primary analysis language. Python is introduced so you can (a) read and adapt Python code you encounter in the wild, (b) leverage Python-only libraries when needed, and (c) demonstrate bilingual fluency in your portfolio.

Notebooks vs Scripts

- **Notebook (.ipynb):** Best for exploration, teaching, and narrative analysis. Cells run interactively; output appears inline.
- **Script (.py):** Best for reusable utilities, automation, and clean code. Runs top-to-bottom.

In this week's activities, you will work primarily in a notebook but also create one short `.py` script to see the difference.

python2.qmd Setup: Jupyter in Codespaces

Create and Run a Notebook

1. **New file:** VS Code Explorer → **New File** → name it `translation.ipynb`.
2. **Select kernel:** Notebook toolbar → **Select Kernel** → choose the recommended Python environment.
3. **First cell:** Type `print("Hello from Python")` and press **Shift + Enter**.

The Variables Pane and Data Viewer

After running a cell that creates a variable, open the **Variables** pane (notebook toolbar). Double-click any DataFrame to open an “Excel-like” grid view—your best friend for verifying that data loaded correctly.

Tip

If notebook features (Variables, Data Viewer) are missing, ensure the **Jupyter** extension is enabled in your Codespace. Check the Extensions panel (block icon on the left sidebar).

python3.qmd Packages and Environment Hygiene

R `library()` → **Python** `import`

R	Python equivalent
<code>library(dplyr) / tidyverse</code>	<code>import pandas as pd</code>
<code>library(ggplot2)</code>	<code>import seaborn as sns</code> or <code>import matplotlib.pyplot</code> <code>as plt</code>
<code>library(readr)</code>	<code>pd.read_csv(...)</code> (built into pandas)

Why `requirements.txt` Matters

Your course Codespace comes with core packages pre-installed. But your **term project must run in a fresh Codespace**. If you add a package, you must record it:

1. Open (or create) `requirements.txt` in your repo root.
2. Add the package name and version:

```
pandas==2.2.0
seaborn==0.13.2
```

3. To install from this file: the Codespace setup script runs `pip install -r requirements.txt` automatically on launch.

Warning

Reproducibility rule: Never install packages by running `pip install X` in the terminal and forgetting about it. Always update `requirements.txt`. This is the Python equivalent of R's `renv`—you will see `renv` in Week 11.

Documenting Your Decisions

When you choose a package or approach, add a brief Markdown cell in your notebook explaining *why*. For example: “Using `seaborn` here because it produces cleaner default aesthetics for categorical comparisons than `matplotlib` alone.” This habit satisfies the reproducibility expectation and helps your future self (and your grader).

python4.qmd Translation Traps: What Breaks When You Switch Languages

Trap 1: The Zero-Index

- R indexes from **1**; Python indexes from **0**.
 - R: `my_vec[1]` → first element
 - Python: `my_list[0]` → first element
- Python slicing is **end-exclusive**: `df.iloc[0:5]` returns rows 0–4 (five rows total).

AI Prompt:

Translate `my_vector <- c(10, 20, 30); my_vector[1]` to Python. Explain 0-based indexing and why `my_list[0:2]` returns only two elements.

Trap 2: The Indentation Trap

- R uses `{ }` to define code blocks; Python uses **indentation** (spaces). Misaligned spaces cause `IndentationError`.

AI Prompt:

Translate this R `if/else` block to Python. Explain how indentation replaces `{ }` and show correct formatting for a Jupyter cell.

Trap 3: Assignment and Equality

- R uses `<-` for assignment (though `=` works too); Python uses `=`.
- Both use `==` for comparison, but Python's `None` check is `is None`, not `== None`.

python5.qmd Tutorial: Loading and Viewing Data

Step-by-Step

```
1 import pandas as pd
2
3 # Load (relative path!)
4 data = pd.read_csv("data/patients.csv")
5
6 # Peek at the first 5 rows
7 data.head()
```

Verify in the Data Viewer

After running the cell above, open the **Variables** pane → double-click **data** → confirm the columns and row count match what you expect from your R analysis.

AI Prompt:

I have `data/patients.csv` in my Codespace. Write the Python pandas code to import it and show the first 5 rows. Also show how to check the number of rows and column names.

python6.qmd Reading Tracebacks (Bottom-Up)

Python errors display a **Traceback**—a stack of function calls. The most useful information is at the **bottom**.

Example

```
-----
FileNotFoundError                                Traceback (most recent call last)
Cell In[3], line 2
      1 import pandas as pd
----> 2 data = pd.read_csv("Data/patients.csv")

FileNotFoundError: [Errno 2] No such file or directory: 'Data/patients.csv'
```

Reading it: The bottom line tells you the file was not found. The capital “D” in “Data/” is the culprit—file paths are case-sensitive on Linux (your Codespace runs Linux).

AI Prompt:

Here is a Python Traceback from my Codespace:
[paste]. Read it bottom-up, explain the error in one sentence, and give corrected code.

python7.qmd The AI Workflow: One Step, One Cell

Rule 1: One Step, One Cell

Ask your AI for **one atomic step** at a time: load data, then preview, then filter, then summarize. Run and verify each cell before requesting the next. This prevents “Frankenstein code”—a giant block of AI-generated code where one hidden error cascades through everything.

Rule 2: Reverse Translation

When AI gives you unfamiliar Python, paste it back and ask:

Explain this Python code step by step as if translating it back to R's `dplyr`. What does each line do?

This converts opaque Python into your familiar mental model.

Rule 3: Parity Audit

After each translated step, compare Python output to your R output:

Check	R	Python
Row count	<code>nrow(df)</code>	<code>len(df)</code> or <code>df.shape[0]</code>
Column names	<code>names(df)</code>	<code>df.columns.tolist()</code>
Unique levels	<code>unique(df\$col)</code>	<code>df['col'].unique()</code>
Summary stats	<code>summary(df\$col)</code>	<code>df['col'].describe()</code>

If the numbers don't match, something went wrong in translation. Fix it before moving on.

python8.qmd Interoperability: A Taste of `reticulate`

Why Interoperability?

Sometimes you don't want to rewrite everything. The R package `reticulate` lets you call Python from an R session (and access Python objects as R objects). This is conceptual exposure—you are not expected to build full projects this way, but you should know it exists.

Demo (in an R Quarto document or R console)

```
1 library(reticulate)
2
3 # Run a line of Python from R
4 py_run_string("import pandas as pd; df = pd.read_csv('data/patients.csv')")
5
6 # Access the Python object in R
7 py_df <- py$df
8 head(py_df)
```

Note

This demo runs inside your Codespace where both R and Python are available. You do not need to install anything locally. The goal is awareness: when you encounter a Python-only package in a future project, you know you can call it from R without rewriting your entire pipeline.

python9.qmd Creating a Python Script (.py)

Not everything belongs in a notebook. When you have a reusable utility (e.g., a data-cleaning function), a .py script is cleaner.

Walkthrough

1. In VS Code Explorer, create a new file: `helpers.py`.
2. Write a simple function:

```
1 import pandas as pd
2
3 def load_and_preview(filepath, n_rows=5):
4     """Load a CSV and return the first n rows."""
```

```

5     df = pd.read_csv(filepath)
6     print(f"Loaded {len(df)} rows, {len(df.columns)} columns")
7     return df.head(n_rows)

```

3. In your notebook, import and use it:

```

1 from helpers import load_and_preview
2
3 preview = load_and_preview("data/patients.csv")
4 preview

```

4. Stage and commit both files with a descriptive message.



Tip

When to use which: Notebook for exploratory analysis and narrative. Script for functions you will reuse across notebooks or projects.

python10.qmd Git Integration: Track Your Translations

Apply your Week 4 skills:

1. After completing each major translation step, **stage and commit** with a meaningful message.
2. Use AI to draft commit messages from your diffs (Week 4, `git6b.qmd`).
3. If AI-generated code breaks your notebook badly, use **Discard Changes** (curved arrow in Source Control) to revert to your last commit.



Warning

Jupyter notebooks and Git: `.ipynb` files contain cell outputs (images, tables) in their JSON. This makes

diffs noisy. Consider clearing outputs before committing:
Command Palette → **Notebook: Clear All Outputs**
→ then stage and commit. This keeps your diffs readable.

python11.qmd Term Project: Milestone 1

Due this Friday: Submit your project proposal on Canvas.

Your proposal should include:

- **Research question:** A clear, focused question about health data.
- **Data source:** Which dataset you will use (e.g., NHANES, a Kaggle health dataset). Confirm it is accessible and appropriately sized.
- **Feasibility check:** Can you load and preview the data in a Codespace? (Try it before submitting.)
- **Planned approach:** A brief sketch of what analyses and visualizations you envision.

Tip

Use what you learned this week: load your candidate dataset in *both* R and Python to confirm it works. This early check prevents surprises later.

python12.qmd Knowledge Check

1. Name one scenario where you would choose Python over R, and one where R is stronger.
2. What is the difference between a `.ipynb` notebook and a `.py` script?

3. Why does `my_list[1]` in Python return the *second* element?
 4. How do you open the Data Viewer for a DataFrame in VS Code?
 5. What is `requirements.txt` and why does it matter for reproducibility?
 6. What does `reticulate` allow you to do?
 7. Why should you clear notebook outputs before committing?
 8. What is the “One Step, One Cell” rule, and what problem does it prevent?
-

python13.qmd Practice Exercise: The Bilingual Translation

Task: Create a Python version of your Week 3 R analysis inside your Codespace.

Workflow

1. Create `translation.ipynb` and select the Python kernel.
2. Use your LLM to translate your R steps to Python (`pandas`) **one cell at a time**.
3. After each cell, verify output using the **Variables / Data Viewer** and run parity checks (row count, column names, summary stats) against your R results.
4. Extract at least one reusable function into a `helpers.py` script. Import and use it in your notebook.
5. Add a Markdown cell at the top documenting: which AI model you used, and any decisions you made (e.g., “Used `seaborn` instead of `matplotlib` for cleaner default bar charts”).
6. Clear notebook outputs (**Command Palette** → **Notebook: Clear All Outputs**).

7. **Git:** Stage both files, write a professional commit message (use AI to draft it from the diff), **Commit**, then **Sync Changes**.

python14.qmd Quick Reference

R	Python	Notes
<code>library(dplyr)</code>	<code>import pandas as pd</code>	Load data manipulation library
<code>read.csv("data/f.csv")</code>	<code>pd.read_csv("data/f.csv")</code>	Always use relative paths
<code>head(df)</code>	<code>df.head()</code>	First 5 rows
<code>nrow(df)</code>	<code>len(df)</code>	Row count
<code>names(df)</code>	<code>df.columns.tolist()</code>	Column names
<code>df\$col</code>	<code>df['col']</code>	Access a column
<code>filter(df, age > 65)</code>	<code>df[df['age'] > 65]</code>	Row filter
<code>mutate(df, bmi = wt/ht^2)</code>	<code>df['bmi'] = df['wt'] / df['ht']**2</code>	New column
<code>summary(df\$col)</code>	<code>df['col'].describe()</code>	Summary statistics
<code>unique(df\$col)</code>	<code>df['col'].unique()</code>	Unique values
<code>ggplot(df, aes(x, y)) + geom_point()</code>	<code>sns.scatterplot(data=df, x='x', y='y')</code>	Scatter plot

23 Tutorial 1: Python vs. R - When and Why?

23.1 Overview

In the health sciences, you don't need to choose one language forever; you are learning when each is the better tool for the job. While R remains your primary language for statistical analysis, Python is essential for machine learning and general-purpose automation.

23.1.1 Two Languages, One Codespace

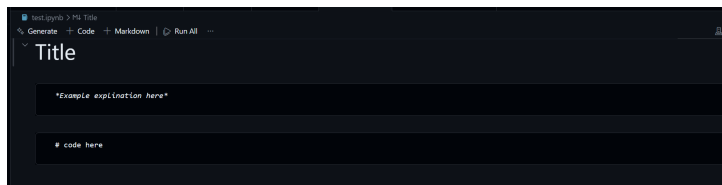
Both R and Python are citizens in your GitHub Codespace. You are learning “bilingual fluency” to:

1. Read and adapt Python code found in research papers.
2. Leverage Python-only libraries for deep learning.
3. Automate repetitive tasks with general-purpose scripting.

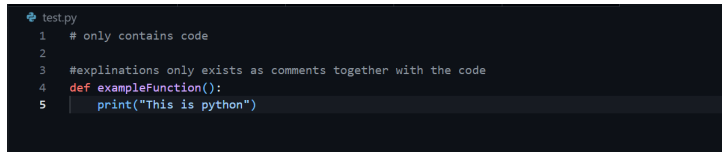
Strength	R	Python
Stats & Wrangling	Mature (dplyr, ggplot2)	Possible, but still growing
Machine Learning	Growing (tidymodels)	Dominant (scikit-learn)
Automation	Less common	Very common

23.1.2 Notebooks vs. Scripts

- **Notebooks (.ipynb):** Best for exploration and teaching where output appears inline.



- **Scripts (.py):** Best for reusable utilities and automation.

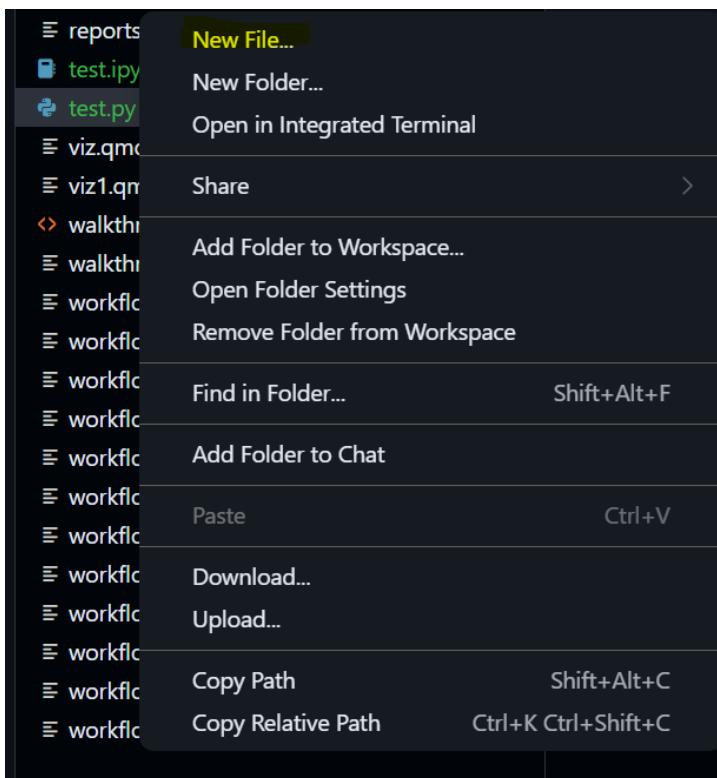


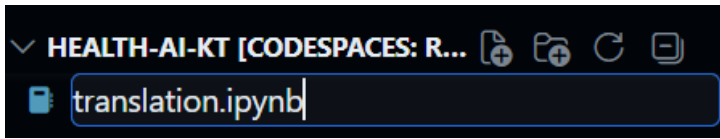
24 Tutorial 2: Setting Up Your Python Environment

24.1 Creating Your First Notebook

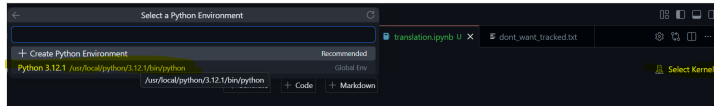
Follow these steps to set up your environment in your GitHub Codespace:

1. **New File:** In the VS Code Explorer, click the **New File** icon and name it `translation.ipynb`.

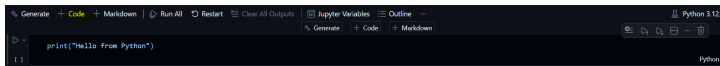




2. **Select Kernel:** Look at the notebook toolbar in the top right. Click **Select Kernel** and choose the recommended **Python 3.x** environment.



3. **Run Code:** Build `print("Hello from Python! My workspace is ready.")` Press **Shift + Enter** to run it.



24.2 The Variables Pane

Unlike RStudio, which shows variables by default, you must toggle the view in VS Code. 1. Click the **Variables** button in the notebook toolbar. 2. Run a cell that creates a variable (e.g., `age = 25`). 3. Observe `age` appearing in the pane at the bottom.

Generate

Code

Markdown

Run All

Restart

Clear All Outputs

Jupyter Variables

Outline

Generate

Code

Markdown

print("Hello from Python")
x = "This variable"

[1] ✓ 0.0s

... Hello from Python

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

JUPYTER

JUPYTER VARIABLES

Name	Type	Size	Value
x	str	13	'This variable'

25 Tutorial 3: Packages and Reproducibility

25.1 R `library()` vs. Python `import`

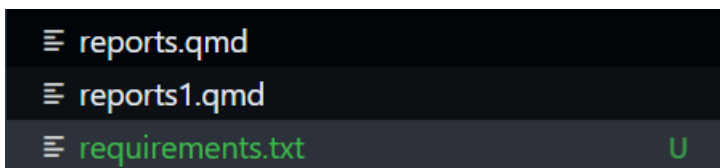
In Python, we “import” packages and often give them nicknames (aliases).

R	Python Equivalent	Purpose
<code>library(dplyr)</code>	<code>import pandas as pd</code>	Data wrangling
<code>library(ggplot2)</code>	<code>import seaborn as sns</code>	Visualization

25.2 Managing `requirements.txt`

Your project must be reproducible. If you use a package, you must record it in a file called `requirements.txt`.

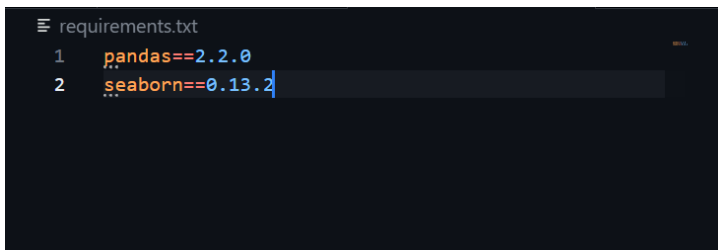
1. Create a file named `requirements.txt` in your root folder.



2. Add:

```
pandas==2.2.0
```

```
seaborn==0.13.2
```



```
requirements.txt
1 pandas==2.2.0
2 seaborn==0.13.2
```

Certain versions of these packages behave differently. By explicitly setting your versions, you can make sure your code will have reproducible results regardless of who is running it.

3. **The Rule:** Never just `pip install` (method on installing packages, which you wouldn't have to deal with in this class but perhaps the future) in the terminal and forget it; always update this file to document your steps.

26 Tutorial 4: Common Translation Traps

26.1 Trap 1: The Zero-Index

R starts counting at **1**. Python starts counting at **0**.

R: `data[1]` is the first item. Note `data[0]` will return nothing

```
> list <- list(1,2,3,4)
> list[1]
[[1]]
[1] 1
```

Python: `data[0]` is the first item.

```
list = [1,2,3,4]
list[0]
✓ 0.0s
1
```

Slicing: `df.iloc[0:5]` returns rows 0, 1, 2, 3, and 4 (5 rows total).

26.2 Trap 2: Indentation

Python uses **indentation** (spaces) instead of curly braces `{}` to group code. Misaligned spaces will cause an `IndentationError`.

```
def test():  
    print("This will not throw an error")  
test()  
✓ 0.0s
```

This will not throw an error

```
def test():  
    print("This will throw an error")  
test()  
✗ 0.0s
```

Cell In[5], line 2
 print("This will throw an error")
 ^
IndentationError: expected an indented block

26.3 Trap 3: Assignment

- **R:** Uses `<-` (or `=`).
- **Python:** Uses `=` exclusively for assignment.

27 Tutorial 5: Loading and Inspecting Health Data

27.1 Loading Data with Pandas

In your notebook, use the following code to load your health data:

```
1 import pandas as pd
2 data = pd.read_csv("data/patients.csv")
```

The code below allows you to view the dataset

```
1 data.head()
```

	day	date	state.in_hospital.in	state.in_hospital.out	state.in_hospital.todate	state.in_hospital
0	1	2020-03-04	NaN	NaN	NaN	NaN
1	2	2020-03-05	NaN	NaN	NaN	NaN
2	3	2020-03-06	NaN	NaN	NaN	NaN
3	4	2020-03-07	NaN	NaN	NaN	NaN
4	5	2020-03-08	NaN	NaN	NaN	NaN



```
import pandas as pd
data = pd.read_csv("data/patients.csv")
data.head()
```

✓ 0.0s

1 2 3 4 5

28 Reading Tracebacks

28.1 How to Read Errors

Python errors (Tracebacks) display a stack of calls.

28.1.1 Common Example:

`FileNotFoundError: [Errno 2] No such file or directory: 'Data/patients.csv'`

```
955 handle = open(handle, 'a')
FileNotFoundError: [Errno 2] No such file or directory: 'Data/patients.csv'
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

The first part of the message tells you what the error is, in this case the file was not found

`FileNotFoundError`

The second part of the error tells you what is missing

`No such file or directory: 'Data/patients.csv'`

At the top of the trace, it tells you exactly where in your code the error was found.

```
FileNotFoundError                                Traceback
Cell In[10], line 2
      1 import pandas as pd
----> 2 data = pd.read_csv("Data/patients.csv")
      3 data.head()
```

The file wasn't found. Because file paths are case-sensitive, `Data/` is different from `data/`.

29 Tutorial 7: The ‘One Step, One Cell’ AI Workflow

29.1 Rule 1: One Step, One Cell

Don’t ask the AI for a giant analysis script at once. Ask for **one atomic step** (load, then filter, then summarize). Run and verify each cell before moving on.

First ask AI to load

```
Python 📄  
  
from sklearn.datasets import load_iris  
import pandas as pd  
  
# Load the data  
iris = load_iris()  
  
# Convert to a DataFrame for easier viewing  
df = pd.DataFrame(data=iris.data, columns=iris.feature_names)  
df['target'] = iris.target  
  
print(df.head())
```

Then ask it to filter by some value

```
# Filter for flowers with petals longer than 5.0 cm  
long_petals = df[df['petal_length'] > 5.0]  
  
print(long_petals.head())
```

Then ask it to summarize

```
# General stats for petal and sepal measurements  
print(df.describe())
```

29.2 Rule 2: Parity Audit

Observe some of the differences between R and python below when it comes to data wrangling.

Check	R	Python
Row count	<code>nrow(df)</code>	<code>len(df)</code>
Column names	<code>names(df)</code>	<code>df.columns.tolist()</code>
Summary stats	<code>summary(df\$col)</code>	<code>df['col'].describe()</code>

30 Tutorial 8: Mixing R and Python”

30.1 Why Interoperability?

The R package `reticulate` lets you call Python from an R session. This is perfect when you need a Python-only package but want to keep your analysis in R.

First define the package as seen below

```
library(reticulate)

# Use a specific conda environment (recommended)
# use_condaenv("my_env")

# Or check if python is available
py_config()
```

Then build python code as shown

```
# Define a Python block to load and filter the iris dataset
py_run_string("
import seaborn as sns
import pandas as pd

# Load the dataset
iris_py = sns.load_dataset('iris')

# Filter for long petals
long_petals = iris_py[iris_py['petal_length'] > 5.0]
")
```

Then you are able to use the results of that python code in R! Super cool!

```

# Access the Python dataframe as an R dataframe
iris_r <- py$long_petals

# Now you can use R's head() function on the Python data
head(iris_r)

# Use R's ggplot2 to plot the data processed by Python
library(ggplot2)
ggplot(iris_r, aes(x = species, y = petal_width, fill = species)) +
  geom_boxplot() +
  labs(title = "Petal Width for Long-Petal Species",
        subtitle = "Data processed in Python, plotted in R")

```

30.1.1 Demo (Run in R)

```

1 #library(reticulate) # Run Python code inside R
2 #py_run_string("import pandas as pd; df = pd.read_csv('data/patients.csv')")

```

Access that Python object as an R object

```

1 #py_df <- py$df
2 #head(py_df)

```

31 Tutorial 9: Creating Reusable Python Scripts

31.1 When to Use a Script

Use a `.py` script for reusable functions (like data cleaning) that you want to use across different notebooks.

1. Create a new file: `helpers.py`.
2. Add a cleaning function:

```
1  # # weeks/week05-polyglot-r-deepening/helpers.py
2  # from pathlib import Path
3  # import pandas as pd
4  #
5  # def load_and_preview(path, n=5):
6  #     p = Path(path)
7  #     if not p.exists():
8  #         raise FileNotFoundError(f"Could not find file: {p.resolve()}")
9  #     df = pd.read_csv(p)
10 #     print(df.head(n))
11 #     return df
```

3. **Import it** in your notebook:

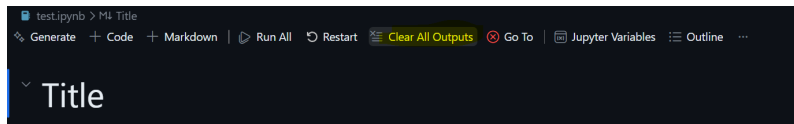
```
1 # from helpers import load_and_preview
2 # load_and_preview("data/patients.csv")
```

32 Tutorial 10: Tracking Your Progress with Git

32.1 Cleaning Your Notebooks

Jupyter notebooks (.ipynb) store images and tables inside the file, making Git history messy.

Before you commit: 1. Open the **Command Palette** (Ctrl+Shift+P). 2. Search for **Notebook: Clear All Outputs**. 3. Stage and commit your changes with a professional message.



33 Tutorial 11: Term Project Milestone 1

33.1 Submission Checklist

Your proposal is due this Friday. Ensure it includes:

- **Research Question:** A focused health data question.
- **Data Source:** A confirmed dataset (NHANES, Kaggle, etc.).
- **Feasibility Check:** Can you load and preview the data in a Codespace?

Pro-Tip: Load your dataset in **both** R and Python now to ensure there are no surprises later in the term.

34 Tutorial 12: Python Knowledge Check

Answer these in a new Markdown cell in your notebook:

1. Why does `my_list[1]` in Python return the *second* element?
2. What is `requirements.txt` and why is it essential for reproducibility?
3. Why should you clear notebook outputs before committing to Git?
4. What is the “One Step, One Cell” rule?

35 Tutorial 13: The Bilingual Translation Exercise

35.1 Cleaning Your Notebooks

Task: Create a Python version of your Week 3 R analysis.

1. Create `translation.ipynb`.
2. Use an LLM to translate your R steps to Python **one cell at a time**.
3. After each cell, verify output using the **Data Viewer**.
4. Extract one function into `helpers.py`.
5. Clear outputs and commit to Git.

36 Tutorial 14: R to Python Cheat Sheet

Task	R	Python
Load Library	<code>library(dplyr)</code>	<code>import pandas as pd</code>
Load CSV	<code>read.csv("data/f.csv")</code>	<code>pd.read_csv("data/f.csv")</code>
Row Count	<code>nrow(df)</code>	<code>len(df)</code>
Filter	<code>filter(df, age > 65)</code>	<code>df[df['age'] > 65]</code>
New Column	<code>mutate(df, bmi = w/h^2)</code>	<code>df['bmi'] = df['w'] / df['h']**2</code>
Summary	<code>summary(df\$col)</code>	<code>df['col'].describe()</code>

Part VIII

Week 6: Data Visualization

Visual Storytelling

Overview

This chapter challenges students to move beyond default charts and create publication-quality visualizations. You will use AI tools to generate initial code for complex plots in ggplot2 or plotly, and then apply human judgment to refine them for clarity, accuracy, and aesthetics. The lesson emphasizes detecting misleading AI-generated visuals and “steering” the code to tell a truthful and compelling data story.

TODO Before Release

Content TODO

- Add a worked ggplot2 example using the shared NHANES data.
- Add one intentionally misleading AI-generated chart for students to audit.
- Add a plotly or interactive chart extension.
- Add Assignment 5 instructions in [assignments/assignment05-visualization/](#).
- Add expected captions and accessibility checks for figures.

Definition of Done

- Students can generate, critique, revise, and caption at least one health data visualization.
- The activity includes a concrete visual audit checklist.

- All example plots render in a fresh Codespace.

Activity: AI Visual Audit

Goal

Students evaluate AI-generated visualizations for correctness, clarity, bias, aggregation choices, and misleading design.

TODO Before Release

Content TODO

- Add a starter dataset and prompt.
- Add one flawed AI-generated plot and one corrected version.
- Add a short answer template for students to identify what changed and why.

Student Output

- One original AI-generated plot.
- One revised plot.
- A short audit note explaining at least three changes.

Part IX

Week 7: EDA, Table 1 & AI Auditing

Exploring Data

Overview

Before modeling, you must understand your data. This chapter guides students through the rigorous process of Exploratory Data Analysis (EDA) and the creation of the standard “Table 1” for demographic characteristics. You will learn to document every cleaning decision and exclusion criteria in a reproducible Quarto log, ensuring that your path from raw data to analysis dataset is transparent and repeatable by anyone.

TODO Before Release

Content TODO

- Build a worked EDA using the NHANES example files in `examples/data/`.
- Add Table 1 expectations and an example table.
- Add a planted-error AI audit exercise.
- Add Assignment 6 instructions in `assignments/assignment06-eda-ai-audit/`.
- Connect the activity to Milestone 2 preliminary analysis.

Definition of Done

- Students can document cleaning decisions from raw or starter data to an analysis-ready table.
- The planted-error exercise has an instructor key.
- The EDA renders without manual intervention.

Activity: Table 1 and AI Audit

Goal

Students create a descriptive summary and audit AI-generated code or interpretation for planted errors.

TODO Before Release

Content TODO

- Add the starter data path.
- Add the expected Table 1 variables.
- Add planted errors in code, summary text, and at least one visualization.
- Add an answer key for instructors or TAs outside the student-facing page.

Student Output

- A rendered Quarto EDA note.
- A Table 1 or equivalent descriptive summary.
- A written audit note listing detected errors and fixes.

Part X

Week 8: Dashboard Prototypes for KT

Interactive Dashboards

Overview

Static PDF reports often sit unread, so this chapter introduces dynamic Knowledge Translation through Shiny apps and dashboards. Students will take their prior EDA results and transform them into interactive web tools that allow users to filter and explore the data themselves. You will learn the best practices of user interface design for health data and deploy your first live dashboard to the web via GitHub Pages or Shinylive.

TODO Before Release

Content TODO

- Decide the required core pathway: Quarto dashboard, Shiny, or static HTML/JS.
- Add a minimal dashboard starter with one filter, one chart, and one plain-language interpretation.
- Add testing instructions for Codespaces.
- Add Assignment 7 instructions in `assignments/assignment07-dashboard/`.
- Add guidance for privacy-safe dashboard publishing.

Definition of Done

- Students can produce a small dashboard-style KT artifact from prior EDA outputs.
- The dashboard runs or renders in the supported workflow.

- The page distinguishes required core work from optional deployment extensions.

Starter Dashboard

Goal

Students adapt a minimal dashboard-style knowledge translation product.

TODO Before Release

Content TODO

- Add the starter dashboard code.
- Add a screenshot or rendered preview.
- Add a fallback static version for students who cannot deploy.
- Add a checklist for audience, filter labels, chart captions, and limitations.

Required Elements

- One audience-specific question.
- One interactive or dashboard-style control.
- One visualization.
- One plain-language interpretation.
- One limitation or caution.

Part XI

Week 9: Communication of Scientific Findings

Dynamic Presentations

Overview

Data analysis is useless if it cannot be understood by key stakeholders. This chapter focuses on “Audience-Specific KT,” teaching you to convert technical findings into plain language summaries and engaging slide decks using Quarto. You will practice peer review and learn to use LLMs to simplify jargon without losing scientific accuracy, ensuring your work can impact policy and practice.

TODO Before Release

Content TODO

- Add a revealjs starter deck.
- Add a plain-language summary exercise.
- Add a peer feedback template.
- Add Assignment 8 instructions in `assignments/assignment08-communication/`.
- Add slide accessibility and citation expectations.

Definition of Done

- Students can render a short Quarto slide deck.
- Each slide has one clear purpose.
- Peer feedback checks clarity, evidence, limitations, and KT value.

Revealjs and Peer Feedback

Goal

Students turn analysis outputs into a short scientific communication product and practice structured peer feedback.

TODO Before Release

Content TODO

- Add a minimal `format: revealjs` YAML example.
- Add a three-slide starter structure.
- Add a peer feedback form.

Peer Feedback Prompts

- What is the main message?
- What evidence supports it?
- What is unclear or overclaimed?
- What would improve reproducibility or audience fit?

Part XII

Week 10: Writing & Publishing Reports

Reproducible Reporting

Overview

This chapter simulates the final stretch of a research project: writing the manuscript. Students learn to assemble their analysis, code, and writing into a single “dynamic report” that automatically updates numbers and figures if the data changes. You will explore advanced publishing workflows to create professional, citation-ready portfolios that demonstrate your ability to execute end-to-end data science.

TODO Before Release

Content TODO

- Add a report skeleton with sections for question, data, methods, findings, limitations, and AI-use note.
- Add citation examples using `references.bib`.
- Add GitHub Pages setup screenshots or a private submission alternative.
- Add Assignment 9 instructions in `assignments/assignment09-reporting/`.
- Add a citation and grounding audit.

Definition of Done

- The report renders from source.
- Citations resolve.

- Outputs are regenerated by code rather than pasted manually.
- The README explains how to reproduce the report.

Report Template

TODO Before Release



Content TODO

- Convert this into a copyable Quarto report starter.
- Add bibliography setup instructions.
- Add a small example table and figure.
- Add an AI-use note template.

Suggested Sections

- Research or KT question
- Data source and stewardship notes
- Methods and reproducibility notes
- Results
- Limitations
- Plain-language summary
- AI-use and audit note

Part XIII

Week 11: Portfolio Surgery

Advanced Topics

Overview

Before the final submission, students must “bulletproof” their work. This chapter focuses on advanced project hygiene, dependency management (ensuring your code runs on someone else’s machine), and auditability. You will act as a peer reviewer to identify reproducibility failures in others’ code, fixing broken paths and missing seeds, to ensure your final project stands up to scientific scrutiny.

TODO Before Release

Content TODO

- Add a fresh Codespace stress-test protocol.
- Add a broken-project debugging exercise.
- Add Assignment 10 instructions in `assignments/assignment10-portfolio-surgery/`.
- Add Milestone 4 peer review instructions.
- Add a final portfolio readiness checklist.

Definition of Done

- A peer can clone or open the repository in a fresh Codespace and reproduce the core outputs.
- Broken paths, missing dependencies, unclear READMEs, and ungrounded AI claims are fixed.
- The final portfolio checklist is linked from this page.

Reproducibility Stress Test

Goal

Students test whether a project can run in a fresh environment and document every failure before final submission.

TODO Before Release

Content TODO

- Add step-by-step fresh Codespace test instructions.
- Add a failure log template.
- Add peer review roles and timing.
- Add final repair checklist.

Failure Log Template

Check	Result	Fix needed	Owner
Repository opens in fresh Codespace			
Packages install or restore			
Report renders			
Dashboard runs or renders			
README instructions are accurate			

Part XIV

Weeks 12-13: Project Presentations

Week 12: Term Project Presentations, Part 1

Overview

This session is the first presentation block for Milestone 5. Students present their portfolio work with an emphasis on scientific clarity, knowledge translation, reproducibility, and auditability.

TODO Before Release



Content TODO

- Add final presentation timing rules.
- Add slide submission instructions.
- Add the peer feedback form or link.
- Add the presentation rubric from [milestones/m5-presentation/](#).

Definition of Done

- Slides render before class.
- The project repository link is available to reviewers.
- The presenter can explain the dataset, workflow, major findings, limitations, and AI-use audit trail.

Week 13: Term Project Presentations, Part 2

Overview

This session completes Milestone 5 presentations and gives students a final opportunity to collect feedback before the Final Portfolio deadline.

TODO Before Release

Content TODO

- Confirm which groups present in Part 2.
- Add final reminders for the Dec 11 portfolio submission.
- Add a closing reflection prompt on reproducibility and knowledge translation.

Definition of Done

- Remaining presentations are complete.
- Peer feedback has been returned to presenters.
- Students know exactly what must change before final portfolio submission.

Part XV

Assignments and Milestones

Assignments

Weekly assignments are scaffolded practice. Assignments 1-3 are foundational gates; Assignments 4-10 support the project portfolio and use the best-5-of-7 grading policy described in the syllabus.

Assignment Map

Assignment	Week	Focus	Template folder
Assignment 1	2	Codespaces, Quarto, commit/sync	assignments/assignment01-workflows/
Assignment 2	3	R with AI, tidyverse, functions, renv	assignments/assignment02-r-with-ai/
Assignment 3	4	Git collaboration and repo hygiene	assignments/assignment03-git/
Assignment 4	5	Polyglot awareness and R deepening	assignments/assignment04-polyglot/
Assignment 5	6	Visualization and AI visual audit	assignments/assignment05-visualization/
Assignment 6	7	EDA, Table 1, planted-error audit	assignments/assignment06-eda-ai-audit/

Assignment	Week	Focus	Template folder
Assignment 7	8	Dashboard-style KT prototype	<code>assignments/assignment07-dashboard/</code>
Assignment 8	9	Scientific communication and peer feedback	<code>assignments/assignment08-communication/</code>
Assignment 9	10	Reporting, citations, publishing	<code>assignments/assignment09-reporting/</code>
Assignment 10	11	Portfolio surgery and reproducibility test	<code>assignments/assignment10-portfolio-surgery/</code>

TODO

Warning

Create a student-facing `README.md` in each assignment folder from `assignments/assignment-template.qmd`.

Assignment Template

Use this template for all weekly applied assignments.

Student-Facing Brief

- **Purpose:**
- **Inputs:**
- **Expected output:**
- **Submission location:**
- **Due date:**

Required Files

- README.md
- One rendered Quarto output or dashboard preview.
- Source .qmd, .R, .py, or supporting files needed to rerun the work.
- Data files, or clear instructions for obtaining them, when redistribution is not allowed.

Reproducibility Contract

- The work runs in a fresh GitHub Codespace.
- Paths are relative to the project root.
- Required packages are declared.
- Outputs can be regenerated without manual editing.
- AI-generated code or text is audited and documented.

Grading Checklist

- Clear question or task.
- Correct and reproducible code.
- Evidence of verification.
- Clear writing and captions.
- Honest limitations and AI-use note.

Project Milestones

The term project is built through staged milestones so students practice reproducibility, feedback, and knowledge translation before the final submission.

Milestone	Due	Deliverable
M0	Week 4 Monday	Group formation and repository setup
M1	Week 6 Monday	Proposal, dataset selection, provenance/stewardship statement
M2	Week 7 Monday	Preliminary analysis, Table 1 or descriptive summary, runnable repository
M3	Week 10 Monday	Draft report and dashboard-style KT product for review
M4	Week 11 Monday	Peer review with reproducibility and auditability checks
M5	Weeks 12-13	In-class presentation
Final Portfolio	Dec 11 Friday	Complete reproducible portfolio

TODO

Warning

Create detailed milestone briefs in each `milestones/` subfolder from `milestones/milestone-template.qmd`.

Milestone Template

Use this template for project milestones M0-M5 and the Final Portfolio.

Deliverable

- **Milestone:**
- **Due:**
- **Submit:**
- **Main artifact:**

Required Evidence

- Repository link.
- Rendered report, dashboard, slides, or review document.
- Commit history showing meaningful progress.
- AI-use note.
- Reproducibility check result.

Definition of Done

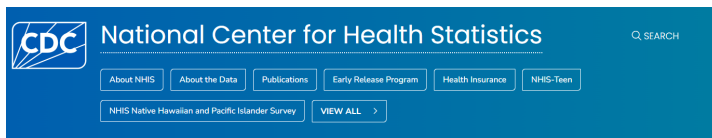
- The artifact opens or renders in the supported workflow.
- Data provenance and stewardship decisions are documented.
- The work avoids unsupported causal or predictive claims.
- A peer or TA can rerun the project from a fresh environment.

Part XVI

Reference: Data Sources

Data Source: NHIS

The National Health Interview Survey (NHIS) is a cross-sectional household interview survey conducted by the US Centers for Disease Control (CDC). The survey collects information on the health status, health care access, and health behaviors of the US population.



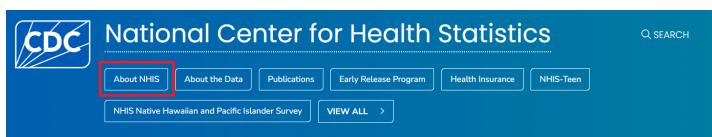
National Health Interview Survey



[CDC website for NHIS](#)

About NHIS

You can find more information about the NHIS survey by clicking on [About NHIS](#):



About NHIS

For Everyone
NOVEMBER 20, 2024

On the **About NHIS** page, you will find information about what the survey collects, data and documentation, the instructions on how to use the data, and information on related NHIS surveys.

Overview

The National Health Interview Survey (NHIS) monitors the health of people across the country—the U.S. population—by collecting and analyzing data on a broad range of health topics. NHIS focuses on the health of U.S. children and adults.

CDC's [National Center for Health Statistics](#) (NCHS) conducts NHIS. It is the nation's largest and oldest national health survey. NHIS began collecting data in 1957.

Each year, NHIS collects data from about 27,000 adults through confidential, face-to-face interviews throughout the year. Many of these adults also provide health information about

ON THIS PAGE

Overview

[What's collected](#)

[Data and documentation](#)

[How the data are used](#)

[Related NHIS surveys](#)

You can click on each of the links or scroll down to find information on each of these topics one by one. For example, if we click on the **How the data are used** link, we will find information as follows:

How the data are used

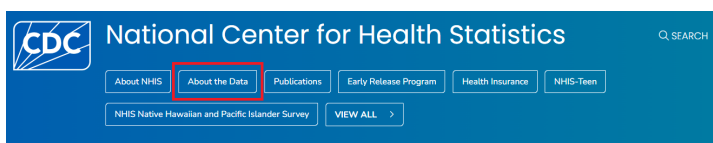
NCHS staff use NHIS data to produce official national health estimates. One of NHIS's key strengths is its ability to produce estimates for many subgroups of the U.S. population. NHIS can do this because it collects data from such a large group of people.

NHIS health statistics provide health information by—

- Demographic characteristics, like age and marital status
- Geographic location, like region and metro area size
- Social and economic circumstances, like employment status and poverty level

About the Data

If you click on **About the Data** link, you will find information about the NHIS questionnaires, datasets, and documentation:



Under the **Overview** tab, you will find information about different NHIS cycles and the questionnaires, datasets, and documentation available in each cycle. As an example, let's click on the **2023 NHIS Questionnaires, Datasets, and Documentation** link to see the details for the 2023 NHIS:

[NHIS Data, Questionnaires and Related Documentation](#)

The image shows the 'Overview' section of the NHIS data page. On the left, under the 'Overview' heading, there is a paragraph of text and a list of links for different years: '2025 NHIS Questionnaires, Datasets, and Documentation', '2024 NHIS Questionnaires, Datasets, and Documentation', '2023 NHIS Questionnaires, Datasets, and Documentation' (highlighted with a red box), '2022 NHIS Questionnaires, Datasets, and Documentation', '2021 NHIS Questionnaires, Datasets, and Documentation', '2020 NHIS Questionnaires, Datasets, and Documentation', and '2019 NHIS Questionnaires, Datasets, and Documentation'. On the right, under the 'ON THIS PAGE' heading, there is a list of links: 'Overview', 'Before 2019', 'Data linked to NHIS data', 'Other NHIS survey data', 'Related surveys', and 'Related pages'.

NHIS Questionnaires, Datasets, and Documentation

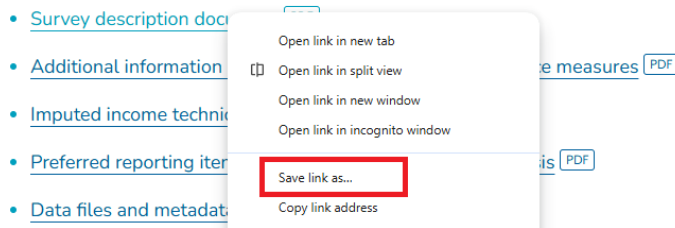
To view the description of the 2023 NHIS, click on the **Survey description document** link:

Using our data

- [Survey description document](#) **PDF**

This will open the PDF of the 2023 NHIS survey description. Alternatively, you can right-click on the link, click on **Save link as...** and save the PDF file on your computer for future reference.

Using our data



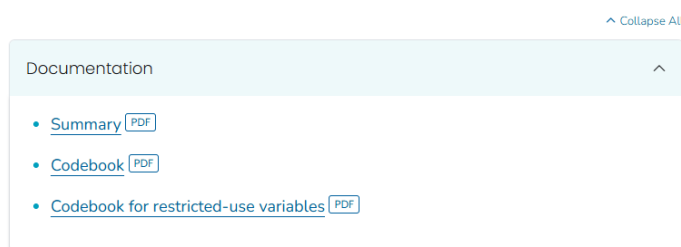
Similarly, you can open and save the questionnaires and related materials under the **Questionnaires and survey materials** tab.

In the recent NHIS, data are available in 5 categories: interview data for adults, interview data for children, data for parent-child pairs, imputed income, and paradata. You can click each link for documentation to see a summary of the data and the codebook.

Data are available in 5 categories:

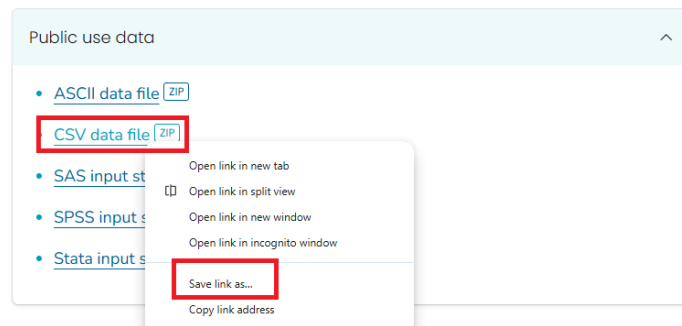
- Sample adult interview
- Sample child interview
- Parent-child pair data
- Imputed income
- Paradata

Sample adult interview

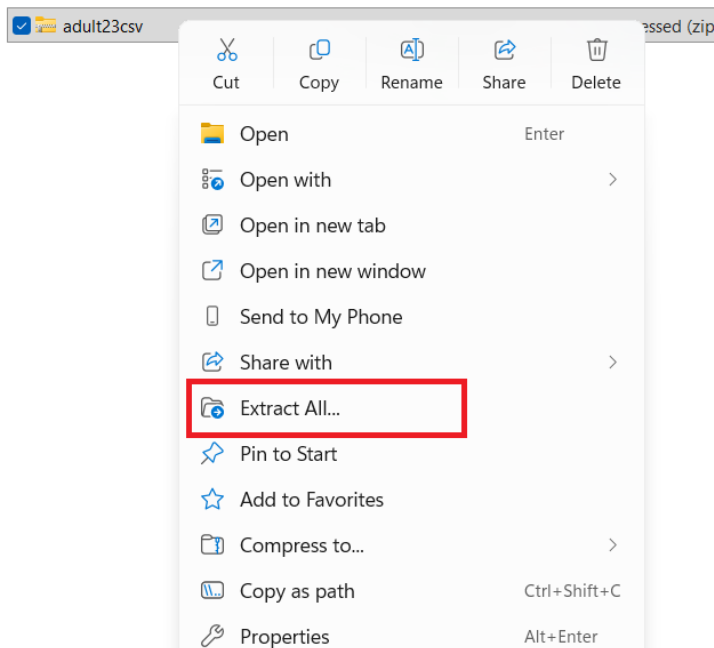


NHIS Data

The data files are stored on the NHIS website in different formats. You can save the dataset in any statistical package that supports these file formats, e.g., ASCII, CSV, SAS. Let us save the `adult` dataset in the CSV format, which is in a zip file.



The next step is to unzip the file to extract the CSV. Depending on your computer, you can unzip the file with or without software. For example, if you are using Windows, you can right-click the zip file and select **Extract All...** to unzip it.



Once unzipped, you will find two files: the CSV file for the dataset and a readme file with some instructions. Below is the example for the `adult` dataset:

 adult23	Microsoft Excel Co...	28,738 KB
 readme	Text Document	3 KB

To open the datafile in Excel, - you can double click on the CSV file, or - right click on the CSV file and click on **Open**.

Later, you will learn how to open the data file in different statistical software. For Excel, it would take a few seconds/minutes because of the large size of the dataset. Once you open the data file, you will see the data in a tabular format with rows and columns.

URBRRL	RATCAT_A	INCTCFLG_A	IMPINCFLG_A	LANGSPECR_A	LANGSOC_A	LANGDOC_A	LANGMED_A	LANGHM_A	PPSU	PSTRAT
3	4	0	0						2	103
4	8	0	0						2	122
4	14	0	0						2	122
4	10	0	0						2	122
4	5	0	0						2	122
4	14	0	0						2	122
3	12	0	0						2	103

- Each row represents a person.
- Each column represents a variable.
- The first row contains the variable names. The remaining rows contain the data for each person.

Use of the codebook

To understand the variables in the dataset, you can refer to the codebook. The codebook provides information about the variable names, labels, and values. For example, we want to understand the variable `SEX_A`:

FQ	FR	FS	FT	FU	FV
SRVY_YR	SEX_A	AGEP_A	AGE65	ASTATNEV	MENTHOL C
2023	1	67		1	
2023	1	73		1	
2023	1	48		1	
2023	2	42		1	
2023	2	50		1	
2023	2	46		1	
2023	2	36		1	
2023	1	44		1	

We can look for `SEX_A` in the codebook, which will provide us with the variable label and the values for that variable (e.g., 1 for Male, 2 for Female, and so on).

17 / 653 | — 117% + |     

2023 NATIONAL HEALTH INTERVIEW SURVEY (NHIS)
Codebook for Sample Adult File (Version: 24 June 2024)
PUBLIC USE

Variable Information:

Variable:	SEX_A
Module:	Roster
Section:	HHC : Household Composition
File(s):	Adult
Data Type:	Numeric
Length:	1
Question Text:	
Description:	Sex of Sample Adult
Recode:	Yes
Universe:	HHSTAT_A=1
Universe Description:	Sample adults 18+
Sources:	
Question ID:	
Keywords:	
Notes:	
Evaluation Report:	

Unweighted frequencies:

SEX_A	Sex of Sample Adult		
Code	Description	Frequency	Percent
1	Male	13457	45.58
2	Female	16059	54.40
7	Refused	4	0.01
8	Not Ascertained	0	0.00
9	Don't Know	2	0.01

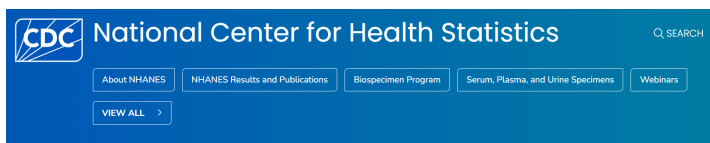
Frequency Missing:

There are so many variables in the dataset, and it is not possible to understand all of them at once. You can refer to the codebook to understand the variables that you are interested in for your analysis. For example, the `WTFA_A` variable is the sample adult weight variable used to calculate weighted estimates

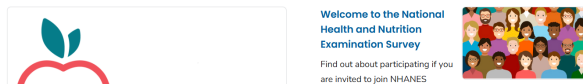
for the US adult population. Similarly, the `HYPEV_A` variable is the hypertension ever variable, indicating whether a doctor or other health professional has ever told the person they have hypertension. Again, you can refer to the codebook to understand the variables that you are interested in.

Data Source: NHANES

The National Health and Nutrition Examination Survey (NHANES) is a cross-sectional survey conducted by the US Centers for Disease Control (CDC). The survey collects information on the health and nutritional status of the US population. The survey includes interviews, physical examinations, and laboratory tests.



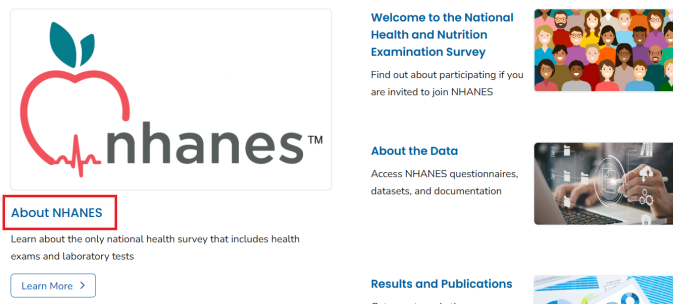
National Health and Nutrition Examination Survey



[CDC website for NHANES](#)

About NHANES

You can find more information about the NHANES survey by clicking on [About NHANES](#):



About NHANES

For Everyone
DECEMBER 18, 2024

AT A GLANCE

- NHANES is the National Health and Nutrition Examination Survey.
- NHANES is a national survey that measures the health and nutrition of adults and children in the United States.
- NHANES is the only national health survey that includes health exams and laboratory tests for participants of all ages.



On the **About NHANES** page, you will find information about what the survey collects, data and documentation, the instructions on how to use the data, and information on related NHANES surveys.

Overview

The National Health and Nutrition Examination Survey (NHANES) collects data about the health of adults and children in the United States. We also collect data about what participants eat, drink, and take as supplements to determine how many nutrients are in their diet. These dietary interviews and blood tests help us measure the nutritional status of U.S. adults and children.

CDC's [National Center for Health Statistics](#) (NCHS) conducts NHANES. NHANES is the only national health survey that includes health exams, laboratory tests, and dietary interviews for participants of all ages.

ON THIS PAGE

- Overview
- What's collected
- Data and documentation
- How the data are used
- Program director
- Contact NHANES

You can click on each of the links or scroll down to find information on each of these topics one by one. For example, if we click on the **Data and documentation** link, we will find information as follows:

Data and documentation

NCHS [publishes reports](#) featuring analysis of NHANES data. Selected estimates also are available as part of [interactive data visualizations](#) that provide tables and charts.

Data files

[NHANES data files](#) and [related documentation](#) are publicly available to download and analyze. The related documentation can help researchers understand and use NHANES data. All potentially identifiable information has been removed to ensure the confidentiality of participants and their households.

[NHANES restricted data files](#) are available through the NCHS Research Data Center (RDC) for a fee. To access restricted NHANES data, researchers must submit requests to the

RELATED PAGES

- What NHANES Covers and How It Works
- Who Participates In NHANES
- How People Use NHANES Data
- What NHANES Data Have Achieved
- Ethics Review Board Approval

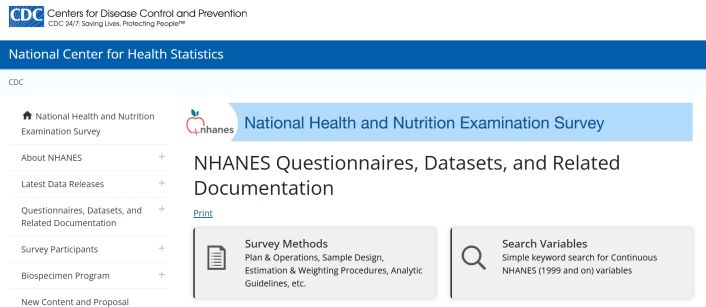
[VIEW ALL](#)
National Health and Nutrition
Examination Survey

Survey Methods and Analytic Guidelines

If you click on **NHANES data files and related documentation** link, you will find information about the NHANES questionnaires, datasets, and documentation:

Data files

[NHANES data files and related documentation](#) are publicly available to download and analyze. The related documentation can help researchers understand and use NHANES data. All potentially identifiable information has been removed to ensure the confidentiality of participants and their households.

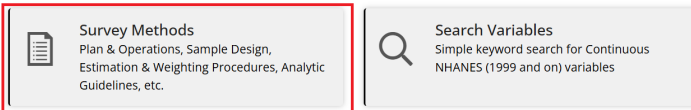


You can click **Survey Methods** to find information on survey methods and analytic guidelines. The **Survey Methods and Analytic Guidelines** page provides information about the plan and operation of the NHANES survey, the sampling design, methods for calculating weights and variance estimation, and guidelines for analyzing NHANES data, along with links to related resources.

[NHANES data files and related documentation](#)

NHANES Questionnaires, Datasets, and Related Documentation

[Print](#)



Search Variables

If you click on **Search Variables** link, you will find a search tool to search for variables in the NHANES datasets (1999 and on).

NHANES Questionnaires, Datasets, and Related Documentation

[Print](#)

**Survey Methods**
Plan & Operations, Sample Design,
Estimation & Weighting Procedures, Analytic
Guidelines, etc.

**Search Variables**
Simple keyword search for Continuous
NHANES (1999 and on) variables

You can search for variables by entering keywords in the search box. For example, if you enter **diabetes** in the search box, you will find a list of variables related to diabetes in the NHANES datasets.

Search Variables

[Print](#)

Search for words and "quoted phrases" in selected fields (Data File Name, SAS Label, Variable Description, Variable Name) of Continuous NHANES (1999 and on) variables that have published documentation. *Multiply imputed Dual Energy X-Ray Absorptiometry variables are not included.*

Refer to the [Survey Content Brochure](#) for the full content of Continuous NHANES.

Search Term	diabetes
Fields to Search	All
Sort By	Variable Name
Limited Access	Exclude
Release Cycle	All
Search Result Page Size	50

Search

Number of variables found: 459

Variable Name	SAS Label	Variable Description	Data File Name	Component Link
DID040	Age when first told you had diabetes	How old (was SP/were you) when a doctor or other health professional first told (you/him/her) that (you/he/she) had diabetes or sugar diabetes?	Diabetes (DIQ_D)	2005-2006 Questionnaire
DID040	Age when first told you had diabetes	How old (was SP/were you) when a doctor or other health professional first told (you/him/her) that (you/he/she) had diabetes or sugar diabetes?	Diabetes (DIQ_E)	2007-2008 Questionnaire
DID040	Age when first told you had diabetes	How old (was SP/were you) when a doctor or other health professional first told (you/him/her) that (you/he/she) had diabetes or sugar diabetes?	Diabetes (DIQ_F)	2009-2010 Questionnaire
DID040	Age when first told you had	How old (was SP/were you) when a doctor or other health professional first told (you/him/her) that	Diabetes (DIQ_G)	2011-2012 Questionnaire

You can restrict your search to a specific NHANES cycle by selecting the cycle from the drop-down menu. For example, if you select the **2017-2018** cycle, you will find a list of variables related to diabetes in the 2017-2018 NHANES dataset.

Limited Access

Exclude

Release Cycle

2017-2018

Search Result Page Size

50

Search

Number of variables found: 58

Variable Name	SAS Label	Variable Description	Data File Name	Component Link
DIQ175C	Age	Why (Do you/Does SP) think (you are/he is/she is) at risk for diabetes or prediabetes? [Anything else?]	Diabetes (DIQ_J)	2017-2018 Questionnaire
RHQ163	Age told you had diabetes while pregnant	How old (were you/was SP) when (you were/she was) first told (you/she) had diabetes during a pregnancy?	Reproductive Health (RHQ_J)	2017-2018 Questionnaire

NHANES Questionnaires, Datasets, and Documentation

Let's explore the questionnaires, datasets, and documentation for the 2017-2018 NHANES cycle. You need to click on **NHANES 2017-2018** under the **Continuous NHANES** tab:

NHANES Questionnaires, Datasets, and Related Documentation

[Print](#)

Survey Methods
Plan & Operations, Sample Design, Estimation & Weighting Procedures, Analytic Guidelines, etc.

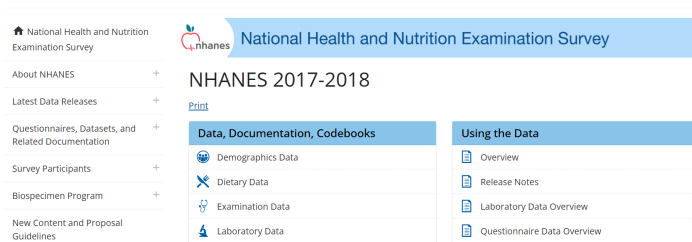
Search Variables
Simple keyword search for Continuous NHANES (1999 and on) variables

[» Guidelines for High Quality Analyses of NHANES Data](#)

Continuous NHANES

NHANES 2025-2026	NHANES 08/2021-08/2023	NHANES 2017-March 2020 Pre-Pandemic Data	
NHANES 2019-2020	NHANES 2017-2018	NHANES 2015-2016	NHANES 2013-2014

This will open the page for the 2017-2018 NHANES cycle, where you can find information on the survey description, questionnaires, datasets, and documentation.



There are six categories of data available for the 2017-2018 NHANES cycle: Demographics, Dietary, Examination, Laboratory, Questionnaire, and Limited Access data. You can click each category to find the datasets and documentation available in that category. For example, if you click on **Demographics Data**, you will find a list of datasets and documentation related to demographics data in the 2017-2018 NHANES cycle.

Data are available in 6 categories:

- Demographics
- Dietary
- Examination
- Laboratory
- Questionnaire
- Limited Access

NHANES 2017-2018

[Print](#)

Data, Documentation, Codebooks	Using the Data
Demographics Data	Overview
Dietary Data	Release Notes
Examination Data	Laboratory Data Overview
Laboratory Data	Questionnaire Data Overview
Questionnaire Data	Examination Data Overview
Limited Access Data	Survey Methods and Analytic Guidelines

2017-2018 Demographics Data - Continuous NHANES

[Print](#)

- [NHANES 2017-2018 Demographics Variable List](#)
- [Questionnaire Instruments](#)
- [Exam Procedure Manuals](#)
- [SAS Universal Viewer](#)
- [Data User Agreement](#)

Data File Name	Doc File	Data File	Date Published
Demographic Variables and Sample Weights	DEMO_1.Doc	DEMO_1 Data [XPT - 3.3 MB]	February 2020

There is a **Doc File** available for each dataset that provides information about the dataset, including the variables it contains, their coding, and any special considerations for using it. It is important to read the **Doc** file before using the dataset to understand the data structure and how to use it properly. The **Data File** contains the dataset data in **.XPT** format, which is a SAS Transport file, not a CSV.

2017-2018 Demographics Data - Continuous NHANES

[Print](#)

- [NHANES 2017-2018 Demographics Variable List](#)
- [Questionnaire Instruments](#)
- [Exam Procedure Manuals](#)
- [SAS Universal Viewer](#)
- [Data User Agreement](#)

Data File Name	Doc File	Data File	Date Published
Demographic Variables and Sample Weights	DEMO_J Doc	DEMO_J Data [XPT - 3.3 MB]	February 2020

Data

You need to click on the [DEMO_J Data \[XPT - 3.3 MB\]](#) link to download the dataset. Note that you cannot easily open an .XPT file in Excel or Google Sheets. In the real world, data isn't always in a friendly format. For this week, let's download the `NHANES_Demo_Converted.csv` file from our Canvas page to look at the rows. Later, we will use an AI co-pilot to write R code that reads .XPT files automatically.

Let's open the `NHANES_Demo_Converted.csv` file in Excel and see the first few rows:

	A	B	C	D	E	F	G
1	SEQN	SDDSRVYR	RIDSTATR	RIAGENDR	RIDAGEYR	RIDAGEMN	RIDRETH1
2	93703	10	2	2	2	NA	5
3	93704	10	2	1	2	NA	3
4	93705	10	2	2	66	NA	4
5	93706	10	2	1	18	NA	5
6	93707	10	2	1	13	NA	5
7	93708	10	2	2	66	NA	5
8	93709	10	2	2	75	NA	4
9	93710	10	2	2	0	11	3
10	93711	10	2	1	56	NA	5
11	93712	10	2	1	18	NA	1
12	93713	10	2	1	67	NA	3
13	93714	10	2	2	54	NA	4
14	93715	10	2	1	71	NA	5

- Each row represents a person.
- Each column represents a variable.
- The first row contains the variable names. The remaining rows contain the data for each person.

For example, `SEQN` is the unique identifier for each person in the dataset, and `RIAGENDR` is the variable for gender.

Use of the Codebook

To understand the variables in the dataset, you can refer to the codebook. The codebook provides information about the variable names, labels, and values. The Doc File explained earlier contains the codebook for the dataset. You need to click on the DEMO_J Doc link to open the codebook on a new page. Let's we want to understand the variables SEQN and RIAGENDR:

National Health and Nutrition Examination Survey

2017-2018 Data Documentation, Codebook, and Frequencies

Demographic Variables and Sample Weights (DEMO_J)

Data File: DEMO_J.xpt

First Published: February 2020

Last Revised: NA

Component Description

The demographics file provides individual, family, and household-level information on the following topics:

- Survey participant's household interview and examination status;

TABLE OF CONTENTS

- Component Description
- Eligible Sample
- Interview Setting and Mode of Administration
- Quality Assurance & Quality Control
- Data Processing and Editing
- Analytic Notes
- References
- Codebook
 - SEQN - Respondent sequence number
 - SDDSRVYR - Data release cycle
 - RIDSTATR - Interview/Examination status
 - RIAGENDR - Gender
 - RIDAGEYR - Age in years at screening
 - RIDAGEMN - Age in months at screening - 0 to 24 mos

You need to click on the **SEQN - Respondent sequence number** link or scroll down to find the information about the SEQN variable.

SEQN - Respondent sequence number

Variable Name: SEQN
SAS Label: Respondent sequence number
English Text: Respondent sequence number.
Target: Both males and females 0 YEARS - 150 YEARS

SDDSRVYR - Data release cycle

Variable Name: SDDSRVYR
SAS Label: Data release cycle
English Text: Data release cycle

TABLE OF CONTENTS

- Component Description
- Eligible Sample
- Interview Setting and Mode of Administration
- Quality Assurance & Quality Control
- Data Processing and Editing
- Analytic Notes
- References
- Codebook
 - SEQN - Respondent sequence number
 - SDDSRVYR - Data release cycle
 - RIDSTATR - Interview/Examination status
 - RIAGENDR - Gender

Similarly, you can click on the **RIAGENDR - Gender** link or scroll down to find the information about the RIAGENDR variable.

RIAGENDR - Gender

Variable Name: RIAGENDR
SAS Label: Gender
English Text: Gender of the participant.
Target: Both males and females 0 YEARS - 150 YEARS

Code or Value	Value Description	Count	Cumulative	Skip to Item
1	Male	4557	4557	
2	Female	4697	9254	
.	Missing	0	9254	

RIDAGEYR - Age in years at screening

Variable Name: RIDAGEYR

TABLE OF CONTENTS

- Component Description
- Eligible Sample
- Interview Setting and Mode of Administration
- Quality Assurance & Quality Control
- Data Processing and Editing
- Analytic Notes
- References
- Codebook
 - SEQN - Respondent sequence number
 - SDDSRVYR - Data release cycle
 - RIDSTATR - Interview/Examination status
 - RIAGENDR - Gender
 - RIDAGEYR - Age in years at screening
 - RIDAGEMN - Age in months at screening - 0 to 24 mos

As you can see, the **RIAGENDR** variable has two values: 1 for Male and 2 for Female, without any missing values. You can also see the frequency of each value in the dataset. These numbers/frequencies should exactly match the frequencies in your dataset. You can use the codebook to understand other variables in the dataset. Similarly, you can use the other data files (e.g., Dietary) and their codebook to understand the variables in those datasets. Unlike the demographic file, which uses only one dataset and one codebook, the other files use multiple datasets and codebooks. As an example, if you click on the **Questionnaire Data** link, you will find multiple datasets and codebooks for questionnaire data:

2017-2018 Questionnaire Data - Continuous NHANES

[Print](#)

- [NHANES 2017-2018 Questionnaire Variable List](#)
- [Questionnaire Instruments](#)
- [2017-2018 Questionnaire Data Overview](#)
- [SAS Universal Viewer](#)
- [Data User Agreement](#)

Data File Name	Doc File	Data File	Date Published
Acculturation	ACO.J.Doc	ACO.J.Data [XPT - 396.3 KB]	February 2020
Alcohol Use	ALO.J.Doc	ALO.J.Data [XPT - 434.4 KB]	December 2020
Audiometry	AUO.J.Doc	AUO.J.Data [XPT - 3.9 MB]	July 2020
Blood Pressure & Cholesterol	BPO.J.Doc	BPO.J.Data [XPT - 531.8 KB]	February 2020
Cardiovascular Health	CDO.J.Doc	CDO.J.Data [XPT - 518.7 KB]	February 2020
Consumer Behavior	CBO.J.Doc	CBO.J.Data [XPT - 435.4 KB]	April 2021
Consumer Behavior Phone Follow-up	CBOPFA.J.Doc	CBOPFA.J.Data [XPT - 2.7]	June 2021

Data Source: BRFSS

The Behavioral Risk Factor Surveillance System (BRFSS) is a cross-sectional survey conducted by the US Centers for Disease Control (CDC). The survey collects information on health-related risk behaviors, chronic health conditions, and use of preventive services among US adults. BRFSS completes more than 400,000 adult interviews each year, making it the largest continuously conducted health survey system in the world.



[CDC website for BRFSS](#)

About BRFSS

You can find more information about the BRFSS survey by clicking on [BRFSS FAQs](#):



On the [BRFSS FAQs](#) page, you will find information about the BRFSS survey, including what the survey collects, how the survey is conducted, and how to use the data.

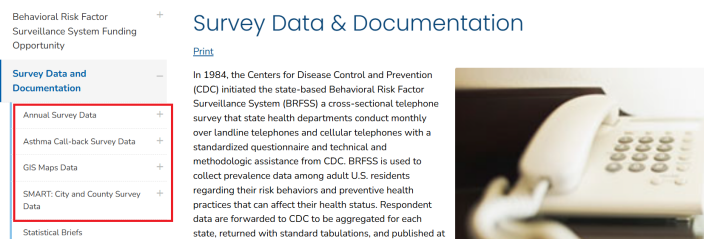
Survey Data & Documentation

If you click on [Survey Data & Documentation](#) link, you will find information about the BRFSS questionnaires, datasets, and documentation:



There are four types of data files available for download: 1) annual data, 2) asthma call-back data, 3) GIS map data, and 4) SMART data. The annual data files include information on health-related risk behaviors, chronic health conditions, and use of preventive services among US adults. The asthma call-back data files include information on asthma-related health outcomes and healthcare utilization among adults with asthma. The GIS map data files include information on the geographic distribution of the data. The SMART (Selected Metropolitan/Micropolitan Area Risk Trends) data provide prevalence rates for selected conditions and behaviors for cities and their surrounding counties.

[BRFSS Survey Data & Documentation](#)



For the annual data, you can click on the **Annual Survey Data** link to find information about the annual data files, including how to access the data, how to use the data, and how to analyze the data. There are two types of annual data files available for download: 1) the raw data files and 2) the prevalence and trends database.

Annual Survey Data

There are separate links for each year of data, and you can click on each link to find information about the data for that year. For example, if you click on the **2023 Annual Survey Data** link, you will find information about the 2023 annual data files, including how to access the data, how to use the data, and how to analyze the data.

Annual Survey Data

2023 BRFSS Survey Data and Documentation

[Print](#)

The 2023 BRFSS data continue to reflect the changes initially made in 2011 for weighting methodology (raking) and adding cell-phone-only respondents. The aggregate BRFSS combined landline and cell phone data set is built from the landline and cell phone data submitted for 2023 and includes data from 48 states, the District of Columbia, Guam, Puerto Rico, and the US Virgin Islands. During 2023, Kentucky and Pennsylvania were unable to collect enough data to meet the minimum requirements to be included in this public data set. The data set has been modified to comply with President Trump's executive orders. There are some missing values which may appear to be inconsistencies in the data based on a respondents' answers to questions that were removed.

2023 Survey Data Information

[2023 BRFSS Overview CDC](#)  [PDF – 213 KB]

The 2023 BRFSS Survey Data and Documentation page provides information about three resources: 1) data information such as the codebook and sampling weights, 2) data files in ASCII and SAS XPT transport formats, and 3) SAS resources. To download/save the data files, click the specific format you want. For example, if you click on the 2023 BRFSS Data (SAS Transport Format) link, the annual survey data for 2023 will be downloaded in a ZIP file that contains the dataset in a .XPT format.

Data Files

There are 433,323 records for 2023. More information on participation is available in the [states conducting surveillance, by year table](#). The data files are provided in ASCII and SAS Transport formats. The September update includes updated race/ethnicity calculated variables for Mississippi. The February update includes changes to the data set which has been modified to comply with President Trump's executive orders. There are some missing values which may appear to be inconsistencies in the data based on a respondents' answers to questions that were removed.

[2023 BRFSS Data \(ASCII\)](#)  [ZIP – 41.5 MB]

February, 2025

This file for the combined landline and cell phone data set is in ASCII format. It has a fixed record length of 2111 positions.

[2023 BRFSS Data \(SAS Transport Format\)](#) 

[ZIP – 64.3 MB]

February, 2025

This file for the combined landline and cell phone data set was exported from SAS V9.4 in the XPT transport

You need to unzip the file to extract the data file in .XPT format. However, opening the data file in Excel or Google Sheets is a bit tricky. The SAS XPT (or ASCII) data file needs to be converted/formatted appropriately before it can be opened in Excel or Google Sheets. Many statistical software packages, such as R, SAS, or STATA, can read the data file in .XPT format. Below is the data file, where the column names are the variable names, and the rows are the observations (i.e., survey respondents):

	STATE FIPS CODE	FILE MONTH	INTERVIEW DATE	INTERVIEW MONTH	INTERVIEW DAY	INTERVIEW YEAR	FINAL DISPOSITION	ANNUAL SEQUENCE NUMBER	PRIMARY SAMPLING UNIT	CORRECT TELEPHONE NUMBER?	PRIVATE RESIDENCE?	DO YOU LIVE IN COLLEGE HOUSING?	RESIDENT OF STATE	CELLULAR TELEPHONE	ARE YOU YEARS OF AGE OR OLDER?
1	1	1	1 03012023	03	01	2023	1100	2023000001	2023000001	1	1	-	1	2	
2	1	1	1 01062023	01	06	2023	1100	2023000002	2023000002	1	1	-	1	2	
3	1	1	1 03082023	03	08	2023	1100	2023000003	2023000003	1	1	-	1	2	
4	1	1	1 03062023	03	06	2023	1100	2023000004	2023000004	1	1	-	1	2	
5	1	1	1 01062023	01	06	2023	1100	2023000005	2023000005	1	1	-	1	2	
6	1	1	1 01092023	01	09	2023	1100	2023000006	2023000006	1	1	-	1	2	
7	1	1	1 0212023	03	21	2023	1100	2023000007	2023000007	1	1	-	1	2	
8	1	1	1 01062023	01	06	2023	1100	2023000008	2023000008	1	1	-	1	2	
9	1	1	1 01152023	03	15	2023	1100	2023000009	2023000009	1	1	-	1	2	
10	1	1	1 01082023	01	08	2023	1100	2023000010	2023000010	1	1	-	1	2	
11	1	1	1 03082023	03	08	2023	1100	2023000011	2023000011	1	1	-	1	2	
12	1	1	1 01082023	01	08	2023	1100	2023000012	2023000012	1	1	-	1	2	
13	1	1	1 03012023	03	01	2023	1100	2023000013	2023000013	1	1	-	1	2	
14	1	1	1 01062023	01	06	2023	1200	2023000014	2023000014	1	1	-	1	2	
15	1	1	1 0312023	03	21	2023	1100	2023000015	2023000015	1	1	-	1	2	
16	1	1	1 0112023	03	13	2023	1100	2023000016	2023000016	1	1	-	1	2	

To understand the variables in the dataset, you can refer to the codebook:

2023 Survey Data Information

[2023 BRFSS Overview CDC](#) [PDF – 213 KB]

Provides information on the background, design, data collection and processing, and the statistical and analytical issues for the combined landline and cell phone data set.

[2023 BRFSS Codebook CDC](#) [ZIP – 3 MB]

Codebook for the file showing variable name, location, and frequency of values for all reporting areas combined for the combined landline and cell phone data set.

You need to unzip the file to extract the codebook in HTML format. The codebook provides information about the variable names, labels, and values. For example, if you want to understand the variable **Drinking and Driving**, you can find the variable name, label, and values in the codebook:

Label: Drinking and Driving				
Section Name: Calculated Variables				
Module Number: 15				
Question Number: 3				
Column: 2110				
Type of Variable: Num				
SAS Variable Name: _DRNKDRV				
Question Prologue:				
Question: Drinking and Driving (Reported having driven at least once when perhaps had too much to drink)				
Value	Value Label	Frequency	Percentage	Weighted Percentage
1	Have driven after having too much to drink	6,192	1.43	1.36
2	Have not driven after having too much to drink	205,541	47.43	45.81
9	Don't know/Not Sure/Refused/Missing	221,590	51.14	52.83

You can similarly download the asthma call-back, GIS map, and SMART datasets. To understand the variables in those datasets, you can refer to the codebook for each dataset.

[Asthma Call-back Data](#)

[GIS Data](#)

[SMART Data](#)

Prevalence and Trends Database

The BRFSS prevalence and trends tool allows users to view and download prevalence estimates through charts, graphs, and maps. You can explore the estimates by selecting the topic or location. For example, if you select by **topic**, e.g., for alcohol consumption for Alabama for the year 2024, you need to select the options from the dropdown menu and click on the **View Results** button to see the results:

Prevalence and Trends Database

CDC BRFSS Prevalence & Trends Data

Per a court order, HHS is required to restore this website to its version as of 12:00 AM on January 29, 2025. Information on this page may be modified and/or removed in the future subject to the terms of the court's order and implemented consistent with applicable law. Any information on this page promoting gender ideology is extremely inaccurate and disconnected from truth. The Trump Administration rejects gender ideology due to the harms and divisiveness it causes. This page does not reflect reality and therefore the Administration and this Department reject it.

[Print](#)

The enhanced BRFSS Prevalence & Trends Tool allows you to explore prevalence data by location or topic.

- Just like the original tool, this version allows you to view and download prevalence estimates through charts, graphs, and maps.
- In addition, this version also provides access to prevalence estimates from the BRFSS core data at the state level as well as data from Selected Metropolitan/Micropolitan Area Risk Trends (SMART).

These prevalence estimates have been updated to include both crude and age-adjusted prevalence.

The tool will continue to be updated as new functions and data become available.

Explore by Location Explore by Topic

Explore bar and trendline graphs for survey questions by selecting a state, territories or Metropolitan/Micropolitan Statistical Areas (MMSAs).

Dataset	Location	Class	Topic	Year
Nationwide	States - Alabama	Alcohol Consumption	Binge Drinking	2024

Question: Binge drinkers (males having five or more drinks on one occasion, females having four or more drinks on one occasion) [variable calculated from one or more BRFSS questions]

View Results Clear Filters

The results will show the (i) prevalence estimates for alcohol consumption in Alabama for the year 2024, and (ii) the trend of alcohol consumption in Alabama over time.

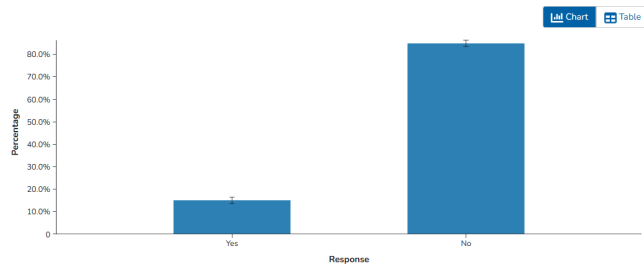
[View Results](#) [Clear Filters](#)

Binge drinkers (males having five or more drinks on one occasion, females having four or more drinks on one occasion) (variable calculated from one or more BRFSS questions)

Alcohol Consumption: Binge Drinking

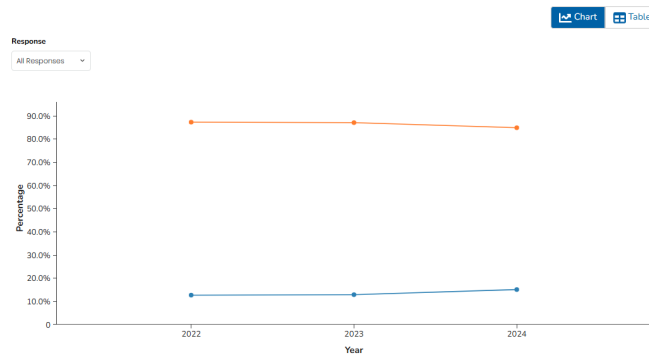
View By: Data Type:

2024, Alabama



The ordering of subcategory rows in the table view may vary across responses.

All Available Years, Alabama



Data Source: CCS

The Canadian Cannabis Survey (CCS) is an annual survey conducted by Health Canada to gather information on cannabis use among people aged 16 years and older living in all Canadian provinces and territories. The survey collects data on various aspects of cannabis consumption, including their knowledge, attitudes and behaviours towards cannabis use.

Government of Canada
Gouvernement du Canada

Franglais

Search Canada.ca

MENU

Canada.ca > Open Government > Search

Canadian Cannabis Survey (CCS) 2024 Public Use Microdata File (PUMF)

Have your say

[Rate this dataset](#)
[Provide feedback](#)

The 2024 Canadian Cannabis Survey (CCS) recruited people aged 16 years and older living in all Canadian provinces and territories. Respondents completed a survey on patterns of cannabis use for medical and non-medical purposes, as well as their knowledge, attitudes and behaviours towards cannabis use. A total of 11,666 respondents completed online surveys between April 4 and July 2, 2024. Summary results are available here: <https://www.canada.ca/en/health-canada/services/drugs-medication/cannabis/research-data/canadian-cannabis-survey-2024-summary.html>

Additional

The Office Cannabis Science and Surveillance releases the PUMF for CCS 2024 to allow

More than 11,000 respondents participated in the 2024 survey. The study provides valuable insights into cannabis use patterns, the cannabis market, and issues of public safety. The survey results are published on Health Canada's website:

[Health Canada website for CCS](#)



Government of Canada
Gouvernement du Canada

Franglais

MENU

Canada.ca
Open Government
Search

Have your say

Rate this dataset

Provide feedback

Additional

The 2024 Canadian Cannabis Survey (CCS) recruited people aged 16 years and older living in all Canadian provinces and territories. Respondents completed a survey on patterns of cannabis use for medical and non-medical purposes, as well as their knowledge, attitudes and behaviours towards cannabis use. A total of 11,666 respondents completed online surveys between April 4 and July 2, 2024. Summary results are available here:

<https://www.canada.ca/en/health-canada/services/drugs-medication/cannabis/research-data/canadian-cannabis-survey-2024-summary.html>

The Office Cannabis Science and Surveillance releases the PUMF for CCS 2024 to allow



Government of Canada
Gouvernement du Canada

Franglais

MENU

Canada.ca
Health
Drug and health products
Drugs and medication
Cannabis
Cannabis research and data

Canadian Cannabis Survey 2024: Summary

On this page

- Introduction
- Results by theme
- Methods
- Considerations

Open Licence

The data from the Canadian Cannabis Survey is available under the Open Government Licence - Canada (OGL-Canada). This licence allows users to freely use, modify, and distribute the data, provided that they attribute the source of the data and do not use it for commercial purposes. The OGL-Canada promotes transparency and encourages the use of government data for research, analysis, and public benefit. You can follow the [Open Government Licence - Canada](#) link for more information on the terms and conditions of the licence.

[Open Licence](#)

247

Additional Information

Keywords:
cannabis | drugs

Subject:
Health and Safety

Maintenance and Update Frequency:

[data/cannabis-survey-2024-summary.html](#)

The Office Cannabis Science and Surveillance releases the PUMF for CCS 2024 to allow researchers, provincial governments and others to conduct their own analyses of this important data.

Publisher - Current Organization Name: Health Canada

Publisher - Organization Name at Publication: Office of Cannabis Science and Surveillance, Strategic Policy Directorate, Controlled Substances and Cannabis Branch, Health Canada

Licence: [Open Government Licence - Canada](#)

Open Government Licence - Canada

You are encouraged to use the Information that is available under this licence with only a few conditions.

Using Information under this licence

- Use of any Information indicates your acceptance of the terms below.
- The Information Provider grants you a worldwide, royalty-free, perpetual, non-exclusive licence to use the Information, including for commercial purposes, subject to the terms below.

Data and Resources

The **Data and Resources** tab on the Health Canada website provides access to the public use data, user guides, and the data dictionary.

Data and Resources

Search resources...

Q

CCS

Canadian Cannabis Survey (CCS) 2024

English

dataset

CSV

Explore

CCS

Canadian Cannabis Survey (CCS) 2024 PUMF User Guide

English

guide

DOCX

Explore

CCS

Canadian Cannabis Survey (CCS) 2024 PUMF User Guide

French

guide

DOCX

Explore

CCS

Public Use Microdata Files (PUMF) Derived Variable Specifications

English

specification

DOCX

Explore

CCS

Public Use Microdata Files (PUMF) Derived Variable Specifications

French

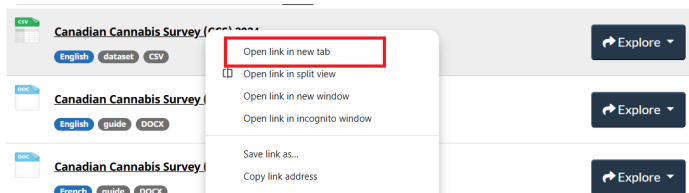
specification

DOCX

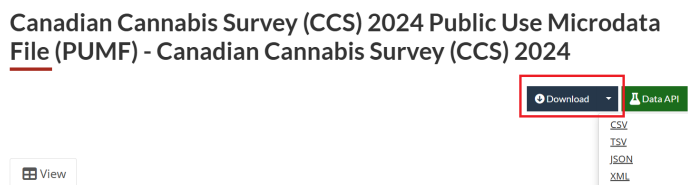
Explore

The CCS 2024 data is available to download in various formats, including CSV, TSV, JSON, and XML. The data can be accessed through the Health Canada website, where you can choose the format that best suits your needs. You need to

open the data link (say, in a new tab in your browser) and select the desired format to download the data.



Under the **Download** tab, you can find the data in different formats. For example, you can download the data in CSV format by clicking on the **CSV** link:



After downloading, you can open the data in Excel, Google Sheets, or any statistical software. The data is structured in a tabular format, with each row representing an individual respondent and each column representing different variables:

1	A	B	C	D	E	F	G	H	I	J	K	L	M
2	seq_id	wave	sample_weight	age65	sex	sexuality_recod	community_size	prov	race_white	health_physical	health_mental	education3	income_recod
3	1	7	2622.409	2	2	1	1	7	1	2	2	3	3
4	2	7	1517.9545	4	1	1	-7	5	1	2	3	3	2
5	3	7	1547.9545	1	1	1	1	1	0	2	4	1	1
6	4	7	3095	1	2	1	2	7	1	4	3	1	2
7	5	7	2475.0746	4	1	2	2	2	0	2	4	3	4
8	6	7	2306.2095	6	1	1	2	2	1	1	1	2	3
9	7	7	4033.9425	4	1	1	1	7	0	4	3	1	2
10	8	7	1933.1395	1	2	1	2	9	1	1	1	1	3
11	9	7	4273.64	3	1	1	2	9	0	4	3	3	-8
12	10	7	1512.9487	5	1	-8	-8	6	-8	3	2	1	-8
13	11	7	2438.4091	4	2	1	2	1	0	3	4	2	3
14	12	7	1960.8133	1	1	1	2	5	1	3	4	1	1

The **Canadian Cannabis Survey (CCS) 2024 PUMF User Guide** gives detailed information on the survey methodology, data collection process, and how to use the public use microdata file (PUMF). You can also find the variable names and their descriptions in this file.

[2024 Public Use Microdata File](#)

Data and Resources

Search resources...

- Canadian Cannabis Survey (CCS) 2024**
English dataset CSV Explore
- Canadian Cannabis Survey (CCS) 2024 PUMF User Guide**
English guide DOCK Explore
- Canadian Cannabis Survey (CCS) 2024 PUMF User Guide**
French guide DOCK Explore
- Public Use Microdata Files (PUMF) Derived Variable Specifications**
English specification DOCK Explore
- Public Use Microdata Files (PUMF) Derived Variable Specifications**
French specification DOCK Explore

Let's find a variable (`access_ease_legal`) in the Excel file and check its description in the data dictionary. You can use the Find function in Excel (Ctrl + F) to search for the variable name:

cansirisk	canli	risk_cand	risk_cane	access_ease_legal	access_e	warn_see	warn_crec	warn_kno	warr
3	3	3	3	3	5	4	-7	-7	
2	2	2	2	4	2	4	-7	-7	
1	1	1	1	4	3	1	1	1	
4	4	4	4	4	3	4	-7	-7	
2	3	5	5	4	5	1	4	4	
5	5	5	5	4	5	4	-7	-7	
2	2	2	2	4	4	3	3	2	
4	4	4	4	1	3	4	-7	-7	
3	3	3	2	4	3	4	-7	-7	
2	2	2	2	4	4	5	7	-7	
3								1	
2								-7	
1								2	
4								-7	
2								3	
3								-7	
3								3	
3								1	
4								-7	
4	4	4	3	4	5	4	-7	-7	
2	2	2	2	4	3	2	1	2	

Find and Replace

Find Replace

Find what: access_ease_legal

Options >>

Find All Find Next Close

Then you can see the description of the variable in the **Canadian Cannabis Survey (CCS) 2024 PUMF User Guide**, which provides information on what the variable represents and how it was coded:

access_ease_legal	How easy or difficult is it for you to get legal cannabis?
<i>[variable added in 2024 to replace access_ease]</i>	☐ 1 Very difficult ☐ 2 Fairly difficult ☐ 3 Fairly easy ☐ 4 Very easy ☐ 5 I don't know ☐ -7 Missing

In this dataset, -7 generally represents missing. Always check the data dictionary for the specific coding of missing values, “don’t know,” or “refused” responses, as they can vary between datasets.

Data Source: CADS

The Canadian Alcohol and Drugs Survey (CADS) is a national survey conducted by Statistics Canada that collects data on the use of substances, such as alcohol, cannabis and other drugs. The study was conducted by Statistics Canada in 2019 with the cooperation and support of Health Canada. The target population for the survey was all persons aged 15 years and over living in Canada, excluding residents of the Yukon, Northwest Territories, and Nunavut, full-time residents of institutions, and residents of Native reserves.



The screenshot shows the top of a Government of Canada website page. At the top left is the Canadian flag and the text "Government of Canada / Gouvernement du Canada". To the right is a search bar with the text "Search Canada.ca" and a magnifying glass icon. Below the header is a "MENU" button with a downward arrow. Under the menu is a breadcrumb trail: "Canada.ca > Open Government > Search". The main heading is "Canadian Alcohol and Drugs Survey: Public Use Microdata File". Below this heading is a "Have your say" section with two links: "Rate this dataset" and "Provide feedback". To the right of these links is a paragraph of text: "This public use microdata file (PUMF) is from the Canadian Alcohol and Drugs Survey and includes information about the use of substances, including alcohol, cannabis and other drugs. This product is for users who prefer to do their own analysis by focusing on specific subgroups in the population or by cross-classifying variables that were not the focus of our release in The Daily." Below the "Have your say" section is a "Publisher - Current Organization Name: Statistics Canada" label.

[CADS 2019 dataset for CADS](#)

Open Licence

Similar to the [Canadian Cannabis Survey](#), the CADS is also licensed under the [Open Government Licence - Canada](#). This means that the data can be freely used, modified, and shared by anyone for any purpose, as long as they provide proper attribution to the source of the data.

Canadian Alcohol and Drugs Survey: Public Use Microdata File

Have your say
[Rate this dataset](#)
[Provide feedback](#)

Additional Information

This public use microdata file (PUMF) is from the Canadian Alcohol and Drugs Survey and includes information about the use of substances, including alcohol, cannabis and other drugs. This product is for users who prefer to do their own analysis by focusing on specific subgroups in the population or by cross-classifying variables that were not the focus of our release in The Daily.

Publisher - Current Organization Name: Statistics Canada

Licence: [Open Government Licence - Canada](#)

 Government of Canada
Gouvernement du Canada

[Canada.ca](#) > [Open Government](#) > [About Open Government](#)

Open Government Licence - Canada

You are encouraged to use the Information that is available under this licence with only a few conditions.

Using Information under this licence

- Use of any Information indicates your acceptance of the terms below.

Data and Resources

The **Data** and **Resources** tab provides access to the public use data, user guides, and the data dictionary.

Data and Resources

 **Canadian Alcohol and Drugs Survey: Public Use Microdata File**
[English](#) [French](#) [dataset](#) [CSV](#)

[Explore](#)

 **Canadian Alcohol and Drugs Survey: Public Use Microdata File**
[English](#) [French](#) [dataset](#) [SAS](#)

[Explore](#)

 **Canadian Alcohol and Drugs Survey: Public Use Microdata File**
[English](#) [website](#) [HTML](#)

[Explore](#)

 **Canadian Alcohol and Drugs Survey: Public Use Microdata File**
[French](#) [website](#) [HTML](#)

[Explore](#)

The public use data file is available in a ZIP format. You need to open the **Canadian Alcohol and Drugs Survey: Public Use Microdata File** link, and click on **Go to resource** to download the ZIP file.

Data and Resources

Search resources... 



Canadian Alcohol and Drugs Survey: Public Use Microdata File

English French dataset CSV

Explore



Canadian Alcohol and Drugs Survey: Public Use Microdata File

English French dataset SAS

Explore



Canadian Alcohol and Drugs Survey: Public Use Microdata File

English website HTML

Explore



Canadian Alcohol and Drugs Survey: Public Use Microdata File

French website HTML

Explore

[Canada.ca](#) > [Open Government](#) > [Search](#) > [Canadian Alcohol and Drugs Survey: Public Use Microdata File](#)

Canadian Alcohol and Drugs Survey: Public Use Microdata File - Canadian Alcohol and Drugs Survey: Public Use Microdata File

 Go to resource

The ZIP file contains the following files: an instruction on how to cite CADS 2019, data documentation, and the dataset in CSV format.

 How to cite CADS2019_Comment citer ECAD2019	27-Jul-21 10:42 AM	Adobe Acrobat D...	388 KB
 Documentation	05-Aug-21 6:56 AM	File folder	
 Data_donnees	05-Aug-21 6:46 AM	File folder	

Data Documentation

The data documentation folder includes three sub-folders: Codebook_Livre des codes, Guide, and Questionnaires.

 Codebook_Livre des codes	05-Aug-21 6:56 AM	File folder
 Guide	05-Aug-21 6:56 AM	File folder
 Questionnaires	05-Aug-21 6:56 AM	File folder

The **Guide** folder contains user guides (in English and French) that provide an overview of the survey, including the methodology, data collection process, data processing, data quality, and sampling weights.

 ECAD2019_GuF	French	14-Jul-21 6:47 AM	Adobe Acrobat D...	1,653 KB
 CADS2019_UgE	English	14-Jul-21 6:45 AM	Adobe Acrobat D...	1,306 KB

The **Codebook_Livre des codes** folder contains codebooks (in English and French) that provide detailed information about the variables in the dataset, including variable names, labels, and values. There are two types of codebooks: those with and those without variable frequencies. But both codebooks provide the same information about the variables in the dataset.

Codebook including frequencies for variables

 ECAD2019_FMGD_lvCd	French	22-Dec-20 11:50 AM	Adobe Acrobat D...	1,948 KB
 CADS2019_PUMF_Cdbk		22-Dec-20 11:41 AM	Adobe Acrobat D...	1,925 KB
 ECAD2019_FMGD_lvCd_zerofreq		22-Dec-20 11:24 AM	Adobe Acrobat D...	1,885 KB
 CADS2019_PUMF_Cdbk_zerofreq	English	20 11:16 AM	Adobe Acrobat D...	1,863 KB

Codebook excluding frequencies for variables

The **Questionnaires** folder contains the survey questionnaires (in English and French) that were used to collect the data.

Data

The **Data_donnees** folder contains the dataset in CSV format. You can open the dataset in Excel or any statistical software to explore the data. The row represents the respondent, and the column represents the variable:

CADS															
File Home Insert Draw Page Layout Formulas Data Review View Automate Help Acrobat															
A1	PUMFID														
	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
	PUMFID	REFYEAR	REFMONT	PROV	DVURBAN	MHHTYPEB	SEX	DV_AGE	HHLI_01	HDSIZE	HWB_05	HWB_10	ALC_05	ALC_10P	ALC_J
1	1000	2019	12	46	2	3	1	8	1	2	2	2	1	16	
2	1001	2019	9	46	2	6	1	5	1	4	1	1	1	17	
3	1002	2019	6	35	1	3	2	8	1	2	3	2	1	18	
4	1003	2019	8	11	1	3	1	8	1	3	2	2	1	18	
5	1004	2019	6	12	1	3	1	9	1	3	2	3	1	16	
6	1005	2019	10	12	1	3	1	8	1	2	2	1	1	99	
7	1006	2019	10	35	1	3	1	6	1	1	1	1	1	18	
8	1007	2019	12	48	1	3	1	7	1	2	4	4	1	16	
9	1008	2019	10	46	1	6	1	9	1	3	1	1	1	17	
10	1009	2019	6	47	1	3	2	9	1	2	2	2	1	16	
11	1010	2019	11	24	1	8	2	7	1	2	3	2	1	16	

You can refer to the codebook to understand the variables that you are interested in for your analysis. For example, if you are interested in the variable DVURBAN, you can refer to the codebook to find out that the variable (e.g., for the English version of the codebook with frequencies):

Variable Name:	DVURBAN	Length:	1.0	Position:	14
Question Name:					
Concept:	Characteristics of community where the respondent lives				
Question Text:					
Universe:	All respondents				
Note:	The category 'Population centre' is historically comparable to 'Urban' which was used in previous years for CTUMS.				
Source:					
Answer Categories	Code	Frequency	Weighted Frequency	%	
Population centre	1	7,208	21,321,106	68.8	
Rural	2	2,438	5,122,120	16.5	
Not stated	9	647	4,552,224	14.7	
Total		10,293	30,995,450	100.0	

Data Source: CTNS

The Canadian Tobacco and Nicotine Survey (CTNS) is a cross-sectional survey that collects information on tobacco and nicotine use among Canadians aged 15 years and older. The survey is conducted by Statistics Canada on behalf of Health Canada and provides valuable insights into the prevalence of cigarette smoking, vaping, cannabis, and alcohol use.



[CTNS 2022 dataset for GfS](#)

Open Licence

The CTNS is licensed under the [Open Government Licence - Canada](#). This means that the data can be freely used, modified, and shared by anyone for any purpose, as long as they provide proper attribution to the source of the data.

Data and Resources

The **Data and Resources** tab provides access to the public use data, user guides, and the data dictionary.

Data and Resources

Search resources...

	Canadian Tobacco and Nicotine Survey: Public Use Microdata File	Explore
English	website	HTML
	Canadian Tobacco and Nicotine Survey: Public Use Microdata File	Explore
French	website	HTML
	Canadian Tobacco and Nicotine Survey: Public Use Microdata File	Explore
English	French	dataset
	Canadian Tobacco and Nicotine Survey: Public Use Microdata File	Explore
English	French	dataset
		SAS

The public use data file is available in a ZIP format. You can click on the Canadian Tobacco and Nicotine Survey: Public Use Microdata File link, followed by Go to resource link to download the ZIP file.

Data and Resources

Search resources...

	Canadian Tobacco and Nicotine Survey: Public Use Microdata File	Explore
English	website	HTML
	Canadian Tobacco and Nicotine Survey: Public Use Microdata File	Explore
French	website	HTML
	Canadian Tobacco and Nicotine Survey: Public Use Microdata File	Explore
English	French	dataset
	Canadian Tobacco and Nicotine Survey: Public Use Microdata File	Explore
English	French	dataset
		SAS

[Canada.ca](#) > [Open Government](#) > [Search](#) > [Canadian Tobacco and Nicotine Survey: Public Use Microdata File](#)

Canadian Tobacco and Nicotine Survey: Public Use Microdata File - Canadian Tobacco and Nicotine Survey: Public Use Microdata File

[Go to resource](#)

Similar to the [Canadian Alcohol and Drugs Survey \(CADS\)](#) tutorial, the ZIP file for CTNS contains the following files: citation instructions, data documentation, and the dataset in CSV format. The data documentation, questionnaire, and codebook are also the same as those for CADS. The only difference is that the data documentation and questionnaire for CTNS are labelled CTNS rather than CADS.

Data Source: CSADS

The Canadian Student Alcohol and Drugs Survey (CSADS) is a national survey that collects data on the alcohol and drug use patterns of Canadian students in grades 7 to 12. The survey is conducted every two years and provides valuable insights into the prevalence and trends of substance use among Canadian youth.

Canadian Student Alcohol and Drugs Survey (CSADS): Public Use Microdata File

Have your say

★★★★★

[Rate this dataset](#)

[Provide feedback](#)

Every two years, Health Canada conducts the Canadian Student Alcohol and Drugs Survey (CSADS), a cross-sectional survey of students in grades 7 to 12 across the Canadian provinces.

With tens of thousands of participants each cycle, it is one of the largest youth surveys in the world. For over 30 years, the survey—previously known as the Youth Smoking Survey (YSS; pre-2014) and the Canadian Student Tobacco, Alcohol and Drugs Survey (CSTADS;

[CSADS Canada website files CTNS](#)

Open Licence

The CTNS is licensed under the [Open Government Licence - Canada](#). This means that the data can be freely used, modified, and shared by anyone for any purpose, as long as they provide proper attribution to the source of the data.

Data and Resources

The **Data and Resources** tab provides access to the public use data, user guides, the data dictionary, and information on bootstrap weights.

Data and Resources

Search resources...

	2021-2022 CSTADS: PUMF — User Guide	Explore
	2021-2022 ECTADE : FMGD — Mode d'emploi	Explore
	2021-2022 CSTADS: PUMF — Data	Explore
	2021-2022 CSTADS: PUMF — Bootstrap Weights	Explore
	2023-2024 CSADS: PUMF — User Guide	Explore
	2023-2024 ECADE : FMGD — Mode d'emploi	Explore
	2023-2024 CSADS: PUMF — Data	Explore
	2023-2024 CSADS: PUMF — Bootstrap Weights	Explore

To download the 2021–2022 CSTADS, you can click on the [2021-2022 CSTADS: PUMF – Dataset in CSV format](#) link. The data is available to download in various formats. Under the **Download** tab, you can find the formats. For example, you can download the data in CSV format by clicking on the [CSV](#) link:

MENU

[Canada.ca](#) > [Open Government](#) > [Search](#) > [Canadian Student Alcohol and Drugs Survey \(CSADS\): Public Use Microdata File](#)

Canadian Student Alcohol and Drugs Survey (CSADS): Public Use Microdata File - 2021-2022 CSTADS: PUMF — Data

 **Download**
 **Data API**

 **View**

[CSV](#)
[TSV](#)
[JSON](#)
[XML](#)

After downloading, you can open the data in Excel, Google Sheets, or any statistical software. The data is structured in a tabular format, with each row representing an individual respondent and each column representing different variables:

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
1	SEQID	PROVID	GRADE	DVGENDEI	DVURBAN	DVRES	DVORIENT	DVDESCR	WTPUMF	GH_010	GH_020	SS_010	SS_020	TS_011	SS_030
2	18338	11	9	2	1	3	2	1	1.164716	2	3	2	96	4	96
3	16111	24	10	1	1	1	2	4	36.14117	3	3	2	96	4	96
4	20587	10	7	2	1	1	2	1	1.448578	2	1	2	96	4	96
5	54568	12	8	99	2	1	1	1	3.03666	4	5	2	96	3	96
6	40991	10	7	1	2	1	2	1	3.745233	2	3	2	96	4	96
7	5254	12	7	1	1	2	2	1	10.0715	3	3	2	96	4	96
8	10440	35	9	99	1	1	99	1	75.0031	5	4	2	96	4	96
9	38149	48	12	1	1	1	2	1	25.83337	1	3	1	16	4	2
10	32243	59	11	2	1	1	2	1	47.16534	1	3	1	16	3	2
11	5513	35	11	1	2	1	99	1	99.52681	2	5	2	96	3	96
12	37146	48	10	1	1	2	2	4	24.62732	3	2	2	96	4	96
13	21979	46	9	1	2	1	2	1	65.69607	1	4	2	96	4	96
14	33925	12	7	1	2	3	1	1	3.60877	3	4	2	96	4	96
15	19919	94	7	2	1	2	00	00	76.14914	9	9	9	00	4	00

The user guide gives detailed information on the survey methodology, data collection process, and how to use the public use microdata file (PUMF). You can also find the variable names and their descriptions in this DOCX file.

2021-2022 CSTADS: PUMF — User Guide

[English](#)
[guide](#)
[DOCX](#)

Explore

2021-2022 ECTADE : FMGD — Mode d'emploi

[French](#)
[guide](#)
[DOCX](#)

Explore

2021-2022 CSTADS: PUMF — Data

[English](#)
[French](#)
[dataset](#)
[CSV](#)

Explore

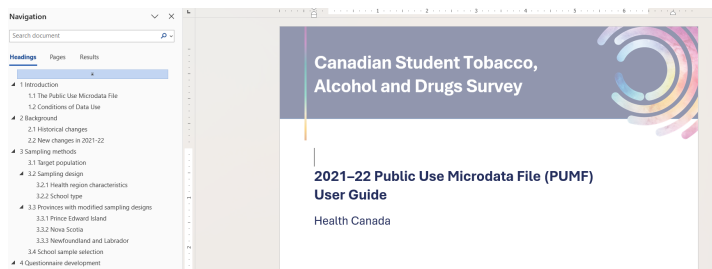
2021-2022 CSTADS: PUMF — Bootstrap Weights

[English](#)
[French](#)
[dataset](#)
[CSV](#)

Explore

Canadian Student Alcohol and Drugs Survey (CSADS): Public Use Microdata File - 2021-2022 CSTADS: PUMF — User Guide

Download



To find a variable description, you are referred to the **User Guide**. For example, the variable GH_010 is described as follows:

Navigation			
SH_010		WTPUMP	9
1 result			
Headings	Pages	Results	
6.2.1 Skip patterns and inconsistent responses			
6.2.2 Imputation of core smoking responses			
6.2.3 Reducing underestimation risk in the PLME			
6.2.4 Standard codes			
6.3 Survey weights			

		Unknown: your response was imputed for perturbation to maintain privacy; more information on how these weights were generated can be found in section 6.3)	
SH_010	10	In general, would you say your physical health is excellent, very good, good, fair or poor?	1 = Excellent 2 = Very good 3 = Good 4 = Fair 5 = Poor 6 = I do not know 99 = Not stated
			1 = Excellent 2 = Very good

Data Source: PHAC Health Infobase

The Health Infobase is a comprehensive digital platform maintained by the Public Health Agency of Canada (PHAC). The Infobase provides access to a wide range of health-related data tools, infographics, charts and blogs. There are 182 data sources available on the Infobase. The Health Infobase serves as a central hub for national surveillance data, interactive data tools, and public health reports. The platform is frequently updated with the latest surveillance data.



[PHAC Health Infobase](#)

Data Categories

The Infobase provides access to health data under six categories:

- **Diseases and disorders:** Comprehensive tracking of chronic conditions such as cancer, cardiovascular diseases, and COVID-19.

- **Drugs and vaccines:** Data on prescription drugs, vaccine coverage, and immunization rates across the country.
- **Mental health, healthy living and environment:** Information on mental health indicators, physical activity levels, nutrition, and environmental factors such as wildfires.
- **Injuries and death:** Data on death, suicide, violence, and drowning across regions.
- **Population groups and demography:** Data on birth, maternity, children, family, and disabilities.
- **Substance use:** Data on substance use such as alcohol, cannabis use, opioids, and smoking.

Under each category, users can select the data for specific health topics. The three top categories are **Respiratory diseases**, **Opioids** and **Mental health**.

Find a data product

Use filters to find specific data products. You can filter by **category**, by **format**, or by typing in **keywords**.

▼ Categories

Diseases and disorders

+

Drugs and vaccines

+

Mental health, healthy living and environment

+

Injuries and death

+

Population groups and demography

+

Substance use

+

▼ Filter

Type in keywords to filter results as you type



Healthy Canadians and Communities Fund

Data exploration tool

Inequalities

Interactive map of funded interventions across Canada that support Canadians who experience health inequalities and are at greater risk of developing chronic disease.

Last updated: 2026-04-07

Find a data product

Use filters to find specific data products. You can filter by **category**, by **format**, or by typing in **keywords**.

▼ Categories

Diseases and disorders

—

☐ All diseases and disorders

☒ Blood and metabolic diseases

☐ Chronic diseases

☐ Cancer

☐ COVID-19

▼ Filter

Type in keywords to filter results as you type

Blood and metabolic diseases X

Showing 5 filtered from 182 total products

Clear all filters



Canadian Chronic Disease Surveillance System (CCDSS)

Data exploration tool

Chronic diseases

Cardiovascular diseases

Musculoskeletal disorders

Neurological diseases

Respiratory diseases

Mental health

Blood and metabolic diseases

Autism

264

You can filter the data by category, by format, or by typing in keywords.

Find a data product

Use filters to find specific data products. You can filter by **category**, by **format**, or by typing in **keywords**.

▼ Categories

- Diseases and disorders +
- Drugs and vaccines +
- Mental health, healthy living and environment +
- Injuries and death +
- Population groups and demography +
- Substance use +

▼ Filter covid

Showing 27 filtered from 182 total products
[Clear all filters](#)

 [COVID-19 Vaccine mistrust among Black Communities in Canada](#)

[Interactive report](#) | [COVID-19](#) | [Vaccines](#)

Interactive report on COVID-19 vaccine mistrust among Black communities in Canada, based on data from the 2022 CanBlack Vax survey.

You can also filter the data by type, such as blog, infographic, or report.

▼ Type

- ☐ Blog
- ☐ Data exploration tool
- ☐ Indicator framework
- ☐ Infographic
- ☐ Interactive report

at greater risk of developing chronic disease.
Last updated: 2026-04-07

 [Transfusion and transplantation-related adverse events](#)

[Interactive report](#) | [Injuries](#)

Summary of national surveillance systems for transfusion errors, transfusion-transmitted injuries, and cells, tissues and

Data Availability

The level of information provided in the data sources varies. For some data sources/products, you can find blogs, infographics, and reports along with individual-level data. However, for some data sources/products, you may only find the summary statistics without individual-level data, i.e., only aggregated-level data. For example, the **Human Papillomavirus Indicator Framework** product provides quick statistics, data tools and a list of publications:

▼ Categories

Diseases and disorders

☐ All diseases and disorders
 ☐ Blood and metabolic diseases
 ☐ Chronic diseases
 ☒ Cancer
 ☐ COVID-19
 ☐ Cardiovascular diseases
 ☐ Developmental

▼ Filter

Type in keywords to filter results as you type

Cancer X

Showing 10 filtered from 182 total products

Clear all filters



[Human Papillomavirus Indicator Framework \(HPVIF\)](#)

[Data exploration tool](#)
[Infections](#)
[Vaccines](#)

Cancer

The Human Papillomavirus Indicator Framework (HPVIF) presents HPV vaccination coverage and HPV outcome indicators for Canada.

Last updated: 2025-12-15



Government of Canada

Gouvernement du Canada

Français

Search Canada.ca

🔍

Canada.ca

> Health

> Health science, research and data

> Health Infobase

Human Papillomavirus Indicator Framework (HPVIF)

The Human Papillomavirus Indicator Framework (HPVIF) presents HPV vaccination coverage and HPV outcome indicators for Canada. Sub-national estimates, sociodemographic breakdowns, and trends over time are presented, when possible, as well as international comparisons. Since HPV vaccination can prevent HPV-associated cancers, this framework will serve as a valuable resource for monitoring Canada's progress against HPV.

Last updated: 2025-12-15

Quick stats

Data tool

Publications

On this page

- [About](#)
- [Quick Stats overview](#)
- [Suggested citation](#)

The Drug and alcohol use in Canada: Postsecondary students product provides summary statistics as well as links to download data in CSV format:

Substance use

☐ All substance use
 ☒ Alcohol
 ☐ Cannabis
 ☐ Opioids
 ☐ Smoking and vaping
 ☐ Stimulants

Key findings from the 2024-2025 Canadian Postsecondary Education Alcohol and Drug Use Survey (CPADS).

Last updated: 2025-12-22



[Drug and alcohol use in Canada: Postsecondary students](#)

[Data exploration tool](#)
[Alcohol](#)
[Cannabis](#)

[Opioids](#)
[Prescription drugs](#)
[Smoking and vaping](#)

Stimulants

Explore Health Canada's latest data on drug and alcohol use by postsecondary students in Canada.

266

Postsecondary students

Drug and alcohol use in Canada

Explore Health Canada's latest data on drug and alcohol use by postsecondary students in Canada.

Last updated: 2025-12-31

General population

Youth

Postsecondary students

About the postsecondary survey

Health Canada monitors the prevalence of substance use and its impact on postsecondary students through the Canadian Postsecondary Education Alcohol and Drug Use Survey (CPADS). Governments and non-governmental organizations use this information to shape policies and programs that address substance use and support students.

In the 2024–25 school year, 29,371 students from 9 provinces and 2 territories completed the survey. The results focus on

Additional resources



[Read](#)

[the 2024-2025 summary report](#)



[Read](#)

[previous summary reports](#)



[Download](#)

[dashboard data \(.csv\)](#)

We'd love your feedback! Please take a minute to complete this short and anonymous survey.

[Next](#)

Data Source: CCDSS

The Canadian Chronic Disease Surveillance System (CCDSS) is a collaborative network of provincial and territorial chronic disease surveillance systems in Canada. The CCDSS is supported by the Public Health Agency of Canada (PHAC) and provincial/territorial governments. The CCDSS was established to provide the estimates of the prevalence and incidence of chronic diseases in Canada, as well as to monitor trends over time for more than 20 chronic diseases to help inform health policies and resource planning. The CCDSS is part of the PHAC Health Infobase that we covered in the previous tutorial on [Health Infobase](#). The CCDSS uses health administrative data, such as hospital discharge records, physician billing claims, and prescription drug data.



[CCDSS website](#)

Key Indicator Categories

The CCDSS website provides a variety of indicators related to chronic diseases, such as autism, cardiovascular diseases,

chronic respiratory diseases, diabetes, mental health disorders, musculoskeletal disorders, neurological disorders, and multimorbidity.

Diseases, conditions and indicators

Table 1: Diseases, conditions and indicators included in the CCDSS

Morbidity and mortality	Health events and complications	Use of health services
- incidence/rate of newly identified cases - prevalence (cumulative and active for select diseases/conditions) - all-cause mortality	- annual prevalence - all-cause mortality following a health event	annual prevalence

Table 2: CCDSS case definitions and surveillance period reported

Disease, condition and indicator	Age	Case definition summary	First reported year	Last reported year
Autism				
Autism	1-19	One or more hospital separation records, or two or more physician claims	2000-2001	2023-2024
Cardiovascular diseases				
Acute myocardial infarction	20+	One or more hospital inpatient admission records	2000-2001	2023-2024

Data Tool

The CCDSS website provides a data tool that allows users to explore and visualize the data on chronic diseases in Canada. The data tool provides interactive charts and tables that allow users to explore the data by province/territory.

About the Canadian Chronic Disease Surveillance System (CCDSS)

The CCDSS is a collaborative network of provincial and territorial surveillance systems, supported by the Public Health Agency of Canada (PHAC). The system collects data on all residents who are eligible for provincial or territorial health insurance. It can generate national estimates and trends over time for over 20 chronic diseases and conditions, and other selected health outcomes. To identify people with chronic diseases and conditions, provincial and territorial health insurance registry records are linked using a unique personal identifier to the corresponding physician claims, hospital separation records and prescription drug records.

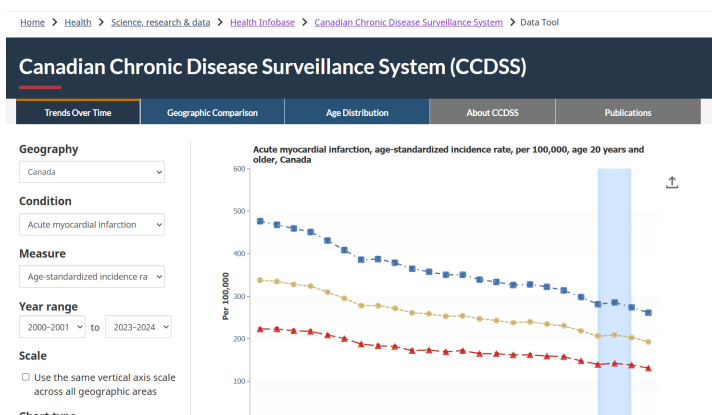
Key information about the CCDSS methods and case definitions is presented below. More detailed documents are available here:

[CCDSS summary of methods](#); [CCDSS case definitions](#); [CCDSS COVID-19 guidance for data users](#).

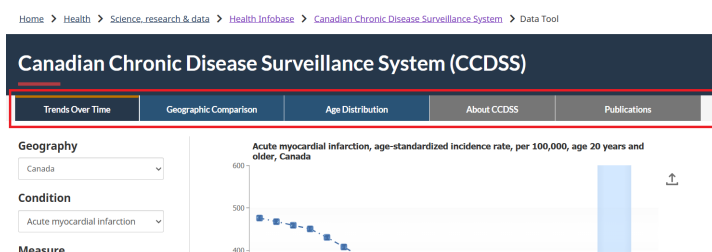
Use the CCDSS Data Tool

Users can use the data tool to explore the prevalence and incidence of cases, mortality rates, and health service utilization, and to track individuals living with two or more chronic conditions.

[CCDSS Data Tool](#)

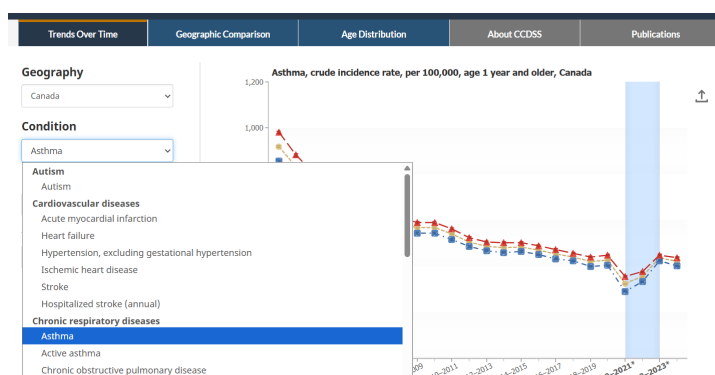


Users can explore trends over time, compare data across provinces/territories, and view the distribution of cases by age group. Under the **Publication** tab, you can find information on the repository for data-driven insights, fact sheets, peer-reviewed publications, and reports.



Interactive chart example

Consider **Asthma** under **Chronic respiratory diseases** as an example.



You can explore different summary measures by filtering by geography, measure, and year. Again, you can further explore trends over time, compare data across provinces/territories, and view the distribution of cases by age group.



You can also download the summary data in CSV format for further analysis. To do that, you need to scroll down to the bottom of the **Summary Table**, and click on the **Download** button.

Males	2019-2020	404	1.54	401-407	0.38	68,605
Males	2020-2021*	292	1.31	290-295	0.45	49,690
Males	2021-2022*	333	1.39	331-336	0.42	57,745
Males	2022-2023*	425	1.55	422-428	0.36	75,625
Males	2023-2024	402	1.48	399-405	0.37	73,400

Download Detailed Table: Asthma, crude incidence rate

Save Summary Table

Once you open the CSV file in Excel or other software, you will find that the data is not at the individual or person

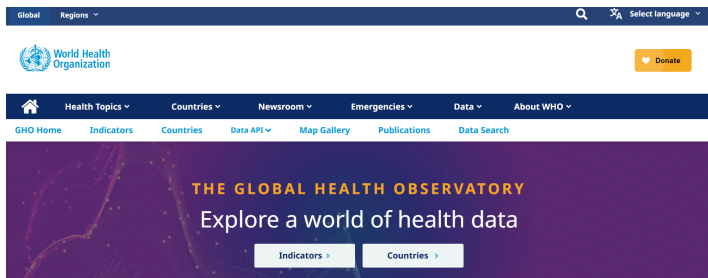
level. Instead, the data are aggregated at the country and province/territory levels, and summary measures are provided for both sexes and for years. For example, see row 4. The **Geography** column says *Canada* and the **Rate (per 100,000)** column says *919*. This is an example of aggregate data.

	A	B	C	D	E	F	G	H	I	J	K	L
1	Data Table: Asthma, Crude incidence rate, per 100,000											
2												
3	Geography	Sex	Age group type	Age group	Fiscal year	Rate (per 100,000)	Standard Error	Lower 95% CI	Upper 95% CI	Rate CV (%)	Case counts	Population
4	Canada	Both sexes	Total	1+	2000-2001	919	1.8	916	923	0.2	260730	28367690
5	Canada	Females	Total	1+	2000-2001	961	2.6206	976	986	0.27	140225	14289240
6	Canada	Males	Total	1+	2000-2001	856	2.4657	851	861	0.29	120505	14078655
7	Canada	Both sexes	Total	1+	2001-2002	833	1.7103	830	837	0.21	237405	28488200
8	Canada	Females	Total	1+	2001-2002	883	2.4814	878	886	0.26	126365	14337965
9	Canada	Males	Total	1+	2001-2002	783	2.3526	779	788	0.3	110820	14150240
10	Canada	Both sexes	Total	1+	2002-2003	762	1.6318	759	765	0.21	218155	28623720
11	Canada	Females	Total	1+	2002-2003	804	2.3637	800	809	0.29	115840	14399350
12	Canada	Males	Total	1+	2002-2003	719	2.2487	715	724	0.31	102310	14224360
13	Canada	Both sexes	Total	1+	2003-2004	727	1.5893	724	730	0.22	209140	28774260
14	Canada	Females	Total	1+	2003-2004	773	2.3117	769	778	0.3	111860	14468175
15	Canada	Males	Total	1+	2003-2004	680	2.1801	676	684	0.32	97275	14306080
16	Canada	Both sexes	Total	1+	2004-2005	696	1.5513	693	699	0.22	201445	28932495
17	Canada	Females	Total	1+	2004-2005	738	2.2531	734	743	0.31	107350	14541550
18	Canada	Males	Total	1+	2004-2005	654	2.1316	650	658	0.33	94095	14398940

Because this data is not at the person-level, and the summary measures are aggregated, you do not need a codebook to translate columns. However, you cannot use this data to calculate the prevalence or incidence of asthma in Canada because it is already aggregated. You can only use this data to explore the trends over time, compare data across provinces/territories, and view the distribution of cases by age group and sex. The individual patients have already been erased to protect their privacy.

Data Source: GHO

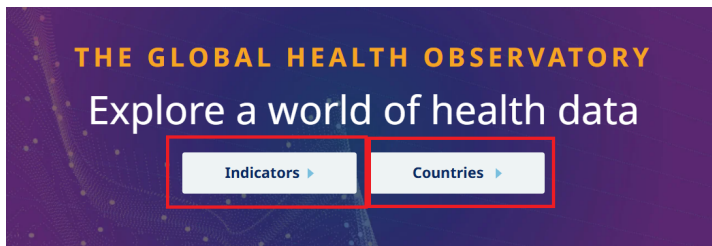
The Global Health Observatory (GHO) is the World Health Organization's (WHO) main health statistics repository. The GHO provides data on a wide range of health topics, such as mortality, morbidity, risk factors, health systems, and health equity. The GHO collects data from member states, international organizations, and other sources to provide a comprehensive picture of global health. The GHO is a valuable resource for researchers, policymakers, and public health professionals who are interested in understanding global health trends and disparities. The GHO provides various ways to interact with its data, including interactive maps and downloadable CSV files.



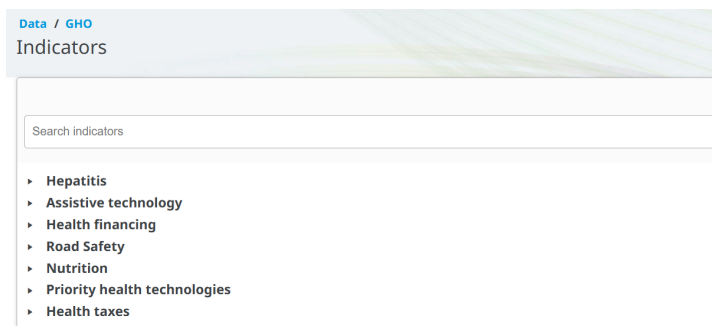
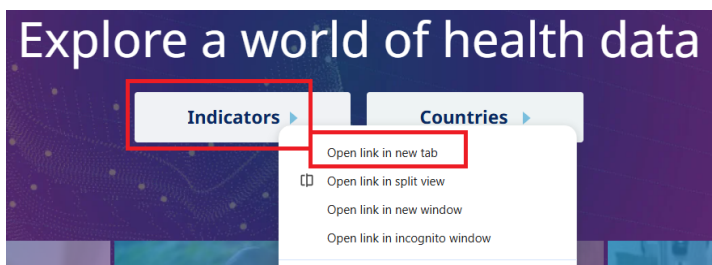
[WHO Global Health Observatory](#)

World Health Data

You can browse information by broader health indicators or by specific countries.



To explore the data by indicators, click on the **Indicators** tab. You can find a list of indicators across categories such as Hepatitis, Nutrition, Mental health, and so on.



You can also use the search bar to search for specific indicators. For example, if you search for **Life expectancy at birth**, you will find a list of indicators related to life expectancy, such as **Life expectancy at birth (years)**.

Data / GHO
Indicators

Life expectancy at birth

- World Health Statistics
 - Life expectancy and life tables
 - Life expectancy at birth (years)
- World Health Statistics
 - Life expectancy at birth (years)
- Global Health Estimates: Life expectancy and leading causes of death and disability
 - Mortality and global health estimates
 - Life expectancy at birth (years)

To explore the data by country, click on the **Countries** tab. You can find a list of countries and regions. You can click the country name to see the data for that country.

World Health Organization

Data
WHO Glob
Indicators Countries Dashb

All A B C D E F G H I J K L M N O P Q R S T U V W X Y Z

A

Afghanistan
Albania
Algeria
Andorra
Angola
Antigua and Barbuda
Argentina
Armenia
Australia
Austria
Azerbaijan

B

Bahamas
Bahrain
Bangladesh
Barbados

Australia
Health data overview for Australia

Population26 451 124 (2023)
Current health expenditure (% of GDP)10.54 (2021)
WHO regionWestern Pacific
World Bank income levelHigh income (HIC)

Population, Australia

In Australia, the current population is 26,451,124 as of 2023 with a projected increase of 23% to 32,506,969 by 2050.

Population growth rate
Australia, 2023
0.95% +0.012 percentage points
rate change since 2022

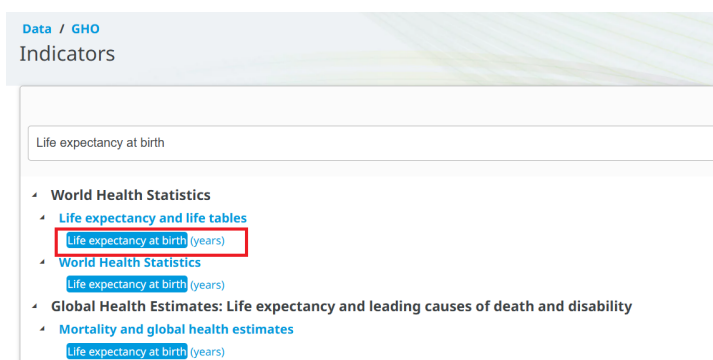
On this page

- Population
- Life expectancy
- Leading causes of death
- Health statistics

275

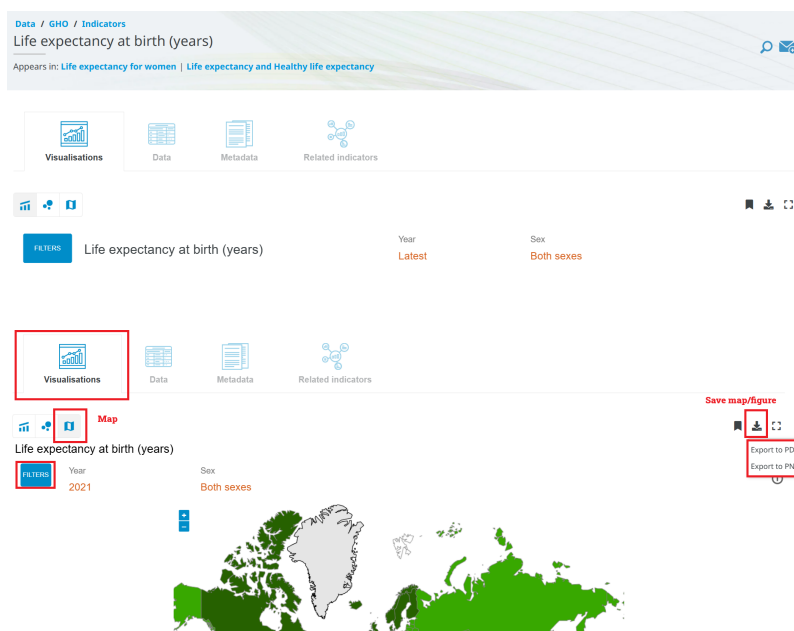
Data by indicator example

Let's click on the Life expectancy at birth (years) indicator to see the details:



You can find different visualization options, such as charts and maps. You can also download the chart and map to your computer.

[GHO Indicators](#)



To see the data table, you can check the **Data** section. You can also download the data in CSV format for further analysis. To do that, you need to click on the **Download** button at the top right corner of the data table.

Visualisations

Data

Metadata

Related indicators

EXPORT DATA in CSV format:
Right-click here & Save link

Filters

Last updated: 2024-08-02

Indicator

Location

Life expectancy at birth (years)

Both sexes

Male

Female

Life expectancy at age 60 (years)

Both sexes

Male

Female

2021

2020

2019

2018

2017

59.1 [58.3-59.9]

57.4 [56.6-58.2]

61.0 [60.2-61.7]

13.7 [13.2-14.2]

13.6 [13.1-14.1]

13.8 [13.3-14.3]

Similar to the [Canadian Chronic Disease Surveillance System \(CCDSS\) database](#), the life expectancy data is also at an aggregated level, which means that the data is summarized at the country level, and does not contain individual-level data:

IndicatorCode	Indicator	ValueType	ParentLocationCode	ParentLocation	Location type	SpatialDimValueCode	Location	Period type	Period	Dim1	FactValue	
1	WHOSIS_000001	Life expectancy at birth (years)	text	AFR	Africa	Country	LSO	Lesotho	Year	2021	Male	48.73
2	WHOSIS_000001	Life expectancy at birth (years)	text	AFR	Africa	Country	CAF	Central Af Year	2021	Male	49.57	
3	WHOSIS_000001	Life expectancy at birth (years)	text	AFR	Africa	Country	LSO	Lesotho	Year	2021	Both sexes	51.48
4	WHOSIS_000001	Life expectancy at birth (years)	text	AFR	Africa	Country	SWZ	Eswatini	Year	2021	Male	51.64
5	WHOSIS_000001	Life expectancy at birth (years)	text	EMR	Eastern Mediterranean	Country	SOM	Somalia	Year	2021	Male	51.75
6	WHOSIS_000001	Life expectancy at birth (years)	text	AFR	Africa	Country	CAF	Central Af Year	2021	Both sexes	52.31	
7	WHOSIS_000001	Life expectancy at birth (years)	text	EMR	Eastern Mediterranean	Country	SOM	Somalia	Year	2021	Both sexes	53.95
8	WHOSIS_000001	Life expectancy at birth (years)	text	AFR	Africa	Country	LSO	Lesotho	Year	2021	Female	54.6
9	WHOSIS_000001	Life expectancy at birth (years)	text	AFR	Africa	Country	SWZ	Eswatini	Year	2021	Both sexes	54.59
10	WHOSIS_000001	Life expectancy at birth (years)	text	AFR	Africa	Country	MOZ	Mozambique	Year	2021	Male	54.8
11	WHOSIS_000001	Life expectancy at birth (years)	text	AFR	Africa	Country	CAF	Central Af Year	2021	Female	55.38	
12	WHOSIS_000001	Life expectancy at birth (years)	text	AFR	Africa	Country	GNB	Guinea-Bi Year	2021	Male	56.08	
13	WHOSIS_000001	Life expectancy at birth (years)	text	AFR	Africa	Country	ZWE	Zimbabwe	Year	2021	Male	56.19
14	WHOSIS_000001	Life expectancy at birth (years)	text	EMR	Eastern Mediterranean	Country	SOM	Somalia	Year	2021	Female	56.53
15	WHOSIS_000001	Life expectancy at birth (years)	text	AFR	Africa	Country	SSD	South Sud Year	2021	Male	56.48	
16	WHOSIS_000001	Life expectancy at birth (years)	text	AFR	Africa	Country	NAM	Namibia	Year	2021	Male	57.27
17	WHOSIS_000001	Life expectancy at birth (years)	text	AFR	Africa	Country	NAM	Namibia	Year	2021	Male	57.27

The data on the website, as well as the downloaded data in CSV format, both provide the point estimate with 95% confidence interval:

Visualisations

Data

Metadata

Related indicators

EXPORT DATA in CSV format:
Right-click here & Save link

Filters

Last updated: 2024-08-02

Indicator

Location

Life expectancy at birth (years)

Both sexes

Male

Female

Life expectancy at age 60 (years)

Both sexes

Male

Female

2021

2020

2019

2018

59.1 [58.3-59.9]

57.4 [56.6-58.2]

61.0 [60.2-61.7]

13.7 [13.2-14.2]

13.6 [13.1-14.1]

13.8 [13.3-14.3]

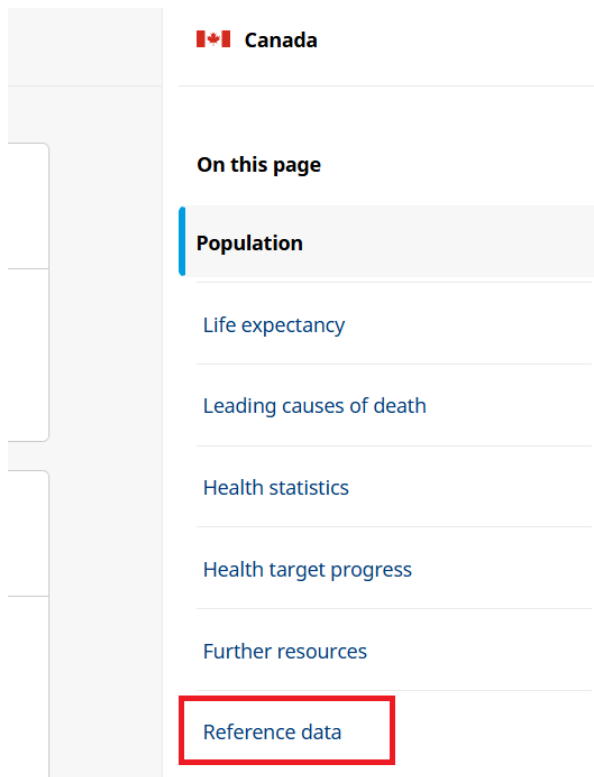
Data by country example

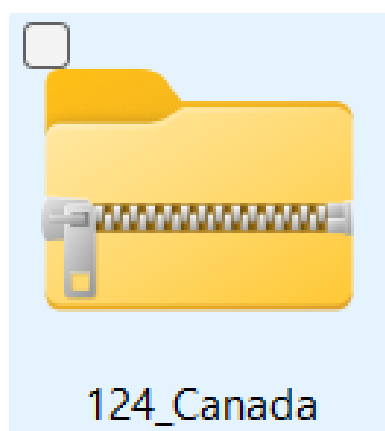
Let's click on **Canada** to explore the data for Canada.



You can explore different indicators for Canada. You can also download all indicators as a ZIP file by clicking **Reference data**.

[GHO Data by Country](#)





Once you unzip the folder, you will find several folders. Each folder contains a codelist, data, a data dictionary, along with a license and terms of use for different indicators.

1F96863_3.4.1 - cardiovascular disease, cancer, diabetes or chronic respiratory disease	03-Mar-26 10:33 AM	File folder
2D6FBE4_3.3.5 - Neglected tropical diseases	03-Mar-26 10:33 AM	File folder
5C8435F_3.C.1 - Health worker density and distribution	03-Mar-26 10:33 AM	File folder
6A64C9A_7.1.2 - Clean fuels	03-Mar-26 10:33 AM	File folder
12EE54A_6.2.1 - Safely managed sanitation & hand-washing	03-Mar-26 10:33 AM	File folder
168BF41_3.4.2 - Suicide	03-Mar-26 10:33 AM	File folder
45CA7C8_3.C.1 - Health worker density and distribution	03-Mar-26 10:33 AM	File folder
75DDA77_3.A.1 - Age-standardized prevalence of tobacco use	03-Mar-26 10:33 AM	File folder
77D059C_3.3.1 - HIV infections	03-Mar-26 10:33 AM	File folder
84FD3DE_3.9.3 - Unintentional poisoning	03-Mar-26 10:33 AM	File folder
608DE39	03-Mar-26 10:33 AM	File folder
1548EA3_6.1.1 - Safely managed drinking water	03-Mar-26 10:33 AM	File folder
217795A_3.C.1 - Health worker density and distribution	03-Mar-26 10:33 AM	File folder
361734E_16.1.1 - Intentional homicide	03-Mar-26 10:33 AM	File folder
1772666_3.1.2 - Births attended by skilled health personnel	03-Mar-26 10:33 AM	File folder

For example, the 77D059C_3.3.1 - HIV infections folder contains the following files:

 124_77D059C_Code list_2025-03-26	13-Apr-26 11:43 PM
 124_77D059C_Dataset_2025-03-26	13-Apr-26 11:43 PM
 124_77D059C_License & Terms of Use_2025-03-26	13-Apr-26 11:43 PM
 124_77D059C_Metadata_2025-03-26	13-Apr-26 11:43 PM
 DDS - Data Dictionary 2024-12-12	13-Apr-26 11:43 PM

Metadata

The GHO provides metadata for each indicator, providing important context for understanding the data and its limitations. For example, the metadata for the **Hepatitis - new infections** indicator includes information on the definition of the indicator, the methods used to calculate the indicator, data sources, and other relevant information.

[Indicator Metadata Registry List](#)

Data / GHO

Indicators

Search indicators

Hepatitis

Hepatitis

Hepatitis - new infections

Hepatitis - number of persons living with chronic hepatitis

Hepatitis - deaths caused by chronic hepatitis, number

Hepatitis - prevalence of chronic hepatitis among the general population

Hepatitis - deaths caused by chronic hepatitis B and C (per 100 000)

Data / GHO / Indicators

Hepatitis - new infections

Appears in: [Hepatitis cases and deaths](#)

Data

Metadata

Related indicators

Indicator name:

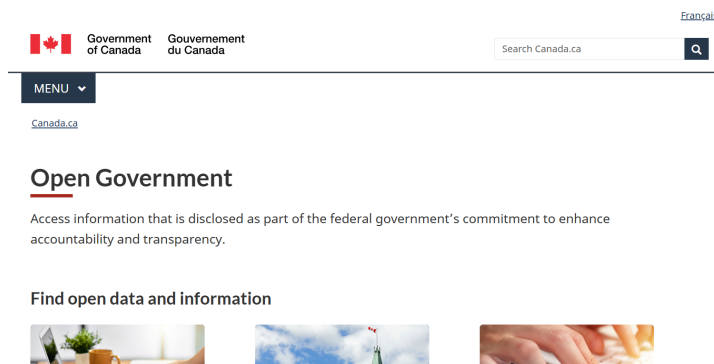
Hepatitis - (chronic HBV and HCV) incidence

Short name:

Chronic hepatitis incidence number

Data Source: Open Portal

Open Government Portal Canada is a platform that provides access to searching for a wide range of public data. Users can explore and download data in different formats under the [Open Government Licence - Canada](#).

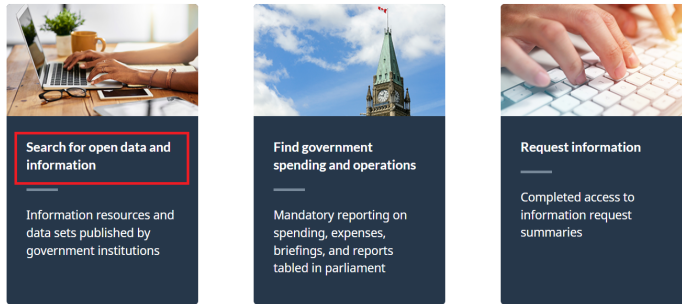


[Open Government Portal Canada](#)

Search for Open Data

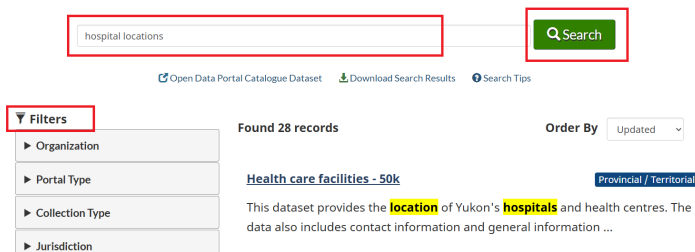
To search for open data, you can use the search bar on the Open Government Portal Canada homepage. For example, if you are interested in finding data on “hospital locations”, you can enter that term in the search bar and click the search icon. There are many filter options to narrow your search results, such as format and organization.

Find open data and information



[Canada.ca](#) > [Open Government](#)

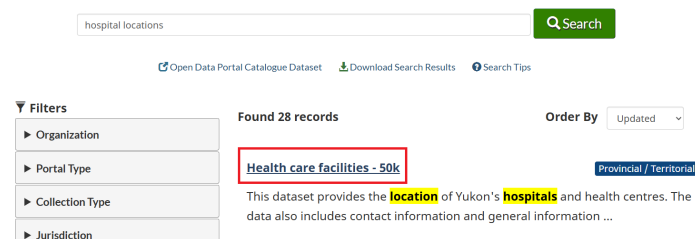
Open Government Portal



To explore a specific dataset, click its title. This will take you to a page with more information about the dataset, including its description, format options, and download links. For example, if you click on the “Health care facilities - 50k” dataset, you will find detailed information about the location of Yukon’s hospitals and health centres.

[Canada.ca](#) > [Open Government](#)

Open Government Portal



Health care facilities - 50k

Have your say

★★★★★

[Rate this dataset](#)
[Provide feedback](#)

This dataset provides the location of Yukon's hospitals and health centres. The data also includes contact information and general information about each facility.

Distributed from GeoYukon by the Government of Yukon. Discover more digital map data and interactive maps from Yukon's digital map data collection. For more information: geomatics.help@yukon.ca

Under the **Data** and **Resources** tab, you can find the available formats for downloading the location of Yukon's hospitals and health centres.

Data and Resources

View on Map

Search resources...

ArcGIS Online layers

English web service ESRI REST

Explore

ArcGIS Online layers

English web service ESRI REST

Explore

Shape files

English dataset HTML

Explore

XML metadata

English dataset HTML

Explore

Original metadata (<https://open.yukon.ca>)

English dataset HTML

Explore

Similarly, say we are interested in “opioid use” and want to find open data related to that topic. We can enter “opioid use” into the search bar and filter by **Open data**:

opioid use

Search

[Open Data Portal Catalogue Dataset](#)
[Download Search Results](#)
[Search Tips](#)

Filters

Portal Type - Open Data

Organization

Portal Type

☒ Open Data 10
 ☐ Open Information 34

Collection Type

Found 10 records

Order By Updated

Opioid agonist treatment archives

Federal

Correctional Service Canada provides Opiate Agonist Treatment to patients with an **opioid use** disorder. Archives of these quarterly web pages ...

Record Modified: Apr 9, 2026
 Record Released: Nov 28, 2025

Publisher: Correctional Service of Canada

Formats: [CSV](#) [PDF](#)

Keywords: CSC OAT OUD opiate treatment Suboxone Methadone Sublocade health drugs institutions

283

To explore the data on opioid agonist treatment, we can click the dataset title. This will take us to a page with more information about the dataset, including its description, format options, and download links.

opiod use Search

[Open Data Portal Catalogue Dataset](#) [Download Search Results](#) [Search Tips](#)

Filters

Portal Type: Open Data

Organization

Portal Type

☒ Open Data 10

☐ Open Information 34

Collection Type

Found 10 records

Order By Updated

Opioid agonist treatment archives Federal

Correctional Service Canada provides Opiate Agonist Treatment to patients with an **opiod use** disorder. Archives of these quarterly web pages ...

Record Modified: Apr 9, 2026 Record Released: Nov 28, 2025

Publisher: Correctional Service of Canada

Formats: [CSV](#) [PDF](#)

Keywords: CSC OAT OUD opiate treatment Suboxone Methadone Sublocade health drugs institutions

Data and Resources

Search resources... Search

Opioid Agonist Treatment (OAT) in CSC: July 2019 Explore

English dataset CSV

Opioid Agonist Treatment (OAT) in CSC: July 2019 Explore

French dataset CSV

Opioid Agonist Treatment: December 2019 Explore

English dataset CSV

Opioid Agonist Treatment: December 2019 Explore

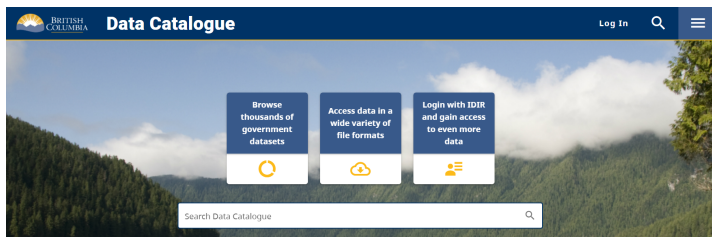
French dataset CSV

Opioid Agonist Treatment: March 2020 Explore

English dataset CSV

Data Source: BC Data Catalogue

The British Columbia Data Catalogue is a central hub provided by the Government of British Columbia (BC). Its primary purpose is to provide public access to thousands of datasets managed by the provincial government and its partners. Essentially, it is a digital library designed to make government information transparent and usable for citizens, researchers, and developers.

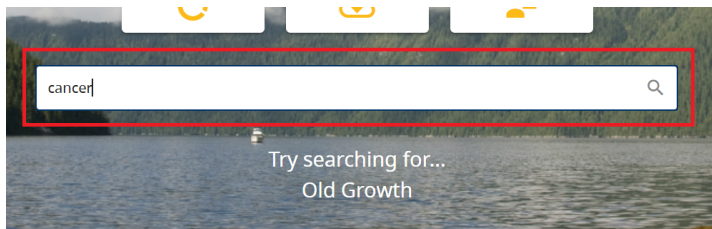


[British Columbia Data Catalogue](#)

Search for Data

To search for data, you can use the search bar on the BC Data Catalogue homepage. For example, if you are interested in finding data on “cancer”, you can enter that term in the search bar and click the search icon.

[Search Data Catalogue](#)



BRITISH COLUMBIA Data Catalogue Log In

Search: cancer

1 dataset found matching "cancer" Order By: Relevance

Metadata for BC Cancer Registry
 Jan 1985 to Dec 2022 the BC Cancer Registry is a population-based dataset of all new reportable cancers diagnosed among BC residents. The Registry contains diagnosis information, as well as personal and demographic information...
 Data Innovation Program (DIP)
 First Published: 2025-01-14
 Data Types: csv

Filter Search Results
 Expand All | Collapse All | Reset
 Licence
 Open Government Licence - British Columbia
 Statistics Canada Open Licence Access Only
 Open Government Licence - Canada
 King's Printer Licence - British Columbia

Similarly, say we are interested in “COVID-19” and want to find open data related to that topic. We can enter “covid” into the search bar:

Search: covid

8 datasets found matching "covid" Order By: Relevance

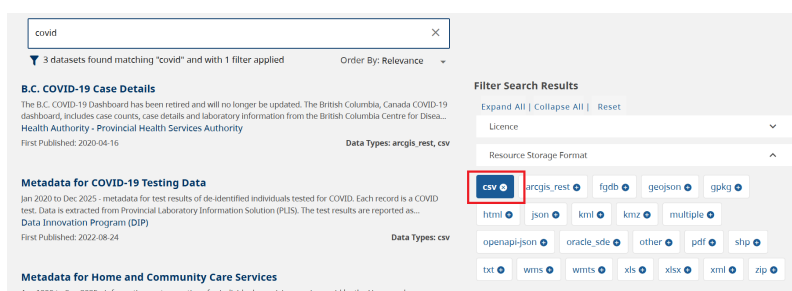
British Columbia COVID-19 Dashboard
 The B.C. COVID-19 Dashboard has been retired and will no longer be updated. The British Columbia, Canada COVID-19 dashboard, includes case counts, case details and laboratory information from the British Columbia Centre for Disease...
 Health Authority - Provincial Health Services Authority
 First Published: 2020-04-15
 Data Types: other

B.C. COVID-19 Case Details
 The B.C. COVID-19 Dashboard has been retired and will no longer be updated. The British Columbia, Canada COVID-19 dashboard, includes case counts, case details and laboratory information from the British Columbia Centre for Disease...
 Health Authority - Provincial Health Services Authority
 First Published: 2020-04-16
 Data Types: arcgis_rest, csv

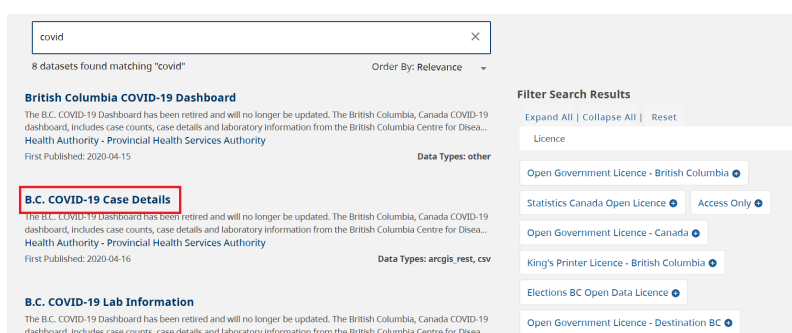
B.C. COVID-19 Lab Information
 The B.C. COVID-19 Dashboard has been retired and will no longer be updated. The British Columbia, Canada COVID-19 dashboard, includes case counts, case details and laboratory information from the British Columbia Centre for Disease...

Filter Search Results
 Expand All | Collapse All | Reset
 Licence
 Open Government Licence - British Columbia
 Statistics Canada Open Licence Access Only
 Open Government Licence - Canada
 King's Printer Licence - British Columbia
 Elections BC Open Data Licence
 Open Government Licence - Destination BC

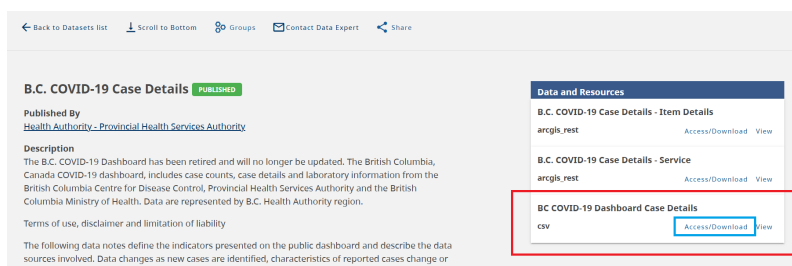
You can filter your search by license, format, group, and so on:



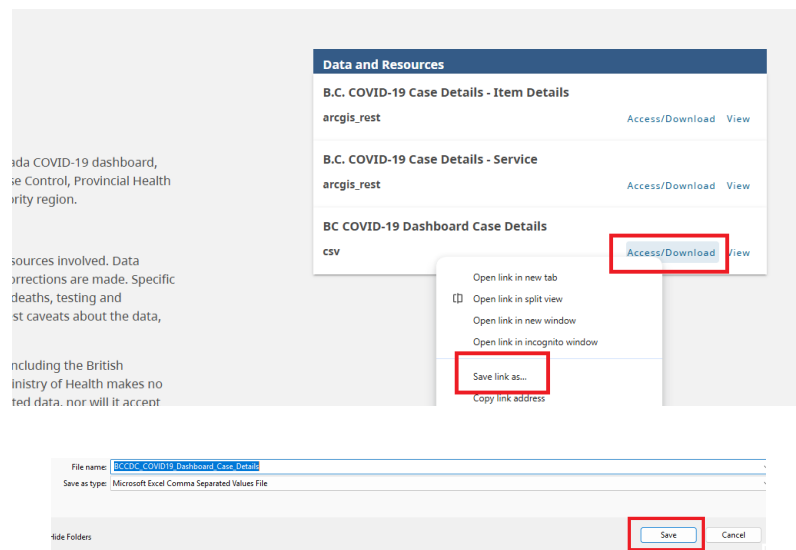
Let's explore the "COVID-19 Case Details" dataset. To do that, we can click the dataset title. This will take us to a page with more information about the dataset, including its description and related links.



You will find information on COVID-19 case details as well as a link to download the data:



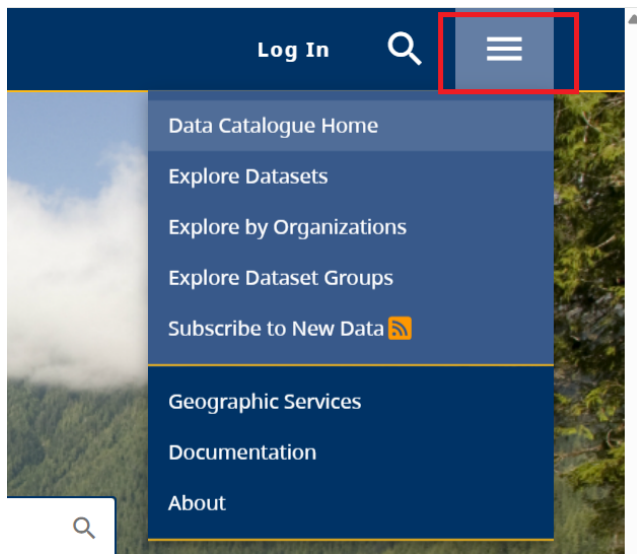
To download the dataset, right-click on the Access/Download link and select Save link as.... This will allow you to save the dataset to your computer.



After downloading, you can open the data in Excel, Google Sheets, or any statistical software. The data is structured in a tabular format, with each row representing an individual case and each column representing different variables.

1	Reported_Date	HA	Sex	Age_Group	Classification_Reported
2	29-01-20	Out of Canada	M	40-49	Lab-diagnosed
3	06-02-20	Vancouver Coastal	F	50-59	Lab-diagnosed
4	10-02-20	Out of Canada	M	30-39	Lab-diagnosed
5	10-02-20	Out of Canada	F	20-29	Lab-diagnosed
6	18-02-20	Interior	F	30-39	Lab-diagnosed
7	24-02-20	Fraser	F	30-39	Lab-diagnosed
8	24-02-20	Fraser	M	40-49	Lab-diagnosed
9	03-03-20	Fraser	M	50-59	Lab-diagnosed
10	03-03-20	Vancouver Coastal	F	60-69	Lab-diagnosed
11	03-03-20	Vancouver Coastal	F	30-39	Lab-diagnosed
12	03-03-20	Vancouver Coastal	F	80-89	Lab-diagnosed
13	03-03-20	Vancouver Coastal	M	60-69	Lab-diagnosed
14	05-03-20	Out of Canada	F	50-59	Lab-diagnosed
15	05-03-20	Vancouver Coastal	M	60-69	Lab-diagnosed
16	05-03-20	Fraser	F	50-59	Lab-diagnosed
17	05-03-20	Vancouver Coastal	M	30-39	Lab-diagnosed

To learn more about BC Data Catalogue, click on the three bars on the right side of the page:



Part XVII

Assistant Playbook

Assistant Playbook

This playbook is for instructors, TAs, and AI assistants helping maintain the HDSx course materials.

Operating Principles

- Treat the syllabus as the source of truth for weeks, deliverables, and deadlines.
- Keep student-facing content beginner-accessible and cloud-first.
- Prefer reproducible Quarto, R, Python, and GitHub Codespaces workflows over local-machine assumptions.
- Keep generated output separate from source content.
- Add content where the normalized structure says it belongs: weeks, assignments, milestones, references, examples, or assets.

Editing Rules

- Update `_quarto.yml` whenever a page should appear in the book.
- Keep paths relative and test them after moving files.
- Use `assets/` for shared images and data that support teaching pages.
- Use `examples/` for runnable code and sample datasets.
- Do not bury assignment requirements only inside the syllabus; add a student-facing assignment or milestone brief.

Reproducibility Expectations

- Every activity should state expected inputs, outputs, and submission location.
- Every code activity should run in a fresh GitHub Codespace.
- Every dataset activity should record source, license, access date, and limitations.
- Every AI-assisted activity should include an audit note: what AI helped with, what changed, and how the result was verified.

Review Cadence

- Before term: verify links, screenshots, Codespace setup, dependencies, and all assignment briefs.
- Weekly: update troubleshooting notes based on student issues.
- Before each milestone: test the milestone instructions from a fresh repository.
- After term: archive changes, refresh screenshots, and update the completion matrix.

Definition-of-Done Checklists

Course Page Done

- The page appears in `_quarto.yml` if it is student-facing.
- The title matches the week or reference purpose.
- Learning goals, activity steps, and expected outputs are clear.
- Links and image paths render from the Quarto project root.
- TODO blocks are either resolved or clearly scoped.

Assignment Done

- The assignment has a student-facing brief.
- Required files and submission location are explicit.
- The work can be completed in GitHub Codespaces.
- Packages, data, and outputs are documented.
- The grading checklist matches the syllabus assessment logic.

Milestone Done

- The milestone states the exact artifact students submit.
- The milestone maps to the final portfolio.
- The rubric includes reproducibility, auditability, data stewardship, and KT value.
- Peer or TA review instructions are actionable.

Reproducibility Done

- A fresh Codespace can render or run the project.
- Paths are relative.
- Dependencies are declared with `renv`, `requirements.txt`, or a clearly documented equivalent.
- No required data exists only on one person's local computer.
- Randomness uses a seed when results depend on it.
- The README explains how to reproduce the output.

AI Audit Done

- The AI-use note says what tool was used and for what task.
- Generated code was checked against real column names and expected outputs.
- Generated summaries are grounded in actual results.
- Security, privacy, and bias risks are considered.
- The final text does not cite sources invented by AI.